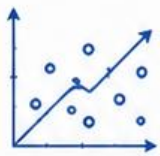


Machine Learning with Python

COMPREHENSIVE LEARNING NOTES



Regression



Classification



Clustering



Model Evaluation

“ *Transforming Data into
Intelligent Decisions* ”



KASHIF MAQBOOL JOIYA
Data Scientist | AI Engineer



GitHub
<https://github.com/KashifMaqbool>



Table of Content

1	MACHINE LEARNING TECHNIQUES AND APPLICATIONS	1
1.1	INTRODUCTION	1
1.2	ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING	1
1.3	WHAT IS MACHINE LEARNING	1
1.4	TYPES OF MACHINE LEARNING	1
1.4.1	SUPERVISED LEARNING	1
1.4.2	UNSUPERVISED LEARNING	2
1.4.3	SEMI-SUPERVISED LEARNING	2
1.4.4	REINFORCEMENT LEARNING	2
1.4.5	CHOOSING A MACHINE LEARNING TECHNIQUE	2
1.4.6	MACHINE LEARNING TECHNIQUES	2
1.4.7	CLASSIFICATION	2
1.4.8	REGRESSION	2
1.4.9	CLUSTERING	2
1.4.10	ASSOCIATION	3
1.4.11	ANOMALY DETECTION	3
1.4.12	SEQUENCE MINING	3
1.4.13	DIMENSION REDUCTION	3
1.4.14	RECOMMENDATION SYSTEMS	3
1.5	SUPERVISED LEARNING TECHNIQUES	3
1.5.1	CLASSIFICATION VS REGRESSION	3
1.6	UNSUPERVISED LEARNING TECHNIQUE	4
1.6.1	CLUSTERING	4
1.7	REAL-WORLD APPLICATIONS OF MACHINE LEARNING	4
1.7.1	HEALTHCARE (CANCER DETECTION)	4
1.7.2	E-COMMERCE AND ENTERTAINMENT	4
1.7.3	BANKING AND FINANCE	4
1.7.4	TELECOMMUNICATIONS	5
1.7.5	COMPUTER VISION (IMAGE RECOGNITION)	5
1.8	ROLE OF HUMANS IN MACHINE LEARNING	5
1.9	EVERYDAY APPLICATIONS OF MACHINE LEARNING	5
2	MACHINE LEARNING MODEL LIFECYCLE	5
2.1	WELCOME TO MACHINE LEARNING MODEL LIFECYCLE	5
2.2	INTRODUCTION	6
2.3	MACHINE LEARNING MODEL LIFECYCLE OVERVIEW	6
2.3.1	PROBLEM DEFINITION	6
2.3.2	DATA COLLECTION	6
2.3.3	DATA PREPARATION	6
2.3.4	MODEL DEVELOPMENT AND EVALUATION	6
2.3.5	MODEL DEPLOYMENT	6
2.4	ITERATIVE NATURE OF THE LIFECYCLE	6
2.5	ETL PROCESS (EXTRACT, TRANSFORM, LOAD)	7

3	A DAY IN THE LIFE OF A MACHINE LEARNING ENGINEER	7
3.1	MACHINE LEARNING MODEL LIFECYCLE IN A PROJECT	7
3.2	PROBLEM DEFINITION / BUSINESS UNDERSTANDING	7
3.3	DATA COLLECTION	7
3.4	DATA PREPARATION AND TRANSFORMATION	8
3.5	MODEL DEVELOPMENT	8
3.5.1	CONTENT-BASED FILTERING	9
3.5.2	COLLABORATIVE FILTERING	9
3.6	MODEL EVALUATION	9
3.7	DEPLOYMENT AND MONITORING	10
3.8	CONTINUOUS IMPROVEMENT	10
3.9	KEY TAKEAWAYS	10
4	INTRODUCTION TO REGRESSION	10
4.1	WHAT IS REGRESSION?	10
4.2	EXAMPLE: CO₂ EMISSIONS DATASET	10
4.3	BUILDING A REGRESSION MODEL	11
4.4	TYPES OF REGRESSION	11
4.4.1	SIMPLE REGRESSION	11
4.4.2	MULTIPLE REGRESSION	12
4.5	APPLICATIONS OF REGRESSION	12
4.5.1	BUSINESS AND ECONOMICS	12
4.5.2	REAL ESTATE	12
4.5.3	INDUSTRIAL AND MAINTENANCE	12
4.5.4	INCOME PREDICTION	12
4.5.5	ENVIRONMENTAL APPLICATIONS	13
4.5.6	PUBLIC HEALTH	13
4.6	REGRESSION ALGORITHMS	13
4.6.1	CLASSICAL METHODS	13
4.6.2	MODERN MACHINE LEARNING METHODS	13
4.7	SUMMARY	13
5	INTRODUCTION TO SIMPLE LINEAR REGRESSION	14
5.1	OVERVIEW	14
5.2	DATASET EXAMPLE: CO₂ EMISSIONS	14
5.3	CONCEPT OF SIMPLE LINEAR REGRESSION	14
5.4	VISUAL REPRESENTATION	14
5.5	BEST-FIT LINE	15
5.6	PREDICTION EXAMPLE	15
5.7	MATHEMATICAL MODEL	15
5.8	COEFFICIENTS OF THE MODEL	15
5.9	RESIDUAL ERROR	15
5.10	MEAN SQUARED ERROR (MSE)	16
5.11	ORDINARY LEAST SQUARES (OLS)	16
5.12	ADVANTAGES OF OLS REGRESSION	16
5.13	LIMITATIONS OF LINEAR REGRESSION	16
5.14	SUMMARY	16

6	INTRODUCTION TO MULTIPLE LINEAR REGRESSION	16
6.1	OVERVIEW	16
6.2	MATHEMATICAL REPRESENTATION	17
6.3	DATASET EXAMPLE: CO ₂ EMISSIONS	17
6.4	PREDICTION EXAMPLE	17
6.5	MODEL INTERPRETATION	18
6.6	RESIDUAL ERROR	18
6.7	MEAN SQUARED ERROR (MSE)	18
6.8	LEAST SQUARES METHOD	18
6.9	METHODS TO ESTIMATE PARAMETERS	19
6.9.1	ORDINARY LEAST SQUARES (OLS)	19
6.9.2	OPTIMIZATION METHODS	19
6.10	GRADIENT DESCENT	19
6.11	APPLICATIONS OF MULTIPLE LINEAR REGRESSION	19
6.12	WHAT-IF SCENARIOS	19
6.12.1	LIMITATIONS OF WHAT-IF ANALYSIS	19
6.13	MULTICOLLINEARITY (PITFALL)	19
6.13.1	SOLUTION:	20
6.14	OVERFITTING PROBLEM	20
6.15	KEY SUMMARY	20
7	INTRODUCTION TO POLYNOMIAL AND NONLINEAR REGRESSION	20
7.1	OVERVIEW	20
7.2	NONLINEAR REGRESSION	20
7.3	LINEAR VS NONLINEAR RELATIONSHIP	21
7.4	POLYNOMIAL REGRESSION	21
7.4.1	GENERAL IDEA	21
7.5	POLYNOMIAL REGRESSION EXAMPLE	21
7.6	OVERFITTING IN POLYNOMIAL REGRESSION	22
7.7	NONLINEAR REGRESSION BEYOND POLYNOMIALS	22
7.7.1	EXPONENTIAL GROWTH	22
7.7.2	LOGARITHMIC RELATIONSHIPS	23
7.7.3	PERIODIC (SINUSOIDAL) PATTERNS	23
7.8	REAL-WORLD EXAMPLE: GDP GROWTH	23
7.9	DIMINISHING RETURNS EXAMPLE	23
7.10	SELECTING A REGRESSION MODEL	23
7.11	OPTIMIZATION METHODS	24
7.12	SUMMARY	24
8	INTRODUCTION TO LOGISTIC REGRESSION	24
8.1	OVERVIEW	24
8.2	CONCEPT OF LOGISTIC REGRESSION	24
8.3	WHEN TO USE LOGISTIC REGRESSION	25
8.4	REAL-WORLD APPLICATIONS	25
8.5	DATASET EXAMPLE: CUSTOMER CHURN	25
8.6	MODEL REPRESENTATION	26
8.7	PROBLEM WITH LINEAR REGRESSION	26

8.8	SIGMOID FUNCTION	26
8.9	LOGISTIC REGRESSION MODEL	27
8.10	DECISION BOUNDARY	27
8.11	PROBABILITY INTERPRETATION	27
8.12	SUMMARY	28
9	TRAINING A LOGISTIC REGRESSION MODEL	28
9.1	OVERVIEW	28
9.2	OBJECTIVE OF LOGISTIC REGRESSION TRAINING	28
9.3	STEPS IN TRAINING A LOGISTIC REGRESSION MODEL	28
9.3.1	INITIALIZE PARAMETERS	28
9.3.2	PREDICT PROBABILITIES	28
9.3.3	COMPUTE ERROR (COST FUNCTION)	28
9.3.4	UPDATE PARAMETERS	28
9.3.5	REPEAT UNTIL CONVERGENCE	29
9.4	OPTIMAL LOGISTIC REGRESSION MODEL	29
9.5	COST FUNCTION: LOG LOSS	29
9.5.1	KEY CHARACTERISTICS OF LOG LOSS	29
9.5.2	BEHAVIOR OF LOG LOSS	29
9.6	OPTIMIZATION USING GRADIENT DESCENT	30
9.6.1	WORKING PRINCIPLE	30
9.6.2	LEARNING RATE	30
9.6.3	KEY IDEA	30
9.7	FEATURES OF GRADIENT DESCENT	31
9.8	STOCHASTIC GRADIENT DESCENT (SGD)	31
9.8.1	KEY CHARACTERISTICS	31
9.8.2	ADVANTAGES	31
9.8.3	LIMITATIONS	31
9.8.4	IMPROVING CONVERGENCE	31
9.9	STOPPING CRITERIA	31
9.10	SUMMARY	31
10	MODULE 2 SUMMARY AND HIGHLIGHTS	32
10.1	UNDERSTANDING REGRESSION MODELS	32
10.2	REAL-WORLD APPLICATIONS	32
10.3	SIMPLE LINEAR REGRESSION AND OLS	32
10.4	MULTIPLE LINEAR REGRESSION AND MODEL BALANCING	32
10.5	NONLINEAR AND POLYNOMIAL REGRESSION	33
10.6	LOGISTIC REGRESSION FOR CLASSIFICATION	33
10.7	OPTIMIZATION VIA GRADIENT DESCENT	33
11	CLASSIFICATION IN MACHINE LEARNING	33
11.1	OVERVIEW	33
11.2	APPLICATIONS OF CLASSIFICATION	33
11.2.1	BUSINESS USE CASES	34
11.3	EXAMPLE: LOAN DEFAULT PREDICTION	34

11.4	BINARY CLASSIFICATION	34
11.5	MULTI-CLASS CLASSIFICATION	35
11.5.1	EXAMPLE: DRUG PRESCRIPTION	35
11.6	CLASSIFICATION ALGORITHMS	35
11.7	EXTENDING BINARY CLASSIFIERS TO MULTI-CLASS	35
11.7.1	ONE-VERSUS-ALL (OVA) STRATEGY	36
11.7.2	ONE-VERSUS-ONE (OVO) STRATEGY	36
11.8	ONE-VS-ALL (OVA) VS ONE-VS-ONE (OVO) EXAMPLE	37
11.8.1	PROBLEM SETUP	37
11.8.2	DATASET DESCRIPTION	37
11.8.3	DATA SPLIT	38
11.8.4	ONE-VS-ALL (OVA)	38
11.8.5	ONE-VS-ONE (OVO)	39
11.8.6	FINAL COMPARISON	41
11.8.7	DEEP INSIGHT	41
11.8.8	WHEN TO USE ONE-VS-ALL (OVA)	41
11.8.9	WHEN TO USE ONE-VS-ONE (OVO)	42
11.8.10	REAL-WORLD MAPPING	42
11.8.11	PRACTICAL DECISION RULE	43
11.9	SUMMARY	43
12	DECISION TREES FOR MACHINE LEARNING	43
12.1	INTRODUCTION	43
12.2	WHAT IS A DECISION TREE?	43
12.3	EXAMPLE DATASET: MEDICAL PREDICTION	44
12.4	HOW DECISION TREES WORK	44
12.5	BUILDING A DECISION TREE	45
12.6	STOPPING CRITERIA (TREE PRUNING)	45
12.7	WHY PRUNING IS IMPORTANT	45
12.8	FEATURE SELECTION AND SPLITTING	45
12.9	EXAMPLE OF FEATURE SPLITTING	46
12.9.1	CHOLESTEROL FEATURE	46
12.9.2	GENDER FEATURE	46
12.10	ENTROPY (MEASURE OF IMPURITY)	46
12.10.1	HOMOGENEOUS (PURE NODE)	47
12.11	INFORMATION GAIN	47
12.12	GINI IMPURITY	48
12.13	HOW DECISION TREES LEARN	48
12.14	ADVANTAGES OF DECISION TREES	48
12.15	SUMMARY	48
13	REGRESSION TREES	49
13.1	INTRODUCTION	49
13.2	WHAT IS A REGRESSION TREE?	49
13.3	CLASSIFICATION TREE VS REGRESSION TREE	49
13.3.1	TARGET VARIABLE	49
13.3.2	PURPOSE	49
13.3.3	LEAF NODE PREDICTION	49

13.4	USE CASES	49
13.4.1	CLASSIFICATION TREES	49
13.4.2	REGRESSION TREES	50
13.5	HOW REGRESSION TREES WORK	50
13.6	SPLITTING PROCESS	50
13.7	PREDICTION AT LEAF NODES	50
13.8	SPLIT QUALITY MEASUREMENT	51
13.9	MEAN SQUARED ERROR CONCEPT	51
13.10	WEIGHTED MSE FOR SPLITS	51
13.11	CHOOSING THE BEST SPLIT	51
13.12	HANDLING MULTI-CLASS FEATURES	52
13.13	SELECTING THRESHOLDS FOR CONTINUOUS FEATURES	52
13.14	EFFICIENCY CONSIDERATIONS	52
13.15	SUMMARY	52
14	SUPERVISED LEARNING WITH SUPPORT VECTOR MACHINES (SVM)	53
14.1	OVERVIEW	53
14.2	CONCEPT OF SUPPORT VECTOR MACHINES	53
14.3	HYPERPLANE AND CLASSIFICATION	53
14.4	MARGIN AND SUPPORT VECTORS	53
14.5	SOFT MARGIN AND REGULARIZATION	54
14.6	BINARY CLASSIFICATION IN SVM	54
14.7	MATHEMATICAL REPRESENTATION	54
14.8	DECISION BOUNDARY	55
14.9	NON-LINEAR DATA AND KERNEL TRICK	55
14.10	KERNEL FUNCTIONS IN SVM	56
14.11	SUPPORT VECTOR REGRESSION (SVR)	56
14.12	ADVANTAGES OF SVM	57
14.13	LIMITATIONS OF SVM	58
14.14	APPLICATIONS OF SVM	58
14.15	SUMMARY	58
15	SUPERVISED LEARNING WITH K-NEAREST NEIGHBORS (KNN)	58
15.1	OVERVIEW	58
15.2	DEFINITION OF KNN	58
15.3	CONCEPT OF NEIGHBORS	58
15.4	WORKING OF THE KNN ALGORITHM (CLASSIFICATION)	59
15.4.1	STEP 1: CHOOSE THE VALUE OF K	59
15.4.2	STEP 2: CALCULATE DISTANCE	59
15.4.3	STEP 3: FIND NEAREST NEIGHBORS	59
15.4.4	STEP 4: MAJORITY VOTING	59
15.5	KNN FOR REGRESSION	59
15.6	DISTANCE AND SIMILARITY	60
15.7	CLASSIFICATION EXAMPLE	60
15.8	DECISION BOUNDARY	61
15.9	MODEL ACCURACY	62
15.10	CHOOSING THE OPTIMAL VALUE OF K	62
15.11	NATURE OF KNN (LAZY LEARNING)	63

15.12	EFFECT OF K ON MODEL PERFORMANCE	63
15.12.1	SMALL K (LOW VALUE)	63
15.12.2	LARGE K (HIGH VALUE)	63
15.12.3	OPTIMAL K	63
15.13	CHALLENGES IN KNN	64
15.13.1	IMBALANCED CLASSES	64
15.13.2	FEATURE SCALING ISSUE	64
15.13.3	IRRELEVANT FEATURES	64
15.13.4	REDUNDANT FEATURES(HIGHLY CORRELATED WITH ANOTHER FEATURE)	64
15.14	FEATURE SELECTION	64
15.15	SUMMARY	65
16	BIAS, VARIANCE, AND ENSEMBLE MODELS	65
16.1	OVERVIEW	65
16.2	UNDERSTANDING BIAS AND VARIANCE (DARTBOARD ANALOGY)	65
16.3	PREDICTION BIAS	66
16.4	PREDICTION VARIANCE	67
16.5	BIAS-VARIANCE TRADEOFF	67
16.5.1	UNDERFITTING	67
16.5.2	OVERFITTING	67
16.5.3	OPTIMAL MODEL	68
16.6	WEAK LEARNERS AND STRONG LEARNERS	68
16.6.1	WEAK LEARNERS	68
16.6.2	STRONG LEARNERS	68
16.7	ENSEMBLE LEARNING	68
16.8	BAGGING (BOOTSTRAP AGGREGATING)	69
16.8.1	CONCEPT	69
16.8.2	KEY CHARACTERISTICS	69
16.8.3	EXAMPLE: RANDOM FOREST	69
16.9	BOOSTING	69
16.9.1	CONCEPT	69
16.9.2	KEY CHARACTERISTICS	69
16.9.3	WORKING MECHANISM	69
16.9.4	POPULAR BOOSTING ALGORITHMS	70
16.10	BAGGING VS BOOSTING	70
16.10.1	BAGGING	70
16.10.2	BOOSTING	70
16.11	MANAGING BIAS-VARIANCE TRADEOFF	70
16.12	SUMMARY	70
17	MODULE 3 SUMMARY AND HIGHLIGHTS	71
17.1	CLASSIFICATION IN MACHINE LEARNING	71
17.1.1	APPLICATIONS OF CLASSIFICATION	71
17.2	TYPES OF CLASSIFICATION	71
17.2.1	BINARY CLASSIFICATION	71
17.2.2	MULTICLASS CLASSIFICATION	71
17.3	DECISION TREES	71
17.3.1	TRAINING DECISION TREES	71

17.3.2	SPLIT EVALUATION METRICS	71
17.4	REGRESSION TREES	72
17.4.1	WORKING OF REGRESSION TREES	72
17.4.2	EVALUATION METRIC	72
17.5	K-NEAREST NEIGHBORS (K-NN)	72
17.5.1	WORKING OF K-NN	72
17.5.2	OPTIMIZATION OF K-NN	72
17.6	SUPPORT VECTOR MACHINES (SVM)	72
17.6.1	CHARACTERISTICS OF SVM	72
17.7	BIAS-VARIANCE TRADEOFF	72
17.7.1	TECHNIQUES TO MANAGE BIAS AND VARIANCE	73
17.8	RANDOM FORESTS	73
17.8.1	ADVANTAGES OF RANDOM FORESTS	73
17.9	SUMMARY	73
18	CLUSTERING STRATEGIES IN REAL-WORLD APPLICATIONS	73
18.1	OVERVIEW	73
18.2	CLUSTERING VS CLASSIFICATION	74
18.3	DATASET EXAMPLE	74
18.4	APPLICATIONS OF CLUSTERING	74
18.4.1	EXPLORATORY DATA ANALYSIS	74
18.4.2	PATTERN RECOGNITION	75
18.4.3	ANOMALY DETECTION	75
18.4.4	FEATURE ENGINEERING	75
18.4.5	DATA SUMMARIZATION	75
18.4.6	DATA COMPRESSION	75
18.4.7	FEATURE IDENTIFICATION	75
18.5	TYPES OF CLUSTERING METHODS	75
18.5.1	PARTITION-BASED CLUSTERING	75
18.5.2	DENSITY-BASED CLUSTERING	76
18.6	VISUALIZATION AND EXAMPLES	76
18.6.1	PARTITION-BASED VS DENSITY-BASED CLUSTERING	76
18.7	HIERARCHICAL CLUSTERING	77
18.7.1	REAL-WORLD EXAMPLE: GENETIC DATA	77
18.8	HIERARCHICAL CLUSTERING STRATEGIES	78
18.8.1	AGGLOMERATIVE CLUSTERING (BOTTOM-UP APPROACH)	78
18.8.2	DIVISIVE CLUSTERING (TOP-DOWN APPROACH)	80
18.9	SUMMARY	81
19	K-MEANS CLUSTERING	81
19.1	OVERVIEW	81
19.2	DEFINITION OF K-MEANS	81
19.3	CONCEPT OF CENTROID	81
19.4	EFFECT OF K VALUE	81
19.5	K-MEANS ALGORITHM STEPS	82
19.5.1	INITIALIZATION	82
19.5.2	ASSIGNMENT STEP	82
19.5.3	UPDATE STEP	82

19.5.4	ITERATION	82
19.5.5	CONVERGENCE	82
19.6	WORKING EXAMPLE OF ITERATIONS	82
19.7	PERFORMANCE ON BALANCED VS IMBALANCED DATA	83
19.7.1	BALANCED CLUSTERS	83
19.7.2	IMBALANCED CLUSTERS	83
19.8	ASSUMPTIONS OF K-MEANS	83
19.9	LIMITATIONS OF K-MEANS	83
19.10	OBJECTIVE FUNCTION OF K-MEANS / K-MEANS OPTIMIZATION	84
19.11	EXPERIMENTAL OBSERVATIONS	84
19.11.1	EFFECT OF STANDARD DEVIATION	84
19.11.2	EFFECT OF INCORRECT K VALUE	85
19.12	CHOOSING THE OPTIMAL VALUE OF K	86
19.12.1	DATA SEPARABILITY	86
19.12.2	HEURISTIC METHODS	86
19.13	ADVANTAGES OF K-MEANS	86
19.14	SUMMARY	87
20	DBSCAN AND HDBSCAN CLUSTERING	87
20.1	OVERVIEW	87
20.2	DBSCAN CLUSTERING	87
20.2.1	DEFINITION	87
20.2.2	KEY CONCEPTS	87
20.2.3	CAPABILITIES	87
20.2.4	WHEN TO USE	87
20.3	COMPARISON WITH CENTROID-BASED CLUSTERING	88
20.3.1	CENTROID-BASED ALGORITHMS	88
20.3.2	LIMITATIONS	88
20.3.3	ADVANTAGE OF DENSITY-BASED CLUSTERING	88
20.4	How DBSCAN Works	88
20.4.1	PARAMETERS	88
20.4.2	POINT CLASSIFICATION	89
20.4.3	CLUSTER FORMATION	89
20.4.4	IMPORTANT PROPERTY	90
20.5	DBSCAN EXAMPLE EXPLANATION	90
20.5.1	DATASET	90
20.5.2	OBSERVATIONS	90
20.5.3	CLUSTERING PROCESS	90
20.5.4	RESULT	91
20.6	HDBSCAN CLUSTERING	91
20.6.1	DEFINITION	91
20.6.2	KEY FEATURES	91
20.7	CLUSTER STABILITY	91
20.8	How HDBSCAN Works	91
20.8.1	PROCESS	91
20.8.2	NATURE OF ALGORITHM	92
20.9	DBSCAN vs HDBSCAN	92
20.9.1	DBSCAN	92
20.9.2	HDBSCAN	92
20.10	REAL-WORLD EXAMPLE	92

20.10.1	DATASET	92
20.10.2	DBSCAN RESULT	92
20.10.3	HDBSCAN RESULT	93
20.10.4	OBSERVATION	93
20.11	ADVANTAGES OF HDBSCAN	93
20.12	SUMMARY	93
21	CLUSTERING, DIMENSION REDUCTION, AND FEATURE ENGINEERING	94
21.1	OVERVIEW	94
21.2	RELATIONSHIP BETWEEN CLUSTERING, DIMENSION REDUCTION, AND FEATURE ENGINEERING	94
21.2.1	CLUSTERING	94
21.2.2	DIMENSION REDUCTION	94
21.2.3	FEATURE ENGINEERING	94
21.3	DIMENSION REDUCTION	95
21.3.1	PURPOSE OF DIMENSION REDUCTION	95
21.3.2	HIGH-DIMENSIONAL DATA CHALLENGES	95
21.3.3	DIMENSION REDUCTION TECHNIQUES	96
21.4	PCA AND FACE RECOGNITION	96
21.4.1	EIGENFACES FOR FACE RECOGNITION	96
21.4.2	BENEFITS	96
21.5	VISUALIZATION OF HIGH-DIMENSIONAL CLUSTERS	97
21.5.1	PROBLEM	97
21.5.2	SOLUTION	97
21.5.3	ADVANTAGES	97
21.6	CLUSTERING FOR FEATURE SELECTION	98
21.6.1	CLUSTERING FEATURES	98
21.6.2	FEATURE ENGINEERING DECISIONS	98
21.7	K-MEANS FEATURE CLUSTERING EXAMPLE	98
21.7.1	DATASET SIMULATION	98
21.7.2	OBSERVATIONS	98
21.7.3	CLUSTERING RESULTS	98
21.8	SUMMARY	99
22	DIMENSION REDUCTION ALGORITHMS	100
22.1	OVERVIEW	100
22.2	PRINCIPAL COMPONENT ANALYSIS (PCA)	100
22.2.1	DEFINITION	100
22.2.2	PURPOSE OF PCA	100
22.2.3	WORKING OF PCA	100
22.2.4	PRINCIPAL COMPONENTS	101
22.3	T-DISTRIBUTED STOCHASTIC NEIGHBOR EMBEDDING (T-SNE)	101
22.3.1	DEFINITION	101
22.3.2	CHARACTERISTICS OF T-SNE	101
22.3.3	WORKING OF T-SNE	101
22.3.4	LIMITATIONS OF T-SNE	101
22.4	UNIFORM MANIFOLD APPROXIMATION AND PROJECTION (UMAP)	101
22.4.1	DEFINITION	101
22.4.2	WORKING OF UMAP	101

22.4.3	ADVANTAGES OF UMAP	102
22.5	COMPARISON OF PCA, T-SNE, AND UMAP	102
22.5.1	SIMULATED DATASET EXAMPLE	102
22.6	PCA RESULTS	102
22.6.1	PERFORMANCE	102
22.6.2	REASON	103
22.7	T-SNE RESULTS	103
22.7.1	PERFORMANCE	103
22.7.2	OBSERVATIONS	103
22.8	UMAP RESULTS	104
22.8.1	PERFORMANCE	104
22.8.2	OBSERVATIONS	104
22.9	KEY DIFFERENCES BETWEEN PCA, T-SNE, AND UMAP	105
22.10	SUMMARY	105
22.10.1	WHAT IS A “MANIFOLD STRUCTURE”?	105
23	MODULE 4 SUMMARY AND HIGHLIGHTS	106
23.1	FUNDAMENTALS OF CLUSTERING	106
23.2	K-MEANS CLUSTERING AND PERFORMANCE ASSESSMENT	106
23.2.1	EVALUATION TECHNIQUES FOR K-MEANS	107
23.3	DENSITY-BASED CLUSTERING ALGORITHMS	107
23.3.1	DBSCAN	107
23.3.2	HDBSCAN	107
23.4	VISUALIZATION: K-MEANS VS DBSCAN	107
23.5	HIERARCHICAL CLUSTERING	107
23.5.1	AGGLOMERATIVE (BOTTOM-UP)	108
23.5.2	DIVISIVE (TOP-DOWN)	108
23.6	DIMENSIONALITY REDUCTION AND PCA	108
23.6.1	PRINCIPAL COMPONENT ANALYSIS (PCA)	108
23.6.2	ROLE IN CLUSTERING	108
23.7	ADVANCED VISUALIZATION TECHNIQUES	108
23.7.1	T-SNE (T-DISTRIBUTED STOCHASTIC NEIGHBOR EMBEDDING)	108
23.7.2	UMAP (UNIFORM MANIFOLD APPROXIMATION AND PROJECTION)	108
23.8	SUMMARY	109
24	CLASSIFICATION METRICS AND EVALUATION TECHNIQUES	109
24.1	OVERVIEW	109
24.2	TRAIN-TEST-SPLIT TECHNIQUE	109
24.2.1	PURPOSE	109
24.2.2	DATASET SPLIT	109
24.3	CLASSIFICATION TASKS	110
24.4	ACCURACY	110
24.4.1	DEFINITION	110
24.4.2	FORMULA	110
24.4.3	EXAMPLE	110
24.5	CONFUSION MATRIX	110
24.5.1	DEFINITION	110
24.5.2	STRUCTURE	111

24.5.3	COMPONENTS	111
24.6	KNN CLASSIFIER EXAMPLE	111
24.6.1	IRIS DATASET	111
24.6.2	ACCURACY	112
24.6.3	VISUALIZATION	112
24.7	HEAT MAP OF CONFUSION MATRIX	112
24.7.1	PURPOSE	112
24.7.2	COLOR REPRESENTATION	112
24.7.3	INTERPRETATION	112
24.8	PRECISION	112
24.8.1	DEFINITION	112
24.8.2	FORMULA	112
24.8.3	IMPORTANCE	112
24.8.4	EXAMPLE	112
24.9	RECALL	113
24.9.1	DEFINITION	113
24.9.2	FORMULA	113
24.9.3	EXAMPLE	113
24.9.4	IMPORTANCE	113
24.9.5	EXAMPLE	113
24.10	F1 SCORE	113
24.10.1	DEFINITION	113
24.10.2	FORMULA	113
24.10.3	IMPORTANCE	113
24.10.4	EXAMPLE	114
24.11	CLASSIFICATION REPORT EXAMPLE	114
24.11.1	IRIS FLOWER CLASSES	114
24.11.2	SETOSA PERFORMANCE	114
24.11.3	WEIGHTED AVERAGE	114
24.12	SUMMARY	114
25	REGRESSION METRICS AND EVALUATION TECHNIQUES	115
25.1	OVERVIEW	115
25.2	UNDERSTANDING REGRESSION ERROR	115
25.2.1	DEFINITION OF ERROR	115
25.3	IMPORTANCE OF REGRESSION METRICS	116
25.4	IMPORTANT REGRESSION METRICS	116
25.4.1	MEAN ABSOLUTE ERROR (MAE)	116
25.4.2	MEAN SQUARED ERROR (MSE)	116
25.4.3	ROOT MEAN SQUARED ERROR (RMSE)	116
25.4.4	R-SQUARED (R^2)	116
25.5	UNDERSTANDING EXPLAINED VARIANCE	117
25.6	NEGATIVE R-SQUARED VALUES	118
25.7	VISUALIZATION IN REGRESSION EVALUATION	118
25.8	ASSUMPTION OF R-SQUARED	118
25.9	EFFECT OF DATA TRANSFORMATIONS	119
25.9.1	OBSERVATION	119
25.9.2	METRIC IMPROVEMENTS	119
25.10	SUMMARY	120

26 EVALUATING UNSUPERVISED LEARNING MODELS: HEURISTICS AND TECHNIQUES

120

26.1	OVERVIEW	120
26.2	CHALLENGES IN EVALUATING UNSUPERVISED LEARNING	120
26.2.1	STABILITY IN UNSUPERVISED LEARNING	120
26.3	APPROACHES FOR EVALUATING UNSUPERVISED LEARNING MODELS	121
26.4	CLUSTERING EVALUATION	121
26.4.1	TYPES OF CLUSTERING EVALUATION METHODS	121
26.5	INTERNAL CLUSTERING EVALUATION METRICS	121
26.5.1	SILHOUETTE SCORE	121
26.5.2	DAVIES-BOULDIN INDEX	122
26.5.3	INERTIA IN K-MEANS CLUSTERING	122
26.6	EXAMPLE: EXCELLENT CLUSTERING RESULTS	122
26.6.1	EVALUATION RESULTS	123
26.6.2	SILHOUETTE PLOT	123
26.7	EXAMPLE: MODERATE CLUSTERING RESULTS	123
26.7.1	EVALUATION RESULTS	123
26.8	EXTERNAL CLUSTERING EVALUATION METRICS	124
26.8.1	ADJUSTED RAND INDEX (ARI)	124
26.8.2	NORMALIZED MUTUAL INFORMATION (NMI)	125
26.8.3	FOWLKES-MALLOWS INDEX (FMI)	125
26.9	EVALUATING DIMENSIONALITY REDUCTION	125
26.9.1	EXPLAINED VARIANCE RATIO IN PCA	126
26.9.2	RECONSTRUCTION ERROR	127
26.9.3	NEIGHBORHOOD PRESERVATION	127
26.9.4	EXPLAINED VARIANCE PLOT	127
26.10	IMPORTANCE OF VISUALIZATION AND DOMAIN EXPERTISE	128
26.11	SUMMARY	128

27 CROSS-VALIDATION AND ADVANCED MODEL VALIDATION TECHNIQUES

129

27.1	OVERVIEW	129
27.2	MODEL VALIDATION	129
27.3	BASIC TRAIN-TEST SPLIT METHOD	129
27.3.1	TRAINING SET	129
27.3.2	TEST SET	130
27.4	HYPERPARAMETERS	130
27.5	DATA SNOOPING	130
27.5.1	PROBLEMS WITH DATA SNOOPING	130
27.6	PROPER MODEL VALIDATION STRATEGY	131
27.6.1	TRAINING SET	131
27.6.2	VALIDATION SET	131
27.6.3	TEST SET	131
27.7	CROSS-VALIDATION	131
27.7.1	CROSS-VALIDATION WORKFLOW	132
27.8	PROBLEMS WITH A SINGLE VALIDATION SET	132
27.8.1	OVERFITTING TO VALIDATION DATA	132
27.8.2	LIMITED TRAINING DATA	132
27.8.3	POOR REPRESENTATION	132

27.8.4	UNSTABLE PERFORMANCE	133
27.9	K-FOLD CROSS-VALIDATION	133
27.9.1	CONCEPT	133
27.9.2	K-FOLD PROCEDURE	133
27.10	BENEFITS OF K-FOLD CROSS-VALIDATION	134
27.10.1	BETTER DATA UTILIZATION	134
27.10.2	REDUCED OVERFITTING	134
27.10.3	MORE RELIABLE EVALUATION	134
27.10.4	IMPROVED GENERALIZATION	134
27.11	STRATIFIED CROSS-VALIDATION	134
27.11.1	PURPOSE OF STRATIFIED CROSS-VALIDATION	134
27.12	SKEWED TARGET VARIABLES IN REGRESSION	134
27.12.1	PROBLEM WITH SKEWED DATA	135
27.13	TRANSFORMING SKEWED TARGETS	135
27.13.1	BENEFITS OF TRANSFORMATIONS	135
27.14	SUMMARY	135
28	REGULARIZATION IN LINEAR REGRESSION	136
28.1	OVERVIEW	136
28.2	PURPOSE OF REGULARIZATION	136
28.3	REGULARIZED COST FUNCTION	136
28.4	LINEAR REGRESSION	137
28.5	RIDGE REGRESSION	137
28.5.1	L2 PENALTY	137
28.5.2	CHARACTERISTICS OF RIDGE REGRESSION	137
28.6	LASSO REGRESSION	137
28.6.1	L1 PENALTY	137
28.6.2	CHARACTERISTICS OF LASSO REGRESSION	137
28.7	SPARSE COEFFICIENTS	138
28.8	SIGNAL-TO-NOISE RATIO (SNR)	138
28.8.1	HIGH SNR	138
28.8.2	LOW SNR	138
28.9	REGRESSION PERFORMANCE IN SPARSE COEFFICIENTS WITH HIGH SNR	138
28.10	REGRESSION PERFORMANCE IN SPARSE COEFFICIENTS WITH LOW SNR	139
28.11	REGRESSION PERFORMANCE IN NON-SPARSE COEFFICIENTS WITH HIGH SNR	140
28.12	REGRESSION PERFORMANCE IN NON-SPARSE COEFFICIENTS WITH LOW SNR	140
28.13	MODEL COMPARISON USING TEST DATA	141
28.13.1	EXPERIMENT SETUP	141
28.13.2	RESULTS	141
28.13.3	MSE COMPARISON	141
28.14	PERFORMANCE SUMMARY	142
28.14.1	ORDINARY LINEAR REGRESSION	142
28.14.2	RIDGE REGRESSION	142
28.14.3	LASSO REGRESSION	142
28.15	KEY CONCEPTS SUMMARY	142
29	DATA LEAKAGE AND OTHER PITFALLS	143
29.1	OVERVIEW	143

29.2	UNDERSTANDING DATA LEAKAGE	143
29.2.1	HOUSE PRICE PREDICTION EXAMPLE	143
29.2.2	DEFINITION OF DATA LEAKAGE	143
29.2.3	WHY DATA LEAKAGE IS DANGEROUS	144
29.3	DATA SNOOPING	144
29.3.1	DEFINITION	144
29.3.2	EXAMPLES	144
29.4	PREVENTING DATA LEAKAGE	144
29.4.1	INDEPENDENT DATA PROCESSING	144
29.4.2	MITIGATION TECHNIQUES	144
29.5	CROSS-VALIDATION AND PIPELINES	145
29.5.1	PROPER PIPELINE USAGE	145
29.5.2	EXAMPLE MACHINE LEARNING PIPELINE	145
29.5.3	HYPERPARAMETER TUNING	146
29.5.4	IMPORTANT BEST PRACTICE	146
29.6	TIME-SERIES CROSS-VALIDATION	146
29.6.1	PROBLEM WITH RANDOM SPLITS	146
29.6.2	CORRECT APPROACH	146
29.6.3	TIME-SERIES SPLIT	146
29.6.4	BEHAVIOR OF TIME-SERIES SPLIT	146
29.7	FEATURE IMPORTANCE INTERPRETATION PITFALLS	147
29.7.1	REDUNDANT OR CORRELATED FEATURES	147
29.7.2	INCORRECT FEATURE SELECTION	147
29.7.3	SCALE SENSITIVITY	147
29.7.4	CORRELATION VS CAUSATION	147
29.7.5	IGNORING FEATURE INTERACTIONS	148
29.8	COMMON MODELING PITFALLS	148
29.8.1	POOR FEATURE ENGINEERING	148
29.8.2	INCORRECT EVALUATION METRICS	148
29.8.3	IGNORING CLASS IMBALANCE	148
29.8.4	BLIND RELIANCE ON AUTOMATION	149
29.8.5	NON-CAUSAL WHAT-IF ANALYSIS	149
29.9	SUMMARY	149
29.9.1	DATA LEAKAGE	149
29.9.2	LEAKAGE PREVENTION TECHNIQUES	149
29.9.3	FEATURE IMPORTANCE PITFALLS	149
29.9.4	ADDITIONAL MODELING PITFALLS	150
30	MODULE 5 SUMMARY AND HIGHLIGHTS	150
30.1	SUPERVISED LEARNING EVALUATION	150
30.1.1	CLASSIFICATION EVALUATION METRICS	150
30.1.2	REGRESSION EVALUATION METRICS	150
30.2	UNSUPERVISED LEARNING EVALUATION	150
30.2.1	COMMON EVALUATION METRICS	150
30.3	DIMENSIONALITY REDUCTION EVALUATION	151
30.3.1	EVALUATION METRICS	151
30.4	MODEL VALIDATION	151
30.4.1	VALIDATION TECHNIQUES	151
30.5	CROSS-VALIDATION	151
30.5.1	COMMON CROSS-VALIDATION METHODS	151

30.6	REGULARIZATION TECHNIQUES	151
30.6.1	TYPES OF REGULARIZATION	152
30.7	DATA LEAKAGE	152
30.7.1	PREVENTION OF DATA LEAKAGE	152
30.8	COMMON MODELING PITFALLS	152
30.8.1	COMMON ISSUES	152
30.9	FEATURE IMPORTANCE CONSIDERATIONS	152
30.9.1	IMPORTANT CONSIDERATIONS	152
31	MACHINE LEARNING COURSE SUMMARY AND KEY CONCEPTS	153
31.1	WHAT IS MACHINE LEARNING?	153
31.2	APPLICATIONS OF MACHINE LEARNING	153
31.2.1	HEALTHCARE	153
31.2.2	FINANCE AND BANKING	153
31.2.3	E-COMMERCE	153
31.3	TYPES OF MACHINE LEARNING METHODS	153
31.4	MACHINE LEARNING TOOLS AND PIPELINES	154
31.5	REGRESSION IN MACHINE LEARNING	154
31.5.1	SIMPLE REGRESSION	154
31.5.2	MULTIPLE REGRESSION	154
31.6	LOGISTIC REGRESSION	154
31.7	CLASSIFICATION IN MACHINE LEARNING	154
31.7.1	APPLICATIONS OF CLASSIFICATION	155
31.8	KEY MACHINE LEARNING ALGORITHMS	155
31.8.1	K-NEAREST NEIGHBORS (KNN)	155
31.8.2	SUPPORT VECTOR MACHINES (SVM)	155
31.8.3	DECISION TREES	155
31.8.4	REGRESSION TREES	155
31.9	CLUSTERING AND DIMENSIONALITY REDUCTION	156
31.9.1	CLUSTERING	156
31.9.2	DIMENSIONALITY REDUCTION	156
31.10	MODEL EVALUATION	156
31.10.1	REDUCED LEARNING EVALUATION	156
31.10.2	CLASSIFICATION METRICS	156
31.10.3	REGRESSION EVALUATION	156
31.10.4	UNSUPERVISED EVALUATION	156
31.11	MODEL VALIDATION	157
31.11.1	IMPORTANT CONCEPT: DATA LEAKAGE	157

Module 1: Introduction to Machine Learning

1 Machine Learning Techniques and Applications

1.1 Introduction

In this section, we will explore the fundamentals of machine learning, its techniques, and its real-world applications. By the end of this content, we will be able to understand what machine learning is, identify different machine learning techniques, and describe how it is applied in various domains.

1.2 Artificial Intelligence and Machine Learning

Artificial Intelligence (AI) is a broad field that enables computers to simulate human cognitive abilities and appear intelligent. It includes multiple domains such as:

- Computer Vision
- Natural Language Processing
- Generative AI
- Machine Learning
- Deep Learning

Machine Learning (ML) is a subset of AI that focuses on developing algorithms that learn from data. It typically requires feature engineering by practitioners.

Deep Learning is a specialized area within ML that uses multi-layered neural networks to automatically extract features from complex and unstructured data.

1.3 What is Machine Learning

Machine Learning teaches computers to:

- Learn from data
- Identify patterns
- Make decisions without explicit programming

ML algorithms rely on computational methods to learn directly from data instead of following fixed rules.

1.4 Types of Machine Learning

1.4.1 Supervised Learning

Supervised learning uses labeled data to train models. It learns patterns from known inputs and outputs to make predictions on new data.

1.4.2 Unsupervised Learning

Unsupervised learning works with unlabeled data. It identifies hidden patterns and structures within the data.

1.4.3 Semi-Supervised Learning

Semi-supervised learning combines a small amount of labeled data with a large amount of unlabeled data. The model iteratively improves by generating and learning from new labels.

1.4.4 Reinforcement Learning

Reinforcement learning involves an intelligent agent interacting with its environment. The model learns by receiving feedback in the form of rewards or penalties.

1.4.5 Choosing a Machine Learning Technique

The selection of a machine learning technique depends on:

- The problem to be solved
- The type of data available
- Computational resources
- Desired outcome

1.4.6 Machine Learning Techniques

1.4.7 Classification

Classification predicts categorical outcomes.

Examples:

- Identifying whether a tumor is benign or malignant
- Predicting customer churn

1.4.8 Regression

Regression predicts continuous numerical values.

Examples:

- House price prediction
- CO₂ emissions estimation

1.4.9 Clustering

Clustering groups similar data points together.

Examples:

- Customer segmentation
- Patient grouping

Noise in clustering refers to data points that do not belong to any cluster.

1.4.10 Association

Association identifies items or events that frequently occur together.

Example:

- Market basket analysis (items bought together)

1.4.11 Anomaly Detection

Anomaly detection identifies unusual or abnormal patterns.

Example:

- Credit card fraud detection

1.4.12 Sequence Mining

Sequence mining predicts the next likely event in a sequence.

Example:

- Website clickstream analysis

1.4.13 Dimension Reduction

Dimension reduction reduces the number of features in a dataset while preserving important information.

1.4.14 Recommendation Systems

Recommendation systems suggest items based on user preferences and similarities with others.

Examples:

- Movie recommendations
- Product suggestions

1.5 Supervised Learning Techniques

1.5.1 Classification vs Regression

Classification:

- Predicts discrete categories
- Assigns input data to predefined classes

Regression:

- Predicts continuous values
- Does not classify data into categories

1.6 Unsupervised Learning Technique

1.6.1 Clustering

Clustering is used to group similar data points. It helps in discovering hidden patterns in data without predefined labels.

1.7 Real-World Applications of Machine Learning

1.7.1 Healthcare (Cancer Detection)

Machine learning can analyze cell characteristics such as:

- Clump thickness
- Uniformity of cell size
- Marginal adhesion

Using historical data of cell samples, ML models can:

- Learn patterns of benign and malignant cells
- Predict whether a new cell sample is cancerous

Steps involved:

- Data collection
- Data cleaning
- Algorithm selection
- Model training
- Prediction

Early detection using ML can significantly improve patient survival rates.

1.7.2 E-Commerce and Entertainment

Platforms like Amazon and Netflix use ML to:

- Recommend products
- Suggest movies and shows

These systems analyze user preferences and behavior patterns.

1.7.3 Banking and Finance

Banks use ML to:

- Predict loan default probability
- Assist in loan approval decisions

1.7.4 Telecommunications

Telecom companies use ML to:

- Analyze customer demographics
- Predict customer churn

1.7.5 Computer Vision (Image Recognition)

Machine learning enables systems to:

- Differentiate between objects (e.g., cats vs dogs)
- Extract features such as eyes, ears, tail, and shape

Traditional rule-based systems failed because:

- They required too many rules
- They could not generalize to new data

Modern ML models:

- Learn features automatically
- Perform accurate image classification

1.8 Role of Humans in Machine Learning

Even with advanced ML systems, humans remain important:

- Reviewing decisions (e.g., loan rejection)
- Providing explanations and validation
- Improving model performance

1.9 Everyday Applications of Machine Learning

Machine learning is widely used in:

- Chatbots and virtual assistants
- Face recognition systems
- Online recommendation engines
- Computer games (e.g., chess)

2 Machine Learning Model Lifecycle

2.1 Welcome to Machine Learning Model Lifecycle

After watching this video, we will be able to list the processes within the lifecycle of a Machine Learning product.

2.2 Introduction

Hello! My name is Isioma. I work as a Machine Learning Engineer at a leading company. We sell machine learning solutions that enhance the user experience for people shopping for beauty products.

Today, I will walk us through a Machine Learning project from start to finish.

2.3 Machine Learning Model Lifecycle Overview

Let's begin by looking at the processes in the lifecycle of a Machine Learning Model.

The main stages include:

2.3.1 Problem Definition

First, we define the problem or state the situation.

2.3.2 Data Collection

Next, we move to data collection.

2.3.3 Data Preparation

After that, we perform data preparation.

2.3.4 Model Development and Evaluation

Then we proceed to model development and evaluation.

2.3.5 Model Deployment

Finally, we deploy the model.

2.4 Iterative Nature of the Lifecycle

In reality, the model lifecycle is iterative, which means we tend to move back and forth between these processes.

For example, if we find some problems with models that are deployed in production, we may need to go back to:

- Data collection, or
- Even the problem definition step

After that, we reiterate the subsequent processes.

2.5 ETL Process (Extract, Transform, Load)

Data collection and preparation is called the Extract, Transform, and Load (ETL) process.

The ETL process involves:

- Collecting data from various sources
- Cleaning the data
- Transforming the data
- Storing it in a new single place

After this, the data becomes accessible to the machine learning engineer, allowing them to perform tasks such as building a machine learning model.

3 A Day in the Life of a Machine Learning Engineer

3.1 Machine Learning Model Lifecycle in a Project

3.2 Problem Definition / Business Understanding

In this project, the goal is to increase business revenue by building and deploying a machine learning model that recommends similar products based on a customer's purchase history.

We work with the client to define the end-user problem:

“As a beauty product customer, I want to receive recommendations for other products based on my purchase history so that I can address my skincare needs and improve the overall health of my skin.”

Defining the problem clearly is very important because it ensures that the machine learning solution aligns with the client's business needs.

3.3 Data Collection

After understanding the client's requirements, the next step is to collect data.

We identify what type of data the company has and where it comes from. This may include:

- User data such as demographics, purchase history, and transaction records
- Product data such as inventory details, product descriptions, ingredients, popularity, and customer ratings
- Behavioral data, such as saved products, liked products, search history, and most visited products

After collecting the data, we perform data integration by:

- Wrangling data
- Aggregating datasets
- Joining and merging sources

- Mapping all data into a single central system

This reduces dependency on multiple databases for data retrieval.

3.4 Data Preparation and Transformation

Data preparation often overlaps with data collection because both can be done in parallel.

Since data comes from multiple sources, it may contain:

- Errors
- Missing values
- Inconsistent formats

In this stage, we prepare a refined dataset by:

- Cleaning data and removing irrelevant entries
- Handling extreme values to avoid bias in the dataset
- Handling missing values (either removing them or generating values depending on context)
- Ensuring correct data formats (e.g., dates as date types, text as strings)

We may also create new features, such as:

- Average time between user transactions
- Most frequently purchased products per user
- Mapping products to skincare concerns they address

We also perform exploratory data analysis (EDA), which includes:

- Creating visualizations to identify patterns
- Validating data with subject matter experts (e.g., beauty industry experts)
- Performing correlation analysis to find important variables affecting user behavior

Additionally, we decide how to split the dataset:

- Random split, or
- Time-based split (e.g., using the most recent transaction as the test set)

In this case, the most recent transaction is used as the test set while ensuring each user still has at least one record in the training set.

3.5 Model Development

In this stage, we build the machine learning model using existing frameworks instead of building everything from scratch.

We use two main techniques:

3.5.1 Content-Based Filtering

This method identifies similarities among products based on their attributes.

For example:

If a user buys a water-based cleanser, they may have dry skin and may also need a highly moisturizing product.

Steps include:

- Creating similarity scores between products
- Ranking products based on similarity
- Recommending top similar products
- Filtering out products that conflict with user preferences (e.g., avoided ingredients)

3.5.2 Collaborative Filtering

This method uses user behavior and similarities between users.

Steps include:

- Grouping users based on characteristics such as age, region, skin type, and purchase history
- Comparing users based on ratings and interactions
- Calculating average ratings from similar users
- Recommending products based on what similar users liked

The final system combines both content-based and collaborative filtering approaches.

3.6 Model Evaluation

After building the model, we evaluate its performance.

Initial evaluation includes:

- Testing the model on a reserved test dataset
- Tuning the model for better performance

After initial validation, we further test the model by:

- Deploying it to a small user group
- Collecting user feedback on recommendations
- Measuring metrics such as:
 - User ratings of recommendations
 - Click-through rates
 - Purchase rates of recommended products

These metrics help determine how well the model performs in real-world scenarios.

3.7 Deployment and Monitoring

Once evaluation is successful, the model is deployed into production as part of the beauty product app and website.

Even after deployment, we continuously:

- Monitor model performance
- Track user behavior and feedback
- Ensure recommendations remain relevant and accurate

3.8 Continuous Improvement

Future improvements include:

- Retraining the model with new data
- Expanding model capabilities
- Improving recommendation accuracy over time

3.9 Key Takeaways

- Every stage of the machine learning lifecycle is important for building a successful solution
- Data collection, preparation, modeling, and evaluation all contribute to model quality
- After deployment, continuous monitoring is required to maintain performance
- Machine learning systems improve over time through retraining and updates

Module 2: The Linear and Logistic Regression

4 Introduction to Regression

4.1 What is Regression?

Regression is a type of **supervised learning model**. It is used to model the relationship between:

- A **continuous target variable (dependent variable)**
- One or more **explanatory features (independent variables)**

In simple terms, regression helps us **predict continuous values** based on given data.

4.2 Example: CO2 Emissions Dataset

Consider a dataset related to CO2 emissions from different cars. The dataset includes:

- Engine size

- Number of cylinders
- Fuel consumption
- CO2 emissions

Using this data, we can answer the question:

Can we predict the CO2 emission of a new car using these features?

Yes, we can.

By using regression, we can:

- Train a model on existing data
- Learn relationships between features and emissions
- Predict CO2 emissions for new or hypothetical cars

4.3 Building a Regression Model

The process involves:

- Using historical data (features and target values)
- Training a regression model
- Applying the model to predict future or unknown values

4.4 Types of Regression

There are several types of regression models. The choice depends on:

- The nature of the data
- The relationship between variables
- The model that best fits the data

4.4.1 Simple Regression

Simple regression involves:

- **One independent variable**
- **One dependent variable**

Example:

- Predicting CO2 emissions using only engine size

4.4.1.1 Types of Simple Regression

Simple Linear Regression

- Assumes a **linear relationship** between variables

Simple Nonlinear Regression

- Assumes a **nonlinear relationship** between variables

4.4.2 Multiple Regression

Multiple regression involves:

- **More than one independent variable**
- One dependent variable

Example:

- Predicting CO2 emissions using:
 - Engine size
 - Number of cylinders

4.4.2.1 Linear vs Nonlinear in Multiple Regression

- If the relationship is straight-line → **Linear Regression**
- If the relationship is curved → **Nonlinear Regression**

4.5 Applications of Regression

Regression is widely used when we need to estimate **continuous values**.

4.5.1 Business and Economics

- Predicting yearly sales based on:
 - Number of customers
 - Leads
 - Order history

4.5.2 Real Estate

- Estimating house prices based on:
 - Size
 - Number of bedrooms

4.5.3 Industrial and Maintenance

- Predicting when machines require maintenance
- Avoiding unexpected failures

4.5.4 Income Prediction

- Estimating employment income based on:
 - Hours of work
 - Education

- Occupation
- Age
- Experience

4.5.5 Environmental Applications

- Estimating rainfall using:
 - Temperature
 - Humidity
 - Wind speed
 - Air pressure
- Predicting:
 - Wildfire probability
 - Severity of wildfires

4.5.6 Public Health

- Predicting the spread of infectious diseases
- Estimating the likelihood of diseases such as:
 - Diabetes
 - Heart disease
 - Cancer

4.6 Regression Algorithms

There are many regression algorithms, each suited for different scenarios.

4.6.1 Classical Methods

- Linear Regression
- Polynomial Regression

4.6.2 Modern Machine Learning Methods

- Random Forest
- XGBoost
- k-Nearest Neighbors (KNN)
- Support Vector Machines (SVM)
- Neural Networks

4.7 Summary

- Regression is a **machine learning technique** used to model relationships between variables
- It predicts **continuous values**
- **Simple regression** uses one independent variable
- **Multiple regression** uses multiple independent variables
- Regression can be **linear or nonlinear**

- It has applications in **business, healthcare, the environment, and many other fields**

5 Introduction to Simple Linear Regression

5.1 Overview

Simple Linear Regression is a statistical method used to model the relationship between a continuous target variable and one independent (explanatory) variable. It helps in predicting continuous outcomes based on input data.

After studying this concept, we understand what Simple Linear Regression is and how it works.

5.2 Dataset Example: CO₂ Emissions

Consider a dataset containing the following features of cars:

- Engine Size
- Number of Cylinders
- Fuel Consumption
- CO₂ Emissions

Using this dataset, Linear Regression can be applied to predict CO₂ emissions, which is a continuous value.

5.3 Concept of Simple Linear Regression

In Simple Linear Regression:

- Only one independent variable is used
- The dependent variable is predicted based on this single input

Example:

- Independent Variable (x_1): Engine Size
- Dependent Variable (y): CO₂ Emissions

5.4 Visual Representation

A scatter plot is used to visualize the relationship between:

- Engine Size (x-axis)
- CO₂ Emissions (y-axis)

The scatter plot shows the correlation between variables. As engine size increases, CO₂ emissions also increase, and the relationship is approximately linear.

5.5 Best-Fit Line

Simple Linear Regression determines a best-fit line through the data. This line represents the relationship between variables and is used to make predictions for new data points.

5.6 Prediction Example

For a car with an engine size of 2.4, the predicted CO₂ emission is calculated as:

$$\text{CO}_2 \text{ Emission} = 108.05 + 39 \times (\text{Engine Size})$$

$$\text{CO}_2 \text{ Emission} = 108.05 + 39 \times 2.4$$

$$\text{CO}_2 \text{ Emission} = 201.65$$

5.7 Mathematical Model

The regression model is represented as:

$$\hat{y} = \theta_0 + \theta_1 x_1$$

Where:

- $\hat{y}$ = Predicted value
- x_1 = Independent variable
- θ_0 = Intercept (bias)
- θ_1 = Slope (coefficient)

5.8 Coefficients of the Model

The coefficients θ_0 and θ_1 are selected by the linear regression algorithm to determine the best-fit line.

These were derived independently by mathematicians Gauss and Legendre.

Given:

- $\bar{x} = 3.0$
- $\bar{y} = 226.2$

Calculated values:

- $\theta_1 = 39$
- $\theta_0 = 125.7$

5.9 Residual Error

Residual error is the difference between the actual value and the predicted value. It represents the vertical distance from a data point to the regression line.

Example:

- Actual CO₂ = 250
- Predicted CO₂ = 340
- Error = 90

5.10 Mean Squared Error (MSE)

Mean Squared Error measures how poorly the regression line fits the data.

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

Linear regression aims to minimize this error.

5.11 Ordinary Least Squares (OLS)

This method is commonly known as Ordinary Least Squares (OLS) Regression. It finds the best-fit line by minimizing the mean of all residual errors.

5.12 Advantages of OLS Regression

- Easy to understand and interpret
- No tuning required
- Fast for small datasets

5.13 Limitations of Linear Regression

- Cannot capture nonlinear relationships
- Sensitive to outliers
- May be too simplistic for complex data

5.14 Summary

Simple Linear Regression predicts a continuous value using one independent variable. It determines a best-fit line and uses Mean Squared Error to evaluate performance. The OLS method minimizes error to produce the most accurate line. However, it may not perform well with complex or nonlinear data.

6 Introduction to Multiple Linear Regression

6.1 Overview

Multiple Linear Regression is an extension of Simple Linear Regression. It is used to:

- Describe Multiple Linear Regression
- Compare Multiple Linear Regression with Simple Linear Regression
- Identify the pitfalls of Multiple Linear Regression

It uses **two or more independent variables** to estimate a dependent variable.

6.2 Mathematical Representation

Multiple Linear Regression models the relationship between variables as a linear combination.

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

Where:

- $\hat{y}$ = Predicted value
- θ_0 = Intercept (bias term)
- $\theta_1, \theta_2, \dots, \theta_n$ = Model coefficients (weights)
- $x_1, x_2, \dots, x_n$ = Independent variables (features)

In matrix form:

- Features are represented as a matrix $\mathbf{X}$
- Parameters are represented as a vector $\boldsymbol{\theta}$

6.3 Dataset Example: CO₂ Emissions

Consider a dataset containing:

- Engine Size
- Number of Cylinders
- Fuel Consumption
- CO₂ Emissions

Multiple Linear Regression can be used to:

- Measure the effect of each independent variable on CO₂ emissions
- Predict CO₂ emissions for a new car (e.g., record 9)

6.4 Prediction Example

For a car with multiple features:

Given:

- $\theta_0 = 62.43$
- $\theta_1 = 9.19$
- $\theta_2 = 8.70$

Prediction formula:

$$\text{CO}_2 \text{ emission} = 62.43 + (9.19 \times \text{Engine Size}) + (8.70 \times \text{Cylinders}) + \dots$$

For a car with:

- Engine Size = 2.4

- Cylinders = 4

Calculation:

$$\text{CO}_2 \text{ emission} = 62.43 + (9.19 \times 2.4) + (8.70 \times 4) = 208.34$$

6.5 Model Interpretation

- With **one feature** → Line
- With **two features** → Plane
- With **more than two features** → Hyperplane

Multiple Linear Regression generalizes a linear function into higher dimensions.

6.6 Residual Error

Residual error is the difference between:

- Actual value
- Predicted value

Example:

- Actual CO₂ = 196
- Predicted CO₂ = 140
- Residual error = 56

Residual errors measure how far predictions are from actual values.

6.7 Mean Squared Error (MSE)

MSE measures model performance by averaging squared errors.

$$\text{MSE} = 1/n * \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Lower MSE = better model
- Used to evaluate regression performance

6.8 Least Squares Method

Multiple Linear Regression aims to minimize the sum of squared errors. This approach is called:

- Least Squares Regression

It finds parameters that minimize prediction error.

6.9 Methods to Estimate Parameters

6.9.1 Ordinary Least Squares (OLS)

- Uses linear algebra
- Computes optimal coefficients directly
- Minimizes MSE

6.9.2 Optimization Methods

- Uses iterative improvement
- Example: Gradient Descent
- Starts with random values and improves gradually
- Useful for large datasets

6.10 Gradient Descent

- Iterative optimization technique
- Adjusts parameters step-by-step
- Reduces error over time
- Efficient for large-scale data

6.11 Applications of Multiple Linear Regression

- Education: predicting exam performance
- Healthcare: analyzing blood pressure changes
- Business: forecasting outcomes using multiple factors
- What-if analysis for decision-making

6.12 What-If Scenarios

What-if analysis involves changing input variables to predict outcomes.

Example:

- Changing BMI to observe impact on blood pressure

6.12.1 Limitations of What-If Analysis

- May produce unrealistic scenarios
- May involve extrapolation beyond training data
- Fails when variables are highly correlated

6.13 Multicollinearity (Pitfall)

When independent variables are correlated:

- They are no longer truly independent

- This is called **collinearity**

Problems caused:

- Unstable predictions
- Misleading interpretation

6.13.1 Solution:

- Remove redundant variables
- Use only relevant, uncorrelated features

6.14 Overfitting Problem

Adding too many variables can cause:

- Overfitting
- Model memorizes training data
- Poor performance on unseen data

6.15 Key Summary

Multiple Linear Regression:

- Uses multiple independent variables
- Produces better predictions than simple regression
- Can overfit if not properly managed
- Uses OLS or optimization methods to find parameters
- Is widely used in real-world prediction problems

7 Introduction to Polynomial and Nonlinear Regression

7.1 Overview

Polynomial and Nonlinear Regression are statistical techniques used to model relationships between a dependent variable and one or more independent variables when the relationship is not linear.

7.2 Nonlinear Regression

Nonlinear Regression is a statistical method used to model relationships where the dependent and independent variables are connected through a **nonlinear equation**.

These equations can include:

- Polynomial functions
- Exponential functions
- Logarithmic functions

- Other non-linear mathematical functions

This method is useful when:

- The relationship between variables is complex
- A straight line cannot properly represent the data

Example:

- Exponential growth patterns in real-world datasets

7.3 Linear vs Nonlinear Relationship

Two charts were discussed:

- A straight line model may fit data reasonably, but often **underfits** real-world data
- Real-world data often follows a **smooth, curved pattern**
- A nonlinear curve provides a better approximation of such data

A straight line fails to capture complex trends, whereas nonlinear curves more accurately represent actual behavior.

7.4 Polynomial Regression

Polynomial Regression is a special case of nonlinear regression where:

- The relationship is modeled using polynomial expressions of features
- It can still be solved using linear regression techniques after transformation

7.4.1 General Idea

Polynomial regression fits data using polynomial curves of increasing degree:

- Linear (1st degree)
- Quadratic (2nd degree)
- Cubic (3rd degree)
- Higher-degree polynomials

These models can continue to higher degrees depending on data complexity.

7.5 Polynomial Regression Example

A cubic polynomial model is given as:

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

To simplify:

- $x_1 = x$

- $x_2 = x^2$
- $x_3 = x^3$

So the model becomes:

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

This transformation allows the use of **multiple linear regression techniques** to estimate parameters.

7.6 Overfitting in Polynomial Regression

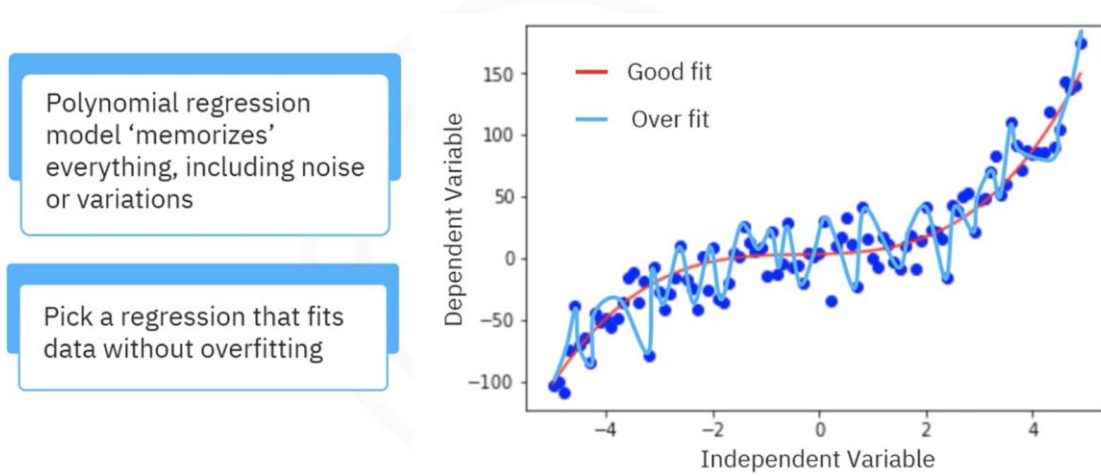
A key issue in polynomial regression is **overfitting**:

- A high-degree polynomial can pass through every data point exactly
- It captures noise instead of the true pattern
- This leads to poor generalization on new data

Key idea:

- Fit the **trend**, not every small variation in data

Overfitting with polynomials



7.7 Nonlinear Regression Beyond Polynomials

Not all nonlinear relationships can be modeled using polynomials.

Common nonlinear patterns include:

7.7.1 Exponential Growth

Example:

- Investment growth with compound interest

7.7.2 Logarithmic Relationships

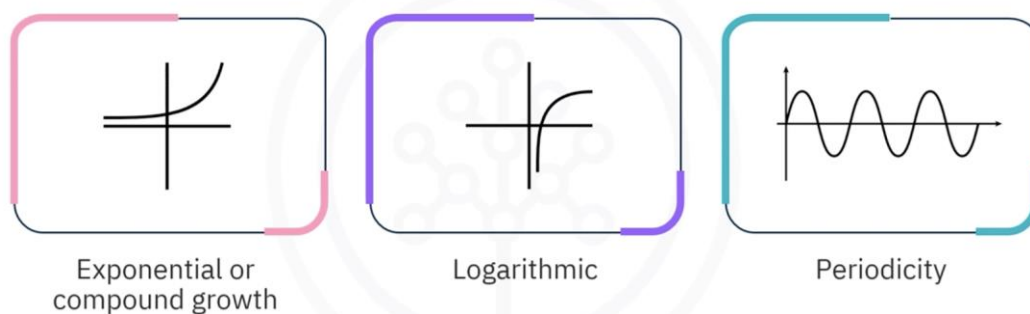
Example:

- Law of diminishing returns in productivity or profit

7.7.3 Periodic (Sinusoidal) Patterns

Example:

- Seasonal changes in temperature or rainfall



7.8 Real-World Example: GDP Growth

A dataset of China's GDP from 1960 to 2014 shows:

- Strong dependence on time
- A nonlinear increasing trend
- Increasing growth rate over time

This pattern is characteristic of **exponential growth**.

7.9 Diminishing Returns Example

In productivity:

- Output increases with work hours initially
- After a threshold (e.g., 6 hours), productivity per hour decreases
- This becomes a **logarithmic behavior**

7.10 Selecting a Regression Model

To determine the right model:

- Use **scatter plots** to analyze relationships
- Identify patterns visually

- Determine if the relationship is linear, exponential, logarithmic, or sinusoidal
- Test different models and evaluate performance

7.11 Optimization Methods

To find optimal nonlinear models:

- If mathematical form is known → use **gradient descent optimization**
- If the model is unknown → use machine learning models such as:
 - Regression Trees
 - Random Forests
 - Neural Networks
 - Support Vector Machines
 - Gradient Boosting Machines
 - K-Nearest Neighbors

7.12 Summary

- Nonlinear regression models complex relationships where linear models fail
- Polynomial regression is a special case of nonlinear regression
- Polynomial models can be converted into a linear form for parameter estimation
- Overfitting is a major risk in high-degree polynomials
- Real-world nonlinear patterns include exponential, logarithmic, and periodic behavior
- Model selection is often guided by data visualization and experimentation

8 Introduction to Logistic Regression

8.1 Overview

Logistic Regression is a statistical modeling technique used to predict the probability of an observation belonging to one of two classes, such as true or false. In machine learning, logistic regression refers to a binary classifier based on statistical logistic regression.

After studying this concept, we understand what Logistic Regression is and how it is used.

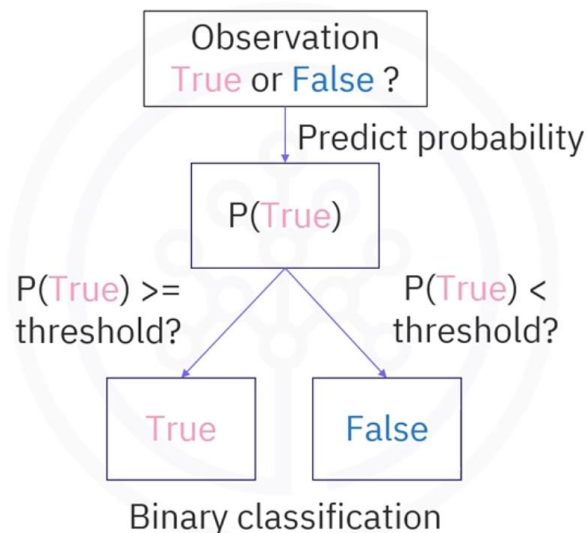
8.2 Concept of Logistic Regression

Logistic Regression predicts probabilities and then converts them into class labels.

- It predicts the probability of a class (0 or 1)
- A threshold is used to assign the final class
- If probability > threshold → Class 1
- If probability < threshold → Class 0

Thus, it acts as both:

- A probability predictor
- A binary classifier



8.3 When to Use Logistic Regression

Logistic Regression is a good choice in the following cases:

- When the target variable is binary (0 or 1)
- When we need probability outputs (e.g., likelihood of an event)
- When the data is linearly separable
- When we want to understand the impact of independent features

If the data is linearly separable, the decision boundary can be:

- A line (2D)
- A plane (3D)
- A hyperplane (higher dimensions)

Example:

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 > 0$$

8.4 Real-World Applications

Logistic Regression can be used in many practical scenarios:

- Predicting heart attack risk based on age, sex, and BMI
- Predicting diseases like diabetes using medical data
- Predicting customer churn in subscription services
- Predicting system or product failure
- Predicting mortgage default risk

8.5 Dataset Example: Customer Churn

Consider a telecommunication dataset where:

- Each row represents a customer
- Features include services, account information, and demographics
- Target variable: **Churn (Yes/No or 1/0)**

Logistic Regression can use these features to predict whether a customer will leave.

8.6 Model Representation

Let:

- x = independent feature (e.g., age)
- y = target variable (churn: 0 or 1)

Using linear regression:

$$\hat{y} = \theta_0 + \theta_1 x_1$$

Where:

- θ_0 = intercept
- θ_1 = slope (weight)

8.7 Problem with Linear Regression

Linear Regression is not suitable for classification because:

- Output values are not limited between 0 and 1
- Predictions can go beyond the valid probability range
- It does not provide meaningful probabilities

Using a simple threshold (e.g., 0.5) also fails because:

- It creates a step function
- It does not differentiate between extreme values

8.8 Sigmoid Function

To solve this problem, Logistic Regression uses the **Sigmoid Function**:

$$\sigma(x) = 1 / (1 + e^{-x})$$

Properties of the sigmoid function:

- Output range: 0 to 1
- $\sigma(0) = 0.5$
- As $x \rightarrow \infty$, $\sigma(x) \rightarrow 1$
- As $x \rightarrow -\infty$, $\sigma(x) \rightarrow 0$

This function converts any continuous value into a probability.

8.9 Logistic Regression Model

The model applies the sigmoid function to the linear equation:

$$\hat{p} = \sigma(\hat{y})$$

Where:

- $\hat{p}$ = predicted probability that $y = 1$
- $\hat{y}$ = linear combination of inputs

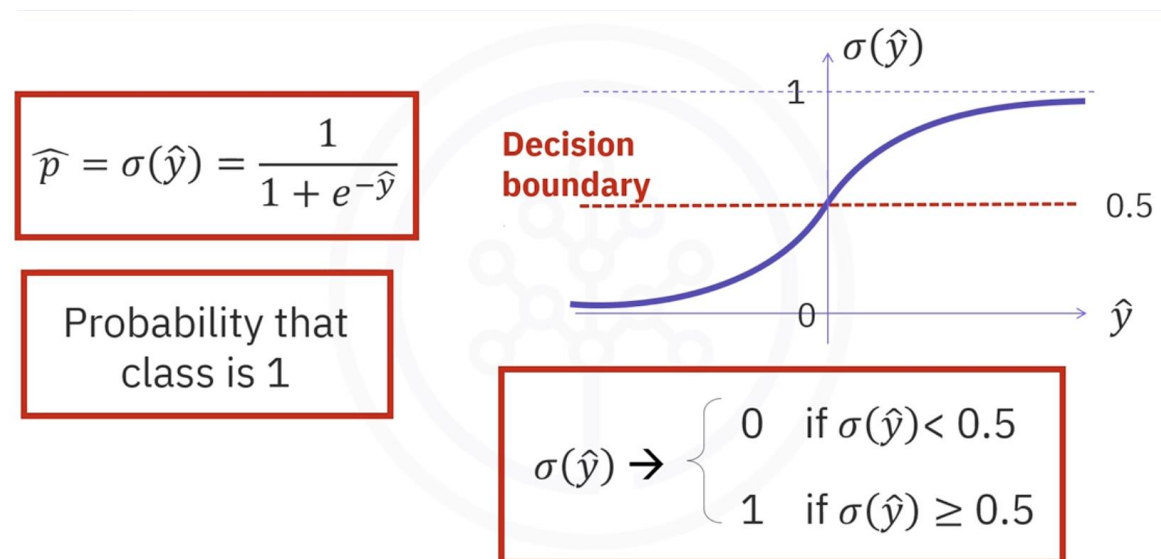
Thus, the model predicts the probability of belonging to a class.

8.10 Decision Boundary

A threshold (commonly 0.5) is used to assign classes:

- If $\hat{p} \geq 0.5 \rightarrow \text{Class 1}$
- If $\hat{p} < 0.5 \rightarrow \text{Class 0}$

This threshold is called the **decision boundary**.



8.11 Probability Interpretation

- Probability of churn = $P(y = 1 | x)$
- Probability of not churning = $1 - P(y = 1 | x)$

Example:

- If churn probability = 0.8
- Then the probability of staying = 0.2

8.12 Summary

Logistic Regression is a machine learning method used for binary classification. It predicts probabilities using the sigmoid function and assigns classes based on a threshold. It is useful when dealing with binary targets, probabilistic outcomes, and feature impact analysis. It serves both as a probability predictor and a binary classifier.

9 Training a Logistic Regression Model

9.1 Overview

Logistic Regression is used to predict **categorical outcomes (classes)**. The objective of training a logistic regression model is to find the best parameters that map input features to target classes with **minimal error**.

9.2 Objective of Logistic Regression Training

The goal of logistic regression training is to:

- Predict class labels accurately
- Minimize prediction error
- Find optimal parameters (θ) that best fit the data

The training process focuses on minimizing a **cost function**.

9.3 Steps in Training a Logistic Regression Model

The training process consists of the following steps:

9.3.1 Initialize Parameters

- Start with an initial set of parameters (θ)
- These values are usually chosen randomly

9.3.2 Predict Probabilities

- Predict the probability that each observation belongs to class 1

9.3.3 Compute Error (Cost Function)

- Measure the difference between predicted and actual values
- This error is represented by a cost function

9.3.4 Update Parameters

- Adjust θ to reduce prediction error

9.3.5 Repeat Until Convergence

- Continue the process until:
 - Log loss becomes sufficiently small, or
 - A maximum number of iterations is reached

9.4 Optimal Logistic Regression Model

A logistic regression model combines:

- A linear model ($\hat{y}$)
- A sigmoid function

This produces a **binary classification model**.

Initially, the model is not optimal. The best model is achieved after:

- Adjusting parameters (θ)
- Performing optimization

An optimization step is required to find the best parameters.

9.5 Cost Function: Log Loss

Log Loss is the cost function used in logistic regression. It measures how well predicted probabilities match actual class labels.

$$\{\text{Log Loss}\} = -\frac{1}{n} * \sum_{i=1}^n \left[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

Where:

- y_i = Actual class label
- $\hat{p}_i$ = Predicted probability
- n = Number of observations

9.5.1 Key Characteristics of Log Loss

- Logarithmic values between 0 and 1 are negative, hence the negative sign
- Lower log loss indicates a better model

9.5.2 Behavior of Log Loss

- **Correct and high prediction** → Small log loss
- **Incorrect and high prediction** → Very large log loss

Understanding log-loss

$$\text{Log-loss} = -\frac{1}{N} \sum_{i=1}^N y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)$$

Confident and correct: Predicted probability of class 1 is high and correct => log-loss is small

Confident and incorrect: Predicted probability of class 0 is high and incorrect => log-loss is large

Log loss:

- Rewards accurate predictions
- Penalizes incorrect predictions heavily

9.6 Optimization Using Gradient Descent

Gradient Descent is an iterative method used to minimize the cost function.

Gradient Descent is just:

“Keep adjusting your model little by little in the direction that reduces error until you can’t improve anymore.”

9.6.1 Working Principle

- Computes the gradient (slope) of the cost function
- Moves parameters in the **direction of steepest descent**(The fastest way to go downhill means reducing error)
- Updates θ to reduce error

9.6.2 Learning Rate

- Controls the step size of parameter updates
- Small learning rate → Slow convergence
- Large learning rate → Risk of missing the minimum

9.6.3 Key Idea

- The gradient points toward the steepest ascent(moving upward or minimizing error)
- Negative gradient points toward the steepest descent

As the algorithm approaches the minimum:

- The slope decreases
- Updates become smaller

9.7 Features of Gradient Descent

- Uses the **entire dataset** to compute gradients
- Can be slow for large datasets
- Sensitive to learning rate selection

9.8 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent is a variation of Gradient Descent.

9.8.1 Key Characteristics

- Uses a **random subset of data** instead of the full dataset
- Faster and more scalable
- Less computationally expensive

9.8.2 Advantages

- Faster convergence
- Works well with large datasets
- Can escape local minima(A point where the error is **low**, but **not the lowest possible overall**) and move towards global minima(the final lowest point or error)

9.8.3 Limitations

- Less stable than standard Gradient Descent
- May fluctuate around the minimum

9.8.4 Improving Convergence

- Decrease the learning rate over time
- Increase sample size gradually

9.9 Stopping Criteria

Training stops when:

- Log loss reaches an acceptable level
- Maximum iterations are completed

9.10 Summary

Logistic regression training aims to predict classes with minimal error by optimizing model parameters (θ). The process involves initializing parameters, predicting probabilities, computing log loss, and updating parameters iteratively.

Log loss is the key metric used for optimization. Gradient Descent is used to minimize this cost function, while Stochastic Gradient Descent provides a faster and scalable alternative.

The model improves by reducing error through repeated updates until it reaches an optimal solution.

10 Module 2 Summary and Highlights

10.1 Understanding Regression Models

Regression is a statistical method used to model the relationship between a continuous target variable and one or more explanatory features.

- **Simple Regression:** Utilizes a single independent variable to estimate a dependent variable.
- **Multiple Regression:** Involves two or more independent variables to predict a single outcome.

10.2 Real-World Applications

Regression analysis is a versatile tool used across various industries for predictive modeling and trend analysis, including:

- **Business:** Forecasting sales and estimating maintenance costs.
- **Environmental Science:** Predicting rainfall levels.
- **Healthcare:** Modeling the spread of diseases.

10.3 Simple Linear Regression and OLS

In simple linear regression, the goal is to find a "best-fit line" that minimizes errors.

- **Mean Squared Error (MSE):** The standard metric used to measure these errors.
- **Ordinary Least Squares (OLS):** The specific approach used to minimize MSE. While OLS is highly interpretable, it is notably sensitive to outliers, which can negatively impact the accuracy of the model.

10.4 Multiple Linear Regression and Model Balancing

Multiple linear regression extends the principles of simple regression to analyze more complex variable relationships.

- **The Overfitting Risk:** Adding an excessive number of variables can lead to overfitting, where the model performs well on training data but poorly on new data.
- **Variable Selection:** Careful selection of features is essential to build a balanced, generalizable model.

10.5 Nonlinear and Polynomial Regression

When data does not follow a straight line, nonlinear models are employed to capture complex relationships.

- **Functional Forms:** Includes polynomial, exponential, or logarithmic functions.
- **Polynomial Caution:** While polynomial regression can fit curved data, it carries a high risk of overfitting by capturing random noise rather than the underlying pattern.

10.6 Logistic Regression for Classification

Unlike linear regression, logistic regression serves as a probability predictor and binary classifier.

- **Suitability:** Ideal for binary targets (e.g., Yes/No, Success/Failure) and assessing the impact of specific features.
- **Error Minimization:** Utilizes **log-loss** to measure performance.
- **Optimization:** Employs **gradient descent** or **stochastic gradient descent** to improve efficiency and accuracy.

10.7 Optimization via Gradient Descent

Gradient descent is a fundamental iterative process used to minimize the cost function. It is a critical component in training logistic regression models, ensuring the model reaches the most accurate state possible through repeated refinement.

Module 3: Supervised Learning Models

11 Classification in Machine Learning

11.1 Overview

Classification is a supervised machine learning method that uses fully trained models to predict labels on new data. These labels are categorical variables with discrete values.

Supervised learning helps us understand data in the correct context when answering a specific question, ensuring accuracy in predictions. As data is input into the model, it is adjusted to fit the algorithm, which then classifies it by defining the input and predicted output.

11.2 Applications of Classification

Classification has a wide range of applications across industries. It is especially useful when problems can be expressed as relationships between features and labeled target variables.

Common applications include:

- Email filtering
- Speech-to-text systems

- Handwriting recognition
- Biometric identification
- Document classification

11.2.1 Business Use Cases

- Churn Prediction: Predicting whether a customer will discontinue a service
- Customer Segmentation: Predicting the category a customer belongs to
- Marketing Prediction: Predicting whether a customer will respond to an advertising campaign

11.3 Example: Loan Default Prediction

A bank may want to identify customers who are likely to default on loans.

- Historical data is used to train the model
- Features include age, income, and credit debt levels
- The model predicts whether a new customer will default

This is an example of a **binary classification problem**, where:

- Output has only two possible classes (e.g., Default / No Default)

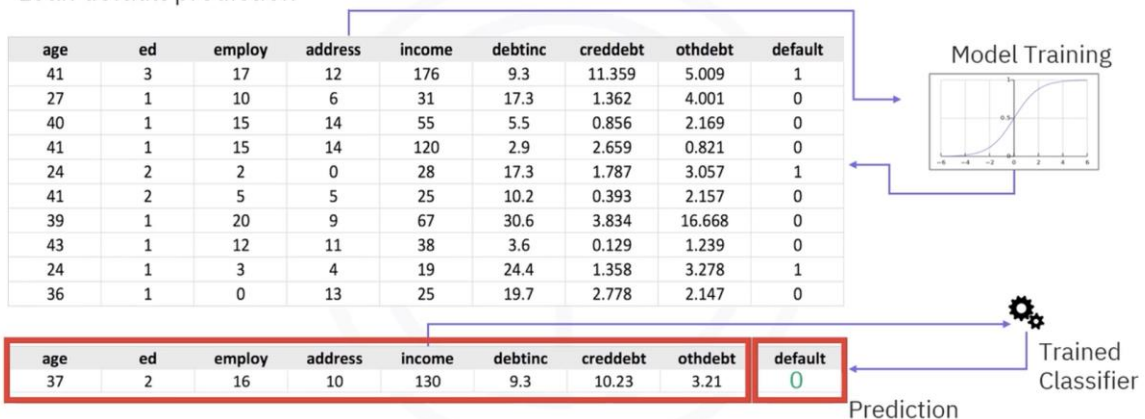
11.4 Binary Classification

Binary classification limits predictions to two classes. It is commonly used in:

- Fraud detection
- Spam detection
- Loan approval systems

Use cases of classification

Loan default prediction



11.5 Multi-Class Classification

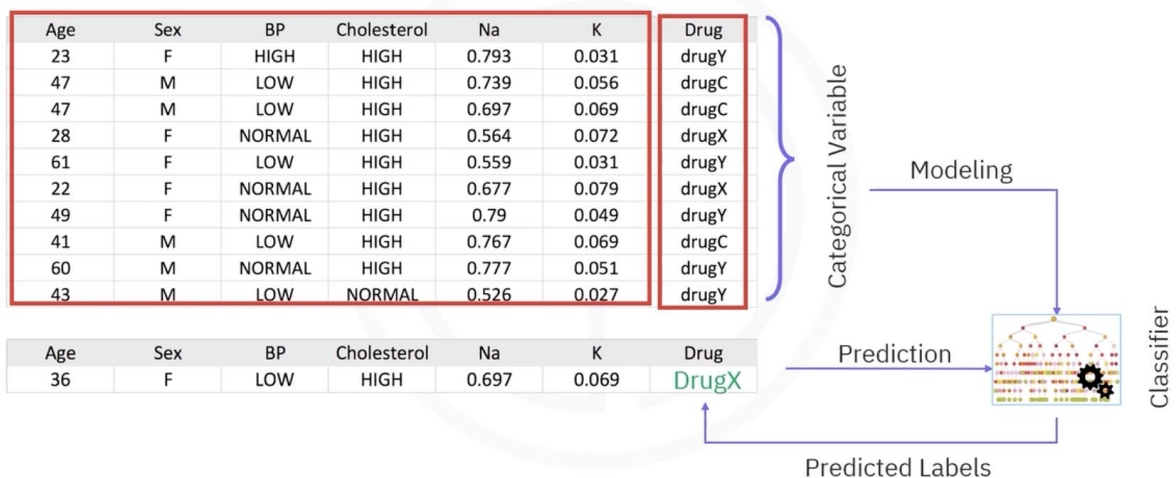
Multi-class classification involves predicting one label from more than two possible classes.

11.5.1 Example: Drug Prescription

- Patients with the same illness respond to different medications
- A dataset contains patient features and corresponding drug responses
- A classification model predicts the most suitable drug for a new patient

Use cases of classification

Multiclass drug prescription



11.6 Classification Algorithms

Several algorithms can be used to build classification models:

- Naive Bayes
- Logistic Regression
- Decision Trees
- K-Nearest Neighbors (KNN)
- Support Vector Machines (SVM)
- Neural Networks

Some algorithms (like Logistic Regression, KNN, and Decision Trees) can directly handle multiple classes.

11.7 Extending Binary Classifiers to Multi-Class

Some algorithms are naturally binary but can be extended to handle multiple classes using specific strategies.

11.7.1 One-Versus-All (OvA) Strategy

- Creates one binary classifier per class
- Each classifier predicts whether a data point belongs to its assigned class or not
- If there are k classes, there will be k classifiers

Key points:

- Each classifier focuses on one class vs all others
- A data point may not be classified by any classifier
- Such cases may indicate outliers or noise

One-versus-all strategy

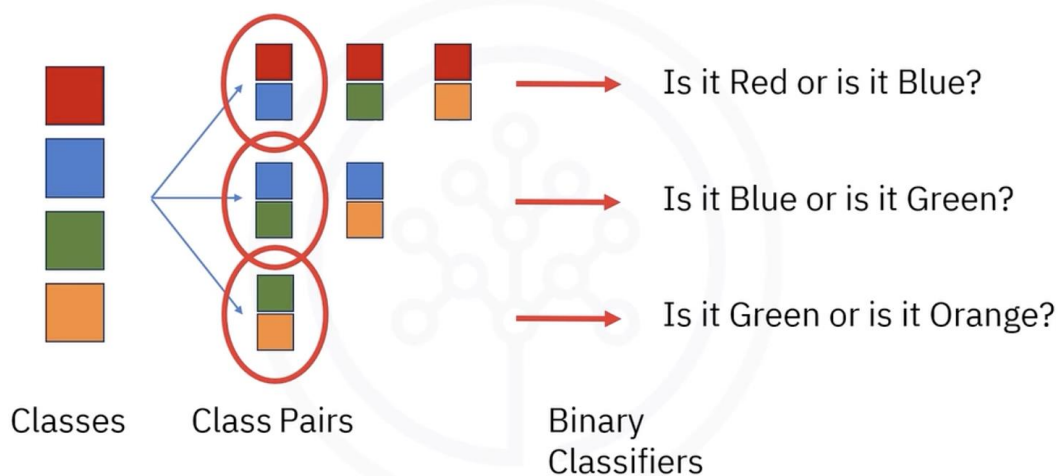
- **Binary classifier:** One for each class label
- Assigned a single label that defines target class
- **Task:** Binary prediction for every data point for a one-versus-the-rest classifier
- **K-classes** = K binary classifiers



11.7.2 One-Versus-One (OvO) Strategy

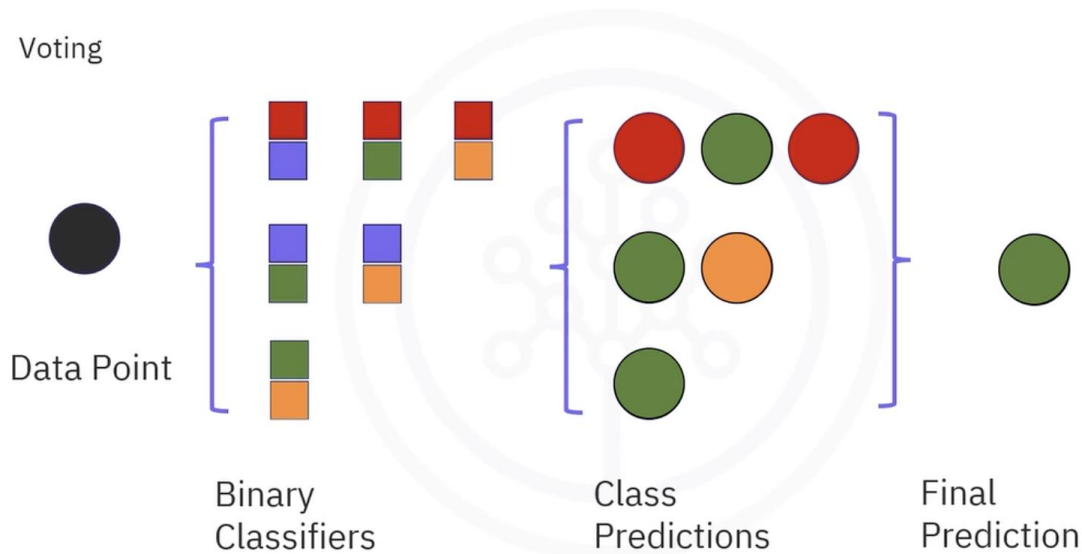
- Builds classifiers for every pair of classes
- Each classifier decides between two classes only

One-versus-one strategy



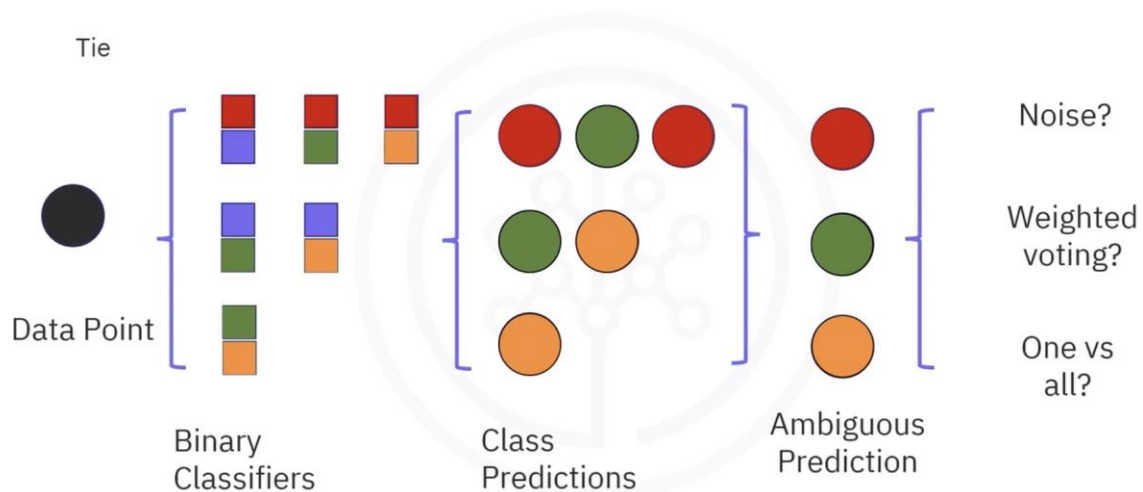
Process:

- Train classifiers on subsets of data for each pair
- Each classifier votes for a class
- The final prediction is based on voting



11.7.2.1 Voting Mechanism

- The class with the most votes is selected
- In case of a tie:
 - Use confidence/probability weighting
 - Or switch to another strategy like One-Versus-All



11.8 One-vs-All (OvA) vs One-vs-One (OvO) Example

11.8.1 Problem Setup

11.8.2 Dataset Description

- Total Rows = 1000
- Features = 10
- Target Variable = Animal (4 classes)

Classes:

- Cat
- Dog
- Monkey
- Cow

11.8.3 Data Split

- Training Data = 800 rows
- Testing Data = 200 rows

11.8.4 One-vs-All (OvA)

11.8.4.1 Number of Classifiers

For $K = 4$ classes:

- Number of Models = $K = 4$

Models:

- Cat vs (Dog, Monkey, Cow)
- Dog vs (Cat, Monkey, Cow)
- Monkey vs (Cat, Dog, Cow)
- Cow vs (Cat, Dog, Monkey)

11.8.4.2 Dataset Transformation

Example: Cat vs All

Original Label	New Label
Cat	1
Dog	0
Monkey	0
Cow	0

- Same 800 rows are used
- Only labels are changed
- This process is repeated 4 times (once per class)

11.8.4.3 Training Iteration

Each model:

- Uses all 800 rows
- Uses all 10 features
- Learns weights using logistic regression

Total Data Processed:

- $4 \times 800 = 3200$ row-iterations

11.8.4.4 Prediction Example

For one test sample:

Feature1 Feature2 ... Feature10

2.1 0.5 ... 1.2

Model Outputs:

Model	Probability
Cat vs All	0.65
Dog vs All	0.20
Monkey vs All	0.10
Cow vs All	0.05

Final Prediction:

- Class = $\text{argmax}(\text{probabilities}) = \text{Cat}$

11.8.5 One-vs-One (OvO)

11.8.5.1 Number of Classifiers

For $K = 4$:

- Number of Models = $K(K-1)/2 = 6$

Models:

- Cat vs Dog
- Cat vs Monkey
- Cat vs Cow
- Dog vs Monkey
- Dog vs Cow
- Monkey vs Cow

11.8.5.2 Dataset Splitting per Model

Each model uses only a subset of the dataset.

Example: Cat vs Dog

- Only rows where Animal = Cat or Dog are used

Assumed Distribution:

Class	Rows
Cat	200
Dog	200
Monkey	200
Cow	200

For Cat vs Dog:

- Only 400 rows are used

11.8.5.3 Training Iteration

Model	Rows Used
Cat vs Dog	400
Cat vs Monkey	400
Cat vs Cow	400
Dog vs Monkey	400
Dog vs Cow	400
Monkey vs Cow	400

Total Data Processed:

- $6 \times 400 = 2400$ row-iterations

11.8.5.4 Prediction (Voting Mechanism)

For one test sample:

Classifier	Winner
Cat vs Dog	Cat
Cat vs Monkey	Monkey
Cat vs Cow	Cat
Dog vs Monkey	Monkey
Dog vs Cow	Cow
Monkey vs Cow	Monkey

11.8.5.5 Vote Count

Class	Votes
Cat	2
Monkey	3
Cow	1
Dog	0

Final Prediction:

- Class = Monkey (highest votes)

11.8.6 Final Comparison

Aspect	OvA	OvO
Classes	4	4
Models	4	6
Rows per Model	800	~400
Total Row Usage	3200	2400
Prediction	Max Probability	Voting
Data Used	Full Dataset	Subset per Pair

11.8.7 Deep Insight

11.8.7.1 OvA

- Learns: “How is one class different from all others?”
- Harder problem due to class imbalance

11.8.7.2 OvO

- Learns: “How is one class different from another specific class?”
- Easier problem with clearer decision boundaries

11.8.7.3 Mental Model

- OvA = Multiple judges give scores → pick the highest
- OvO = Multiple pairwise battles → majority wins

11.8.8 When to Use One-vs-All (OvA)

11.8.8.1 Large Dataset

- Suitable for thousands or millions of rows
- Fewer models → faster training

11.8.8.2 Many Classes

- Works well when K is large

Example:

- K = 50
- OvO = 1225 models
- OvA = 50 models

11.8.8.3 Faster Training and Deployment

- Ideal for APIs and real-time systems
- Works well with limited computational resources

11.8.8.4 Baseline Model

- Simple and reliable
- Good starting point

11.8.8.5 Limitations

- Struggles with overlapping classes
- Imbalanced binary classification problem

11.8.9 When to Use One-vs-One (OvO)

11.8.9.1 Overlapping Classes

- Works better when classes are similar

Example:

- Cat vs Dog vs Fox

11.8.9.2 Small to Medium Dataset

- Manageable even with multiple models

11.8.9.3 High Accuracy Requirements

- Suitable for critical systems
- Example: medical or financial applications

11.8.9.4 Complex Decision Boundaries

- Simplifies classification into pairwise comparisons

11.8.9.5 Limitations

- Too many models when classes increase
- Slower training and prediction
- Higher memory usage

11.8.10 Real-World Mapping

Scenario	Best Choice
Image classification (many categories)	OvA
Text classification (many labels)	OvA
Face recognition (few similar faces)	OvO
Medical diagnosis	OvO
Quick prototype	OvA
High-precision system	OvO

11.8.11 Practical Decision Rule

Ask three questions:

11.8.11.1 Number of Classes

- Many $\rightarrow$ OvA
- Few $\rightarrow$ OvO

11.8.11.2 Data Complexity

- Simple $\rightarrow$ OvA
- Overlapping $\rightarrow$ OvO

11.8.11.3 Priority

- Speed $\rightarrow$ OvA
- Accuracy $\rightarrow$ OvO

11.8.11.4 Memory Trick

- OvA = Fast and Scalable
- OvO = Accurate and Detailed

11.9 Summary

Classification is a supervised machine learning method used to predict categorical labels. It is widely applied in real-world problems such as churn prediction, customer segmentation, loan default prediction, and drug prescription.

Classification can be performed using various algorithms, and binary classifiers can be extended to multi-class problems using strategies like One-Versus-All and One-Versus-One.

12 Decision Trees for Machine Learning

12.1 Introduction

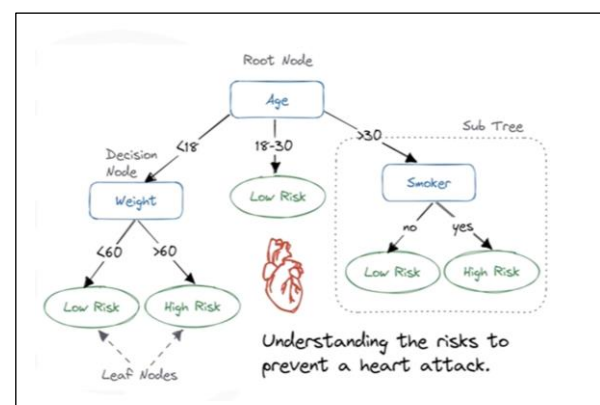
Decision Trees are a machine learning algorithm used for classification tasks.

12.2 What is a Decision Tree?

A Decision Tree is an algorithm that can be visualized as a flowchart used for classifying data points.

In a Decision Tree:

- Each **internal node** represents a test on a feature
- Each **branch** represents the outcome of that test
- Each **leaf (terminal node)** represents a class label



12.3 Example Dataset: Medical Prediction

Consider a medical dataset containing patient information:

- Age
- Gender
- Blood Pressure
- Cholesterol

Target variable:

- Drug response (Drug A or Drug B)

This dataset is used to predict which drug a future patient may respond to based on their features.

12.4 How Decision Trees Work

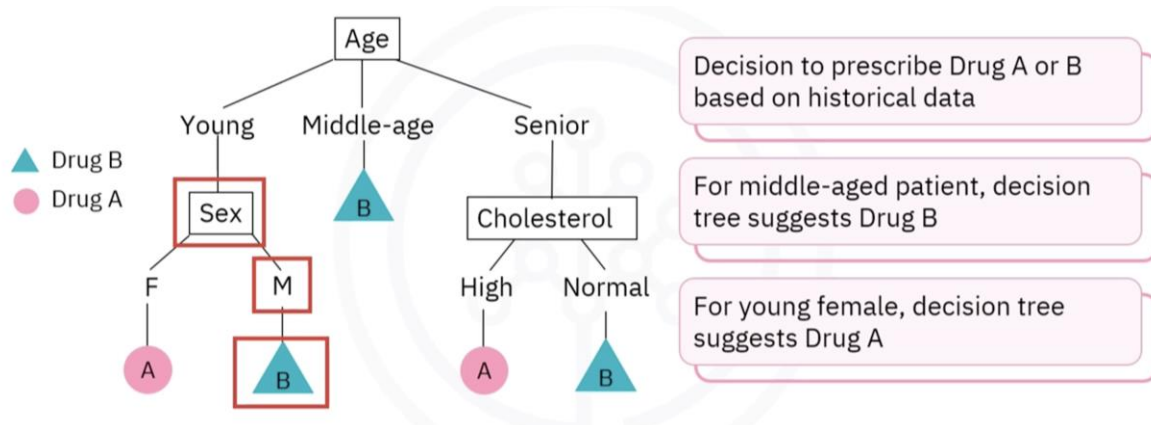
A Decision Tree is built by analyzing features step by step and splitting data based on conditions.

Example process:

- Patients are first categorized by **age group**:
 - Young
 - Middle-aged
 - Senior

Decision outcomes:

- Middle-aged → Drug B
- Young male → Drug B
- Senior with normal cholesterol → Drug B
- Young female → Drug A
- Senior with high cholesterol → Drug A



12.5 Building a Decision Tree

The training process follows these steps:

- Start with a root node and labeled training data
- Select the best feature to split the data
- Split the dataset into branches
- Pass each subset to new nodes
- Repeat the process recursively

The tree stops when:

- Each node contains a single class
- No features remain
- A stopping condition is met

12.6 Stopping Criteria (Tree Pruning)

Decision Trees stop growing when certain conditions are met (pre-emptive pruning), such as:

- Maximum tree depth reached
- Minimum number of samples per node exceeded
- Minimum samples per leaf reached
- Maximum number of leaf nodes reached

12.7 Why Pruning is Important

Pruning means stopping or trimming unnecessary branches of the tree.

Benefits of pruning:

- Prevents overfitting
- Reduces noise and irrelevant patterns
- Improves generalization
- Makes the model simpler and easier to understand
- Can improve predictive accuracy

12.8 Feature Selection and Splitting

Decision Trees use recursive partitioning to split data.

At each node:

- The algorithm selects the best feature to split the data
- A splitting criterion is used to measure the quality of the split

Common split criteria:

- Information Gain (Entropy Reduction)
- Gini Impurity

12.9 Example of Feature Splitting

12.9.1 Cholesterol Feature

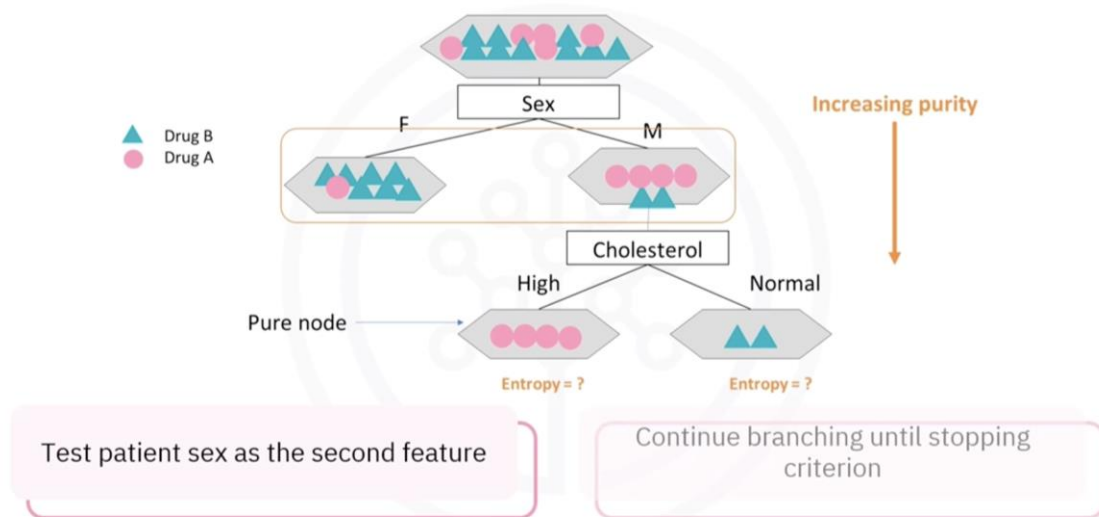
- High cholesterol → Mixed results (not very informative)
- Normal cholesterol → Still unclear classification

This shows that cholesterol is not the best feature.

12.9.2 Gender Feature

- Female → Mostly Drug B
- Male → Mixed results

Further splitting the male group improves classification accuracy.



12.10 Entropy (Measure of Impurity)

Entropy measures randomness or disorder in a dataset.

- Low entropy → Pure node (clear classification)
- High entropy → Mixed classes

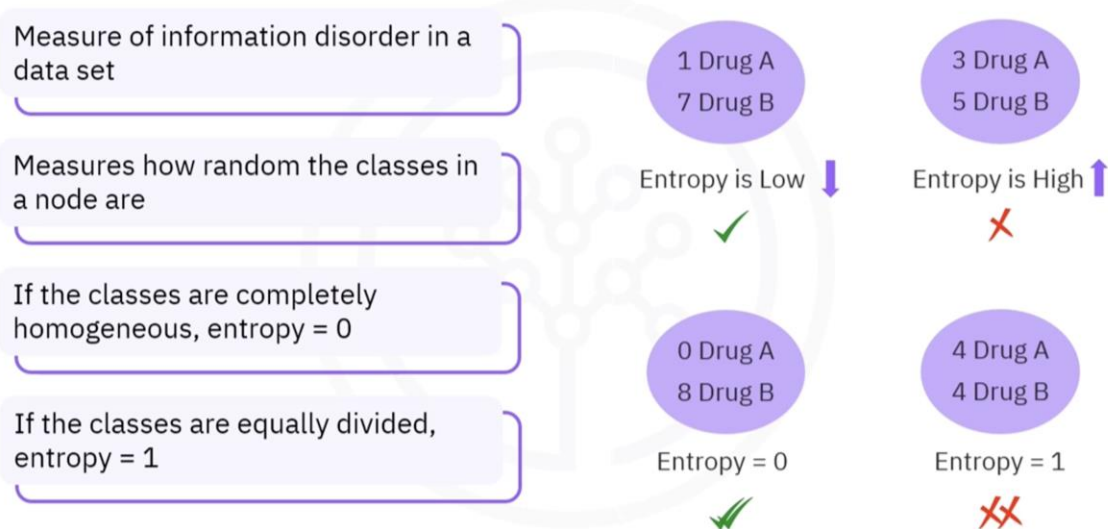
If:

- p_A = proportion of Drug A
- p_B = proportion of Drug B

Then entropy measures uncertainty in classification.

Key idea:

- Entropy = 0 → perfectly pure node
- Entropy = 1 → completely mixed classes



12.10.1 Homogeneous (Pure Node)

Imagine this node:

- 10 samples → all are **Drug A**

So:

- Probability of Drug A = 1
- Probability of Drug B = 0

👉 There is **no confusion at all**

So entropy = 0

✓ Meaning: “We are 100% sure.”

12.11 Information Gain

Information Gain measures how much a feature reduces entropy.

Formula idea:

- Information Gain = Entropy before split – Weighted Entropy after split

Example:

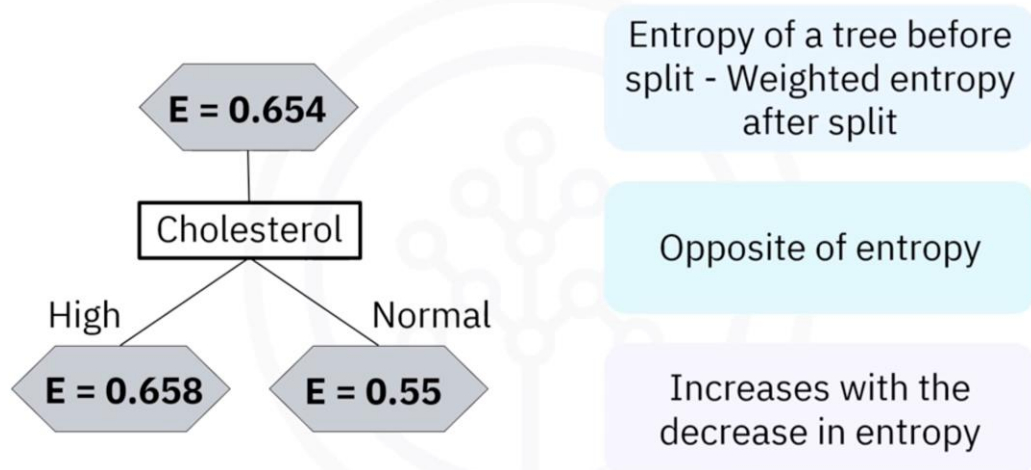
- Cholesterol split gives Information Gain = 0.042

Key relationship:

- Lower entropy $\rightarrow$ Higher information gain

Decision Trees aim to:

- Maximize information gain at each split



12.12 Gini Impurity

Gini Impurity is another measure used to evaluate split quality. It helps determine how well a feature separates classes.

12.13 How Decision Trees Learn

Decision Trees learn by:

- Selecting the best feature at each step
- Splitting data recursively
- Minimizing impurity using entropy or the Gini index
- Continuing until stopping criteria are met

12.14 Advantages of Decision Trees

- Easy to visualize
- Highly interpretable
- Shows the decision-making process clearly
- Provides feature importance insights

12.15 Summary

Decision Trees are classification models that:

- Split data based on features
- Build a flowchart-like structure

- Use entropy and information gain to choose splits
- Stop when a stopping condition is reached
- Can be improved using pruning

They are powerful because they are simple, visual, and interpretable, making them useful for understanding how predictions are made.

13 Regression Trees

13.1 Introduction

Regression Trees are a type of decision tree used in machine learning to predict **continuous values** instead of discrete classes.

13.2 What is a Regression Tree?

A regression tree is similar to a decision tree, but it is used for **predicting continuous numerical values** rather than categorical labels.

The main difference depends on the **target variable**:

- Classification → categorical output (e.g., true/false)
- Regression → continuous output (e.g., temperature, salary)

13.3 Classification Tree vs Regression Tree

13.3.1 Target Variable

- Classification Tree → categorical values
- Regression Tree → continuous values

13.3.2 Purpose

- Classification Tree → classifies data into discrete categories
- Regression Tree → predicts continuous numerical values

13.3.3 Leaf Node Prediction

- Classification Tree → majority vote (class label)
- Regression Tree → average of target values in the node

13.4 Use Cases

13.4.1 Classification Trees

- Spam detection
- Image classification
- Medical diagnosis

13.4.2 Regression Trees

- Revenue prediction
- Temperature forecasting
- Wildfire risk prediction

13.5 How Regression Trees Work

Regression trees are built by **recursively splitting the dataset** into smaller subsets. The goal is to improve prediction accuracy by reducing variability within each split.

This process:

- Creates a tree-like structure
- Reduces randomness in target values
- Improves prediction quality

13.6 Splitting Process

For a continuous feature:

- A threshold value (α) is selected
- Data is split into two groups:
 - Left node $\rightarrow$ values less than α
 - Right node $\rightarrow$ values greater than α

For binary features:

- Data is split directly based on the two categories

13.7 Prediction at Leaf Nodes

The predicted value ($\hat{y}$) at a node is calculated as:

- The **average of all actual target values** in that node

In some cases:

- Median can also be used (useful for skewed data)
- Mean is commonly used for normally distributed data

Formulae;

$$\hat{y} = 1/n * \sum_{i=1}^n (y_i)$$

13.8 Split Quality Measurement

Unlike classification trees (which use entropy or information gain), regression trees use:

- Mean Squared Error (MSE)

MSE measures how close predictions are to actual values.

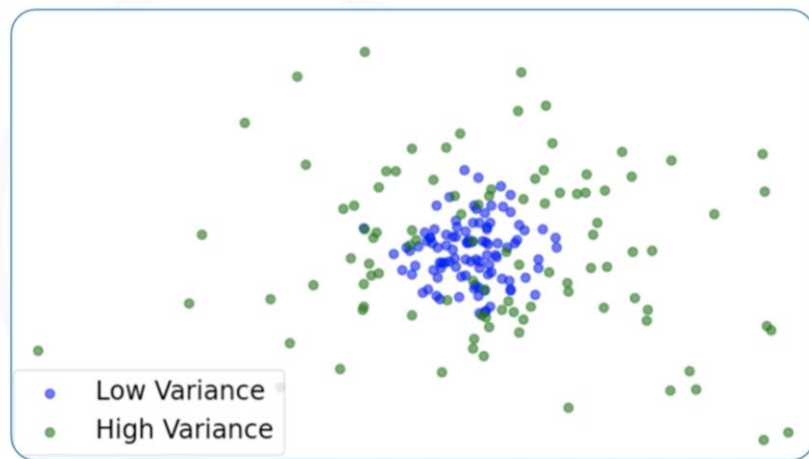
13.9 Mean Squared Error Concept

MSE represents the variance within a node:

- Lower MSE → values are closer together
- Higher MSE → values are more spread out

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$$

A measure of
target variance



13.10 Weighted MSE for Splits

To evaluate a split, regression trees compute a weighted average of MSE:

$$\text{Weighted MSE} = (1/N) \times [(N_{\text{left}} \times \text{MSE}_{\text{left}}) + (N_{\text{right}} \times \text{MSE}_{\text{right}})]$$

Where:

- N = total observations
- $N_{\text{left}}, N_{\text{right}}$ = number of observations in left and right nodes
- $\text{MSE}_{\text{left}}, \text{MSE}_{\text{right}}$ = MSE of each split

13.11 Choosing the Best Split

During training:

- The tree tests different features and thresholds
- For each split, it calculates weighted MSE

- The split with the **lowest weighted MSE** is selected

This process:

- Reduces variance
- Improves prediction accuracy

13.12 Handling Multi-Class Features

For multi-class features:

- One-vs-one or one-vs-all strategies are used
- Each possible binary split is tested
- The best split is selected based on the lowest weighted MSE

Binary Feature	Multiclass Feature
Seperated into two classes	Use one-vs-one or one-vs-all
Calculate weighted average of MSEs	Calculate weighted average for each binary split
No minimization needed	Select split that minimizes wieghted MSE

13.13 Selecting Thresholds for Continuous Features

A common method:

- Sort feature values
- Remove duplicates
- Compute midpoints between consecutive values as candidate thresholds

Formula:

$$\alpha_i = (X_i + X_{i+1}) / 2$$

The best threshold is the one that minimizes weighted MSE.

13.14 Efficiency Considerations

- Exhaustive search works well for small datasets
- Not efficient for large datasets
- For large data, sampling thresholds improve speed but may reduce accuracy

13.15 Summary

- Regression trees predict **continuous values**
- They differ from classification trees in output type and prediction method
- They use the **mean of values at leaf nodes** for prediction

- Splits are chosen using **Mean Squared Error (MSE)**
- The goal is to minimize variance and improve prediction accuracy
- Threshold selection plays a key role in performance

14 Supervised Learning with Support Vector Machines (SVM)

14.1 Overview

Support Vector Machines (SVM) is a supervised learning technique used for building both classification and regression models. It is widely used for identifying patterns and separating data into distinct classes.

14.2 Concept of Support Vector Machines

SVM maps each data instance as a point in a multidimensional space where:

- Each feature represents a coordinate
- Each data point corresponds to a position in that space

SVM classifies data by identifying a **hyperplane** that distinctly separates different classes.

14.3 Hyperplane and Classification

In a classification task with two features:

- The hyperplane becomes a **straight line**
- This line separates the dataset into two classes

When new data points are introduced:

- Their class depends on which side of the hyperplane they fall

14.4 Margin and Support Vectors

The main goal of SVM is to:

- Find a hyperplane that separates the data
- Maximize the **margin** between the two classes

Key concepts:

- **Margin:** Distance between the hyperplane and the closest data points
- **Support Vectors:** The nearest data points from each class that define the margin

A larger margin generally leads to better model performance on unseen data.

14.5 Soft Margin and Regularization

In real-world data:

- Perfect separation is often not possible due to noise and overlap

SVM introduces a **soft margin**:

- Allows some misclassifications
- Still tries to maximize the margin

This trade-off is controlled by parameter **C**:

- Small C → Allows more misclassification (softer margin)
- Large C → Enforces strict separation (harder margin)

14.6 Binary Classification in SVM

Binary SVM:

- Works with two classes
- Assumes data is linearly separable (you can separate different classes of data using a **single straight line** (in 2D) or a **plane** (in higher dimensions), which is called linearly separable data).

However, SVM can also be extended for regression tasks.

14.7 Mathematical Representation

SVM finds:

- A weight vector **w**
- A bias term **b**

The objective is to:

- Minimize the length of **w**
- Ensure correct classification using the condition:

$$y(w \cdot x + b) \geq 1$$

Where:

- **x** = Input data point
- **y** = Target label
- **w** = Weight vector
- **b** = Bias

14.8 Decision Boundary

The decision boundary:

- Is the hyperplane that separates classes
- Maximizes the margin

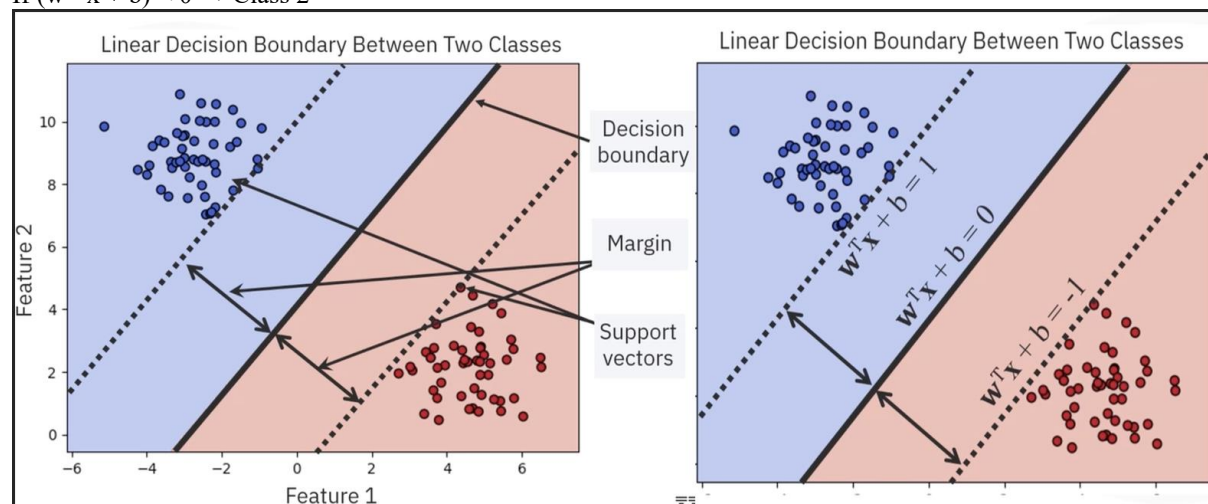
In 2D space:

- It is represented as a straight line

Classification rule:

If $(w \cdot x + b) > 0 \rightarrow \text{Class 1}$

If $(w \cdot x + b) < 0 \rightarrow \text{Class 2}$



14.9 Non-Linear Data and Kernel Trick

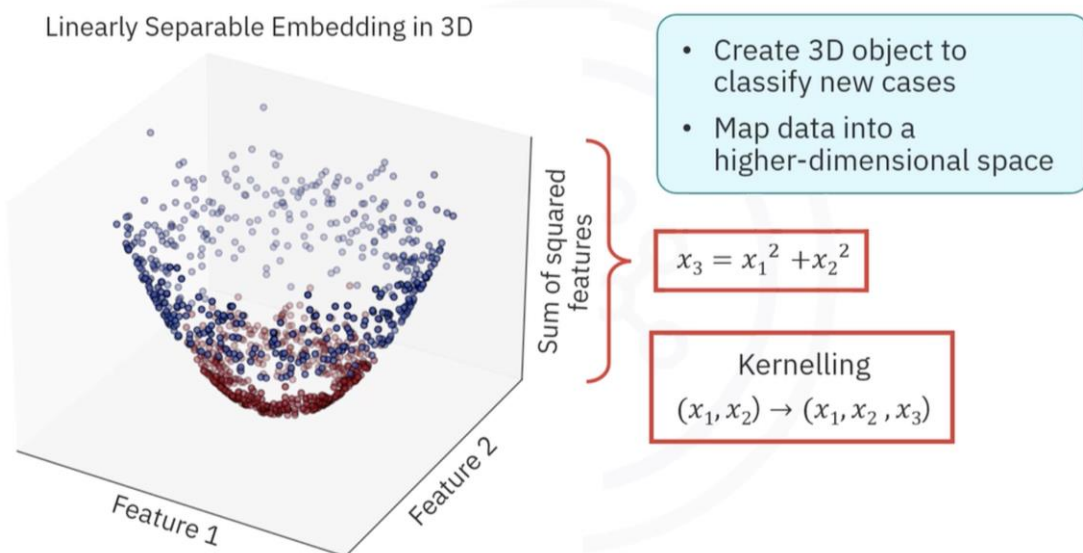
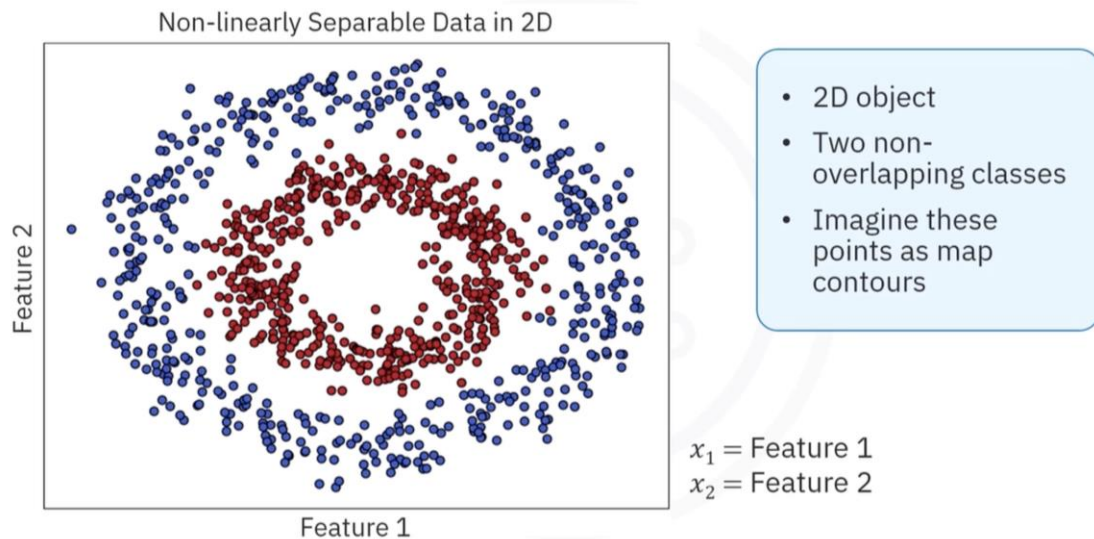
Some datasets are not linearly separable.

To handle this:

- Data is transformed into a higher-dimensional space
- This process is called **kerneling**

Example:

- 2D data transformed into 3D space
- Classes become separable by a plane



14.10 Kernel Functions in SVM

Common kernel functions include:

- **Linear Kernel** → Default SVM
- **Polynomial Kernel** → Captures curved relationships
- **Radial Basis Function (RBF)** → Handles complex patterns
- **Sigmoid Kernel** → Similar to logistic regression

Libraries like Scikit-learn provide these kernel options.

14.11 Support Vector Regression (SVR)

SVM can also be used for regression tasks.

In SVR:

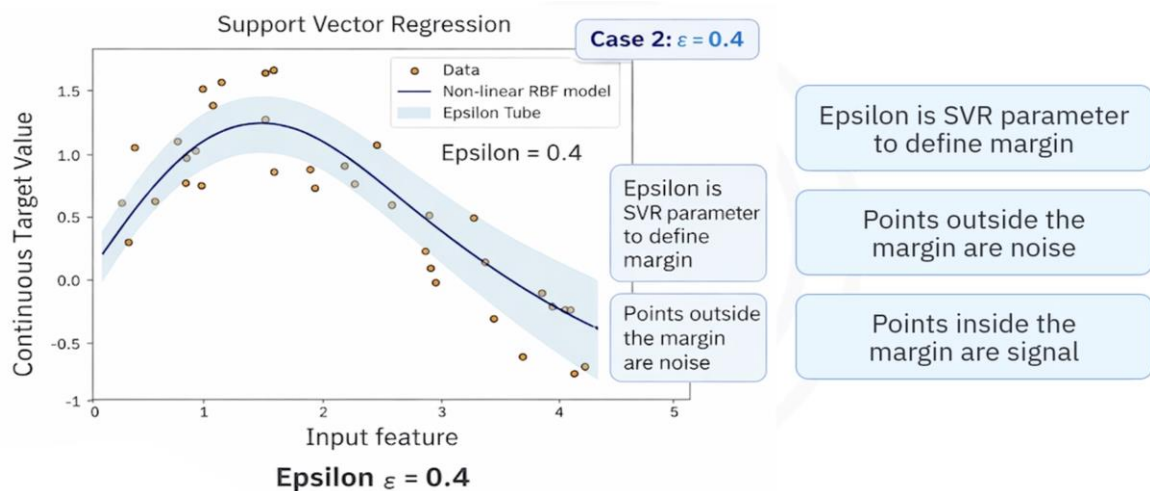
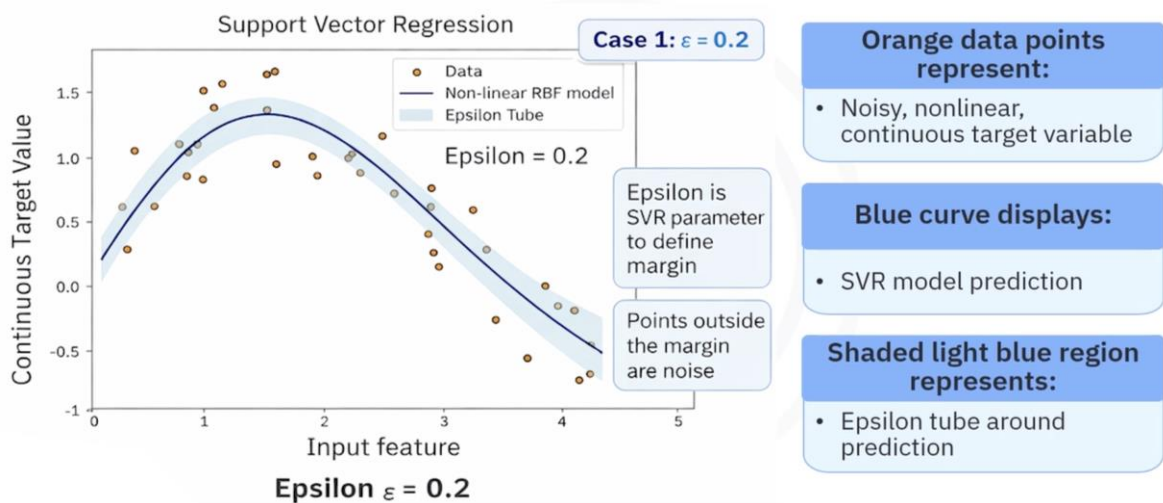
- A prediction curve is created
- An **epsilon (ϵ) tube** defines the margin around the curve

Key concepts:

- Points inside the tube $\rightarrow$ Considered correct (signal)
- Points outside the tube $\rightarrow$ Considered noise

Epsilon controls:

- Width of the margin
- Model sensitivity to errors



14.12 Advantages of SVM

- Effective in high-dimensional spaces
- Robust to overfitting
- Works well with linearly separable data
- Can handle weakly separable data using soft margin

14.13 Limitations of SVM

- Slow training for large datasets
- Sensitive to noise and overlapping classes
- Performance depends on kernel and parameter selection

14.14 Applications of SVM

SVM is widely used in:

- Image classification
- Handwritten digit recognition
- Spam detection
- Sentiment analysis
- Speech recognition
- Anomaly detection
- Noise filtering

14.15 Summary

Support Vector Machine (SVM) is a supervised learning technique used for classification and regression. It separates data using a hyperplane that maximizes the margin between classes. Kernel functions allow SVM to handle non-linear data. While SVM is powerful and effective, it can be sensitive to parameter choices and computationally expensive for large datasets.

15 Supervised Learning with K-Nearest Neighbors (KNN)

15.1 Overview

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm that uses labeled data points to learn how to classify or predict new data points.

15.2 Definition of KNN

K-Nearest Neighbors (KNN) is a supervised learning algorithm that:

- Uses labeled training data
- Classifies or predicts new data points based on their nearest neighbors

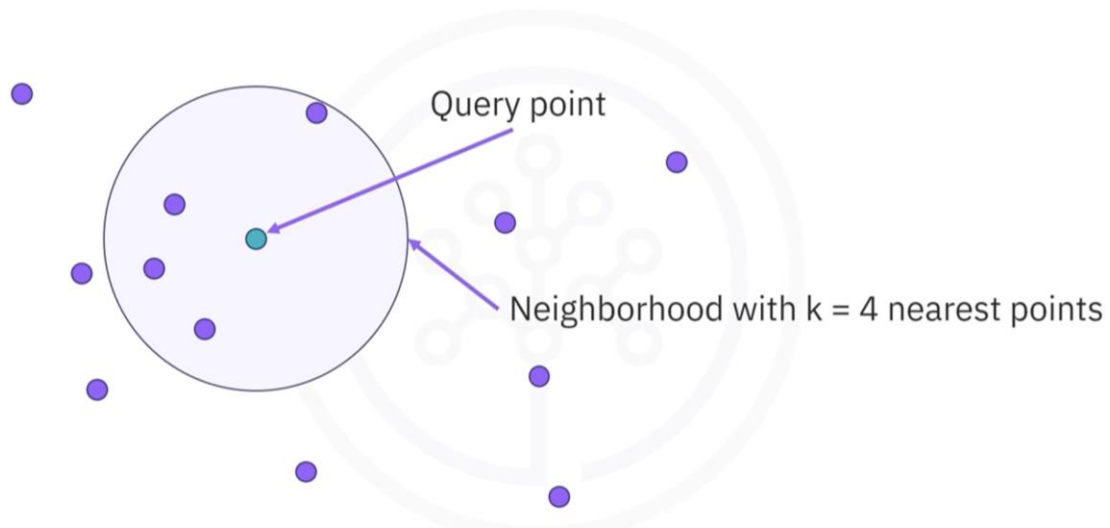
KNN can be used for:

- Classification
- Regression

15.3 Concept of Neighbors

In KNN:

- Data points that are close to each other are considered neighbors
- Points with similar features tend to belong to the same class



15.4 Working of the KNN Algorithm (Classification)

The KNN algorithm works as follows:

15.4.1 Step 1: Choose the Value of K

Select the number of nearest neighbors (K).

15.4.2 Step 2: Calculate Distance

Compute the distance between the query (unlabeled) point and all labeled data points.

15.4.3 Step 3: Find Nearest Neighbors

Identify the K closest data points to the query point.

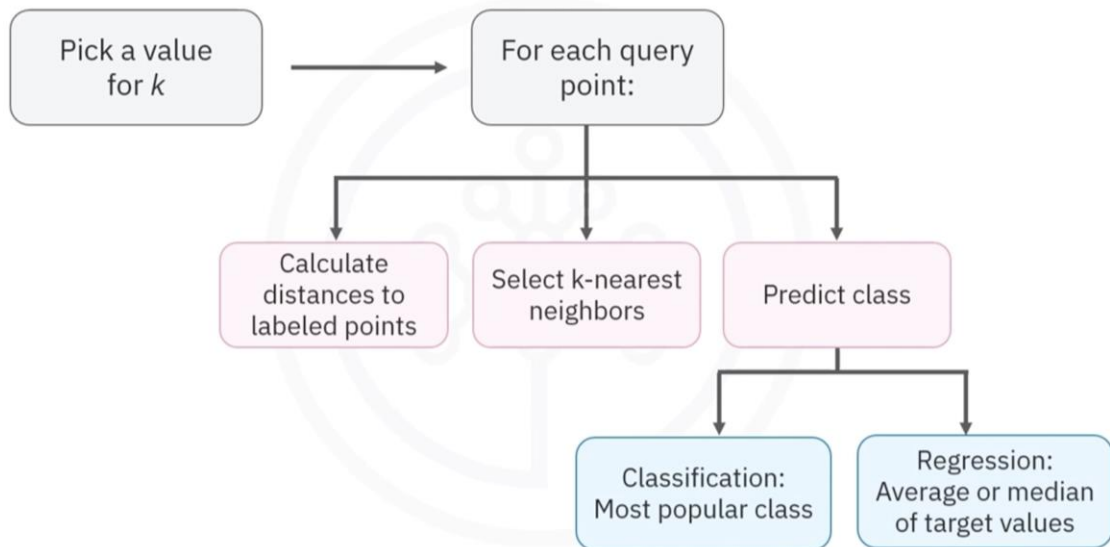
15.4.4 Step 4: Majority Voting

Assign the class label based on the most frequent class among the K nearest neighbors.

15.5 KNN for Regression

In regression problems:

- Instead of majority voting
- The prediction is made using:
 - Mean of neighbors
 - Median of neighbors



15.6 Distance and Similarity

To determine similarity between data points, distance metrics are used.

Example dataset:

- Iris dataset with 50 samples from each species:
 - Iris setosa
 - Iris versicolor
 - Iris virginica

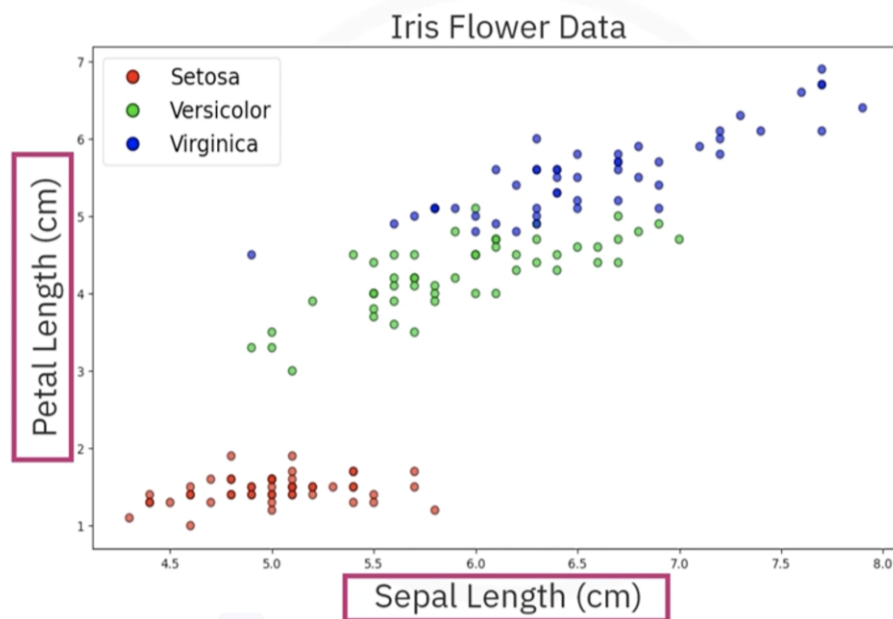
Features include:

- Sepal length
- Sepal width
- Petal length
- Petal width

15.7 Classification Example

Using a scatter plot of:

- Sepal length
- Petal length



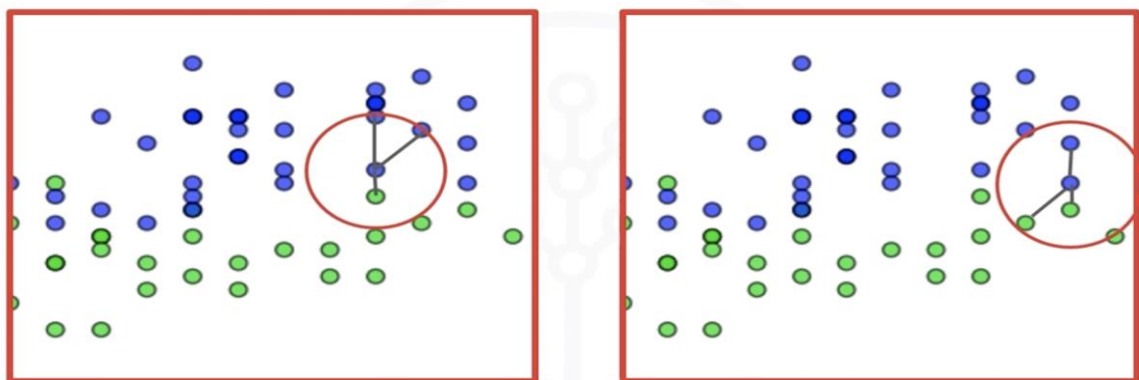
KNN classifies a new point by:

- Finding its nearest neighbors
- Applying majority voting

Example:

- If most neighbors belong to virginica → classified as virginica
- If most neighbors belong to versicolor → classified as versicolor

Some cases may lead to incorrect classification depending on the neighbors.



15.8 Decision Boundary

KNN creates decision boundaries based on:

- Value of K
- Input features

Example:

- Using $K = 3$
- Two features: sepal length and petal length

The model divides the feature space into regions representing different classes.

The model can be implemented using:

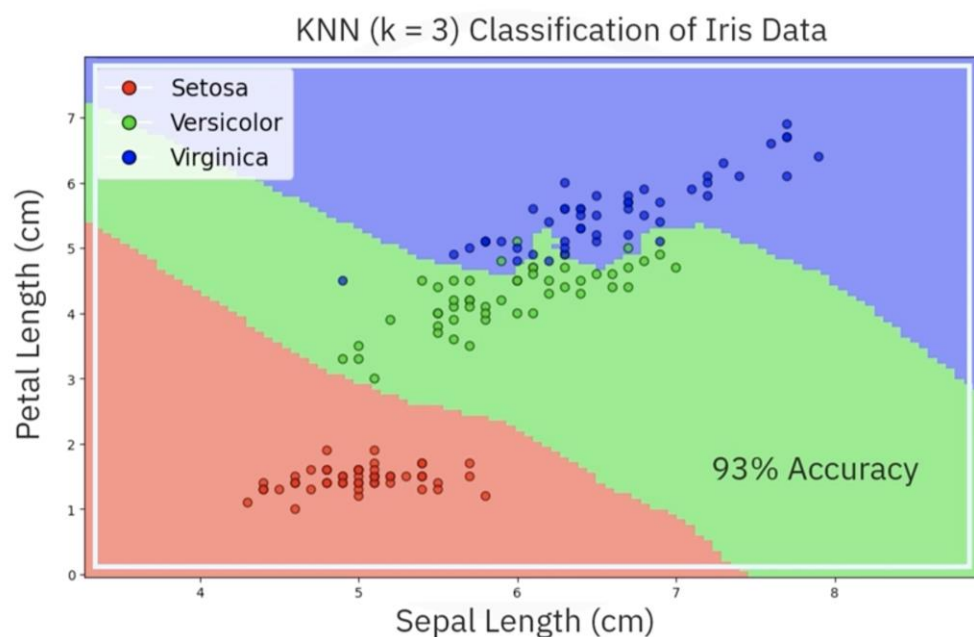
- Scikit-Learn K-Neighbors Classifier

15.9 Model Accuracy

KNN can achieve high accuracy depending on the dataset.

Example:

- Accuracy achieved: 93%



15.10 Choosing the Optimal Value of K

To find the best K:

- Test multiple values of K
- Train the model using training data
- Evaluate accuracy using test data
- Select the K with the highest accuracy

Example:

- $K = 4$ gives the best accuracy in a specific case



15.11 Nature of KNN (Lazy Learning)

KNN is a lazy learner:

- It does not build a model during training
- It stores all training data
- It makes predictions during query time

Steps during prediction:

- Compute distances to all training points
- Sort distances
- Select top K neighbors

15.12 Effect of K on Model Performance

15.12.1 Small K (Low Value)

- Sensitive to noise
- High variance
- Leads to overfitting

15.12.2 Large K (High Value)

- Smooths the decision boundary
- High bias
- Leads to underfitting

15.12.3 Optimal K

- A balance between overfitting and underfitting

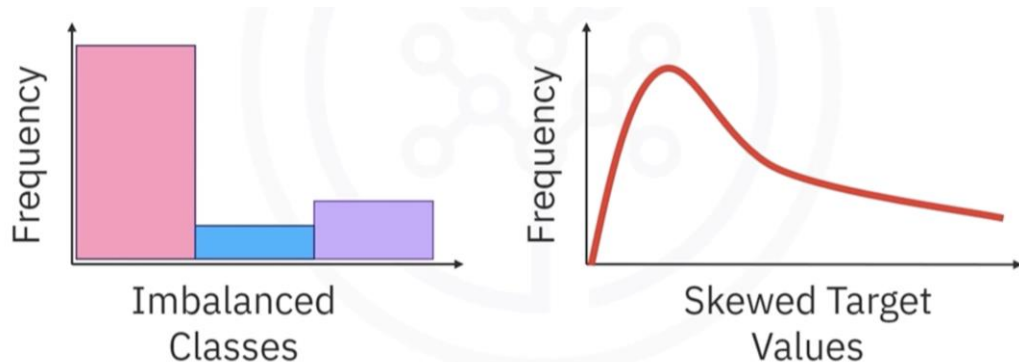
15.13 Challenges in KNN

15.13.1 Imbalanced Classes

- The majority class dominates predictions
- Leads to biased classification

Solution:

- Use distance-weighted voting



15.13.2 Feature Scaling Issue

- Features with large values dominate distance calculations
- Leads to incorrect predictions

Solution:

- Apply feature scaling (e.g., standardization)

15.13.3 Irrelevant Features

- Add noise to the model
- Reduce accuracy

15.13.4 Redundant Features (highly correlated with another feature)

- Increase computational cost
- Do not improve performance

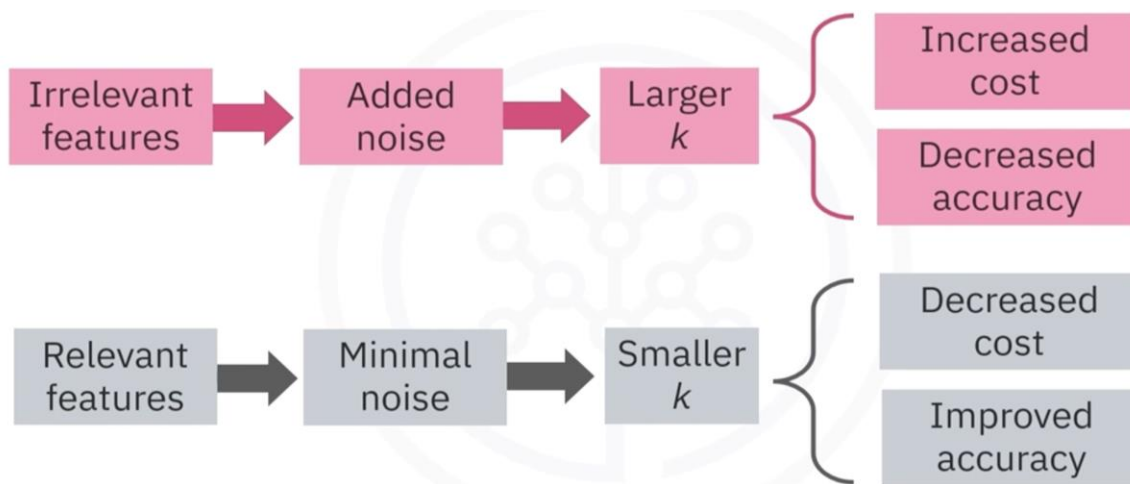
15.14 Feature Selection

Selecting relevant features:

- Improves accuracy
- Reduces computational cost
- Helps in choosing optimal K

Method:

- Train the model with and without a feature
- Compare performance



15.15 Summary

KNN is a supervised learning algorithm used for classification and regression. It predicts values based on nearest neighbors. The value of K plays a crucial role in model performance. Proper feature selection and scaling improve accuracy. KNN is simple but can be computationally expensive and sensitive to data quality.

16 Bias, Variance, and Ensemble Models

16.1 Overview

Bias, variance, and ensemble models are fundamental concepts in machine learning that help us understand model performance in terms of accuracy and precision. These concepts also explain how different models behave when trained on data and how their performance can be improved.

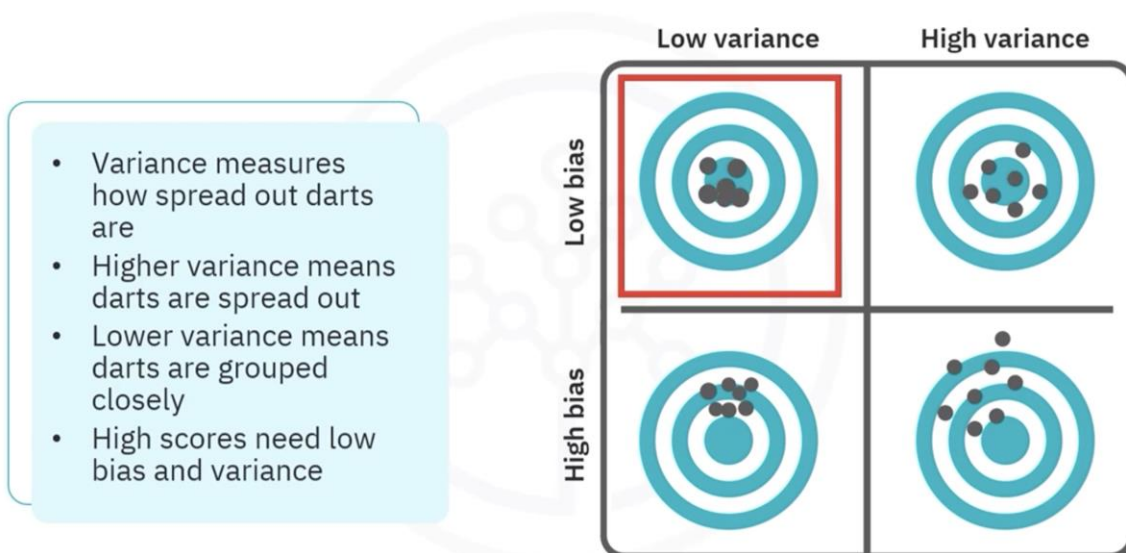
16.2 Understanding Bias and Variance (Dartboard Analogy)

Bias and variance can be understood using a dartboard example:

- **Low Bias (High Accuracy):** Darts are close to the center
- **High Bias (Low Accuracy):** Darts are far from the center
- **Low Variance (High Precision):** Darts are closely grouped
- **High Variance (Low Precision):** Darts are spread out

Key idea:

- Bias represents how **accurate** the predictions are
- Variance represents how **consistent (precise)** the predictions are



The ideal model has:

- **Low Bias + Low Variance**

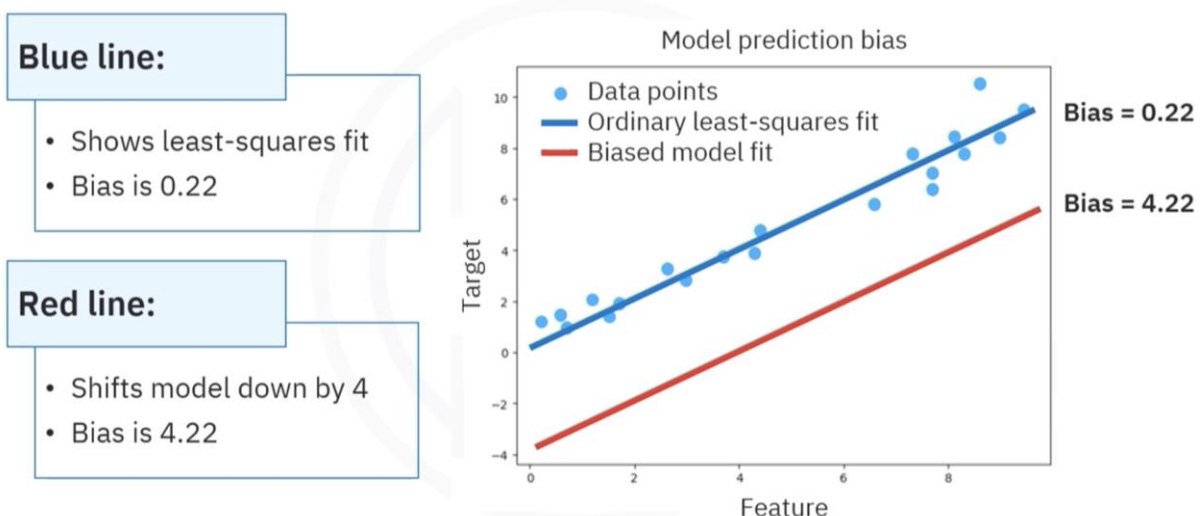
16.3 Prediction Bias

Prediction bias refers to how far the model's predictions are from the actual values.

- It is measured as the **average difference between predicted and actual values**
- A perfect model has **zero bias**

Example:

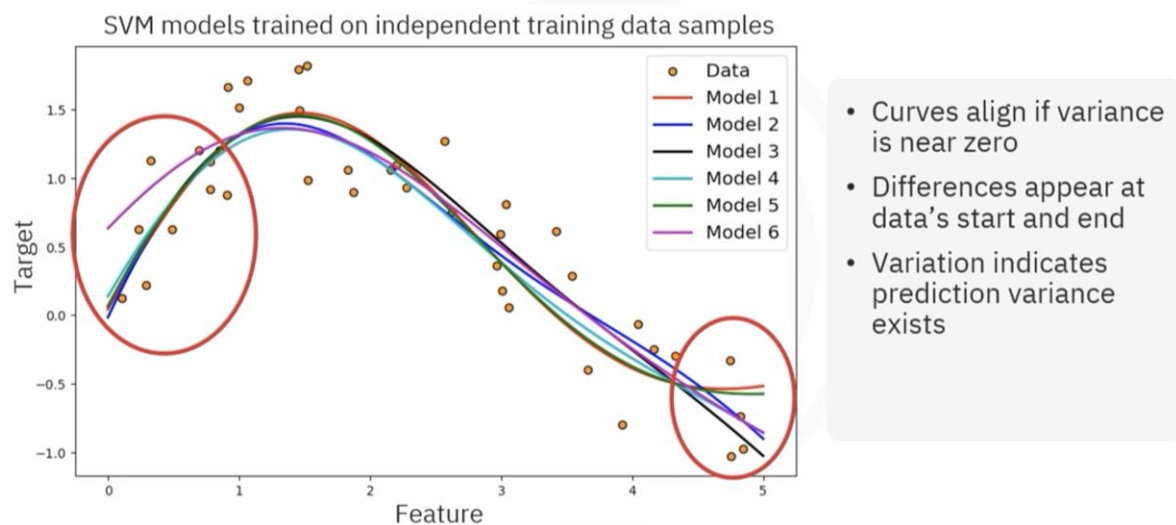
- A model with bias = 0.22 → Low bias (better accuracy)
- A model with bias = 4.22 → High bias (poor accuracy)



16.4 Prediction Variance

Prediction variance measures how much predictions change when the model is trained on different subsets of data.

- **High Variance:**
 - The model is sensitive to training data
 - Leads to **overfitting**
 - Captures noise and outliers
- **Low Variance:**
 - The model is stable
 - Generalizes well to unseen data



16.5 Bias-Variance Tradeoff

Bias and variance are inversely related to model complexity.

- As **model complexity increases**:
 - Bias **decreases**
 - Variance **increases**

16.5.1 Underfitting

- Occurs when model complexity is low
- High bias, low variance
- Poor performance even on training data

16.5.2 Overfitting

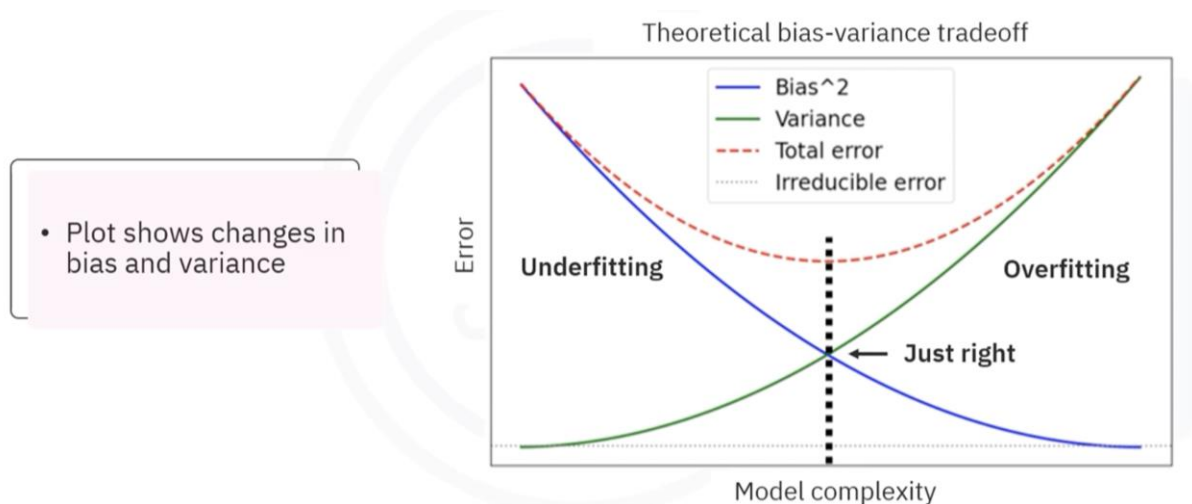
- Occurs when model complexity is high
- Low bias, high variance
- Performs well on training data but poorly on unseen data

16.5.3 Optimal Model

- There exists a **balance point** where:
 - Bias and variance are optimized
 - The model performs best

Note:

- Some error always remains due to **random noise in the data**



16.6 Weak Learners and Strong Learners

16.6.1 Weak Learners

- Perform slightly better than random guessing
- High bias, low variance
- Tend to underfit

16.6.2 Strong Learners

- Low bias, high variance
- Tend to overfit

16.7 Ensemble Learning

Ensemble methods combine multiple models to improve performance by balancing bias and variance.

Common base learners:

- Decision Trees
- Regression Trees

These models are preferred because their:

- Bias and variance can be adjusted (e.g., by changing tree depth)

16.8 Bagging (Bootstrap Aggregating)

16.8.1 Concept

Bagging involves:

- Training multiple models on **bootstrapped (randomly sampled) subsets** of data
- Averaging their predictions

16.8.2 Key Characteristics

- Models are trained **in parallel**
- Reduces **variance**
- Helps prevent **overfitting**

16.8.3 Example: Random Forest

Random Forest is a popular bagging technique:

- Uses multiple decision trees
- Trains each tree on different bootstrapped datasets

Key idea:

- Individual trees may have high variance
- Averaging reduces overall variance

16.9 Boosting

16.9.1 Concept

Boosting builds models **sequentially**, where each new model:

- Focuses on correcting errors made by previous models

16.9.2 Key Characteristics

- Reduces **bias**
- Models are trained **one after another**
- The final model is a **weighted combination** of all models

16.9.3 Working Mechanism

- Misclassified data points are given **higher weights**
- Correctly classified points are given **lower weights**
- Each new model focuses more on difficult cases

16.9.4 Popular Boosting Algorithms

- Gradient Boosting
- XGBoost
- AdaBoost

16.10 Bagging vs Boosting

16.10.1 Bagging

- Reduces variance
- Uses high-variance, low-bias models
- Models trained in parallel
- Helps mitigate overfitting

16.10.2 Boosting

- Reduces bias
- Uses low-variance, high-bias models
- Models trained sequentially
- Helps mitigate underfitting

Ensemble	Objective	Base Learners	Training	Outcome
Bagging	Mitigate overfitting	High variance Low bias	Parallel on bootstrapped data	Reduced variance
Boosting	Mitigate underfitting	Low variance, High bias	Builds on previous result	Reduced bias

16.11 Managing Bias-Variance Tradeoff

- **Increase model complexity → Reduce bias (use boosting)**
- **Reduce model variance → Use bagging**

Ensemble methods help find the right balance between bias and variance.

16.12 Summary

- Bias measures accuracy, while variance measures consistency
- High bias leads to underfitting, high variance leads to overfitting
- The bias-variance tradeoff is controlled through model complexity
- Weak learners have high bias, strong learners have high variance
- Bagging reduces variance by averaging multiple models
- Boosting reduces bias by sequentially improving models
- Ensemble methods effectively balance bias and variance for better performance

17 Module 3 Summary and Highlights

17.1 Classification in Machine Learning

Classification is a supervised machine learning method used to predict labels on new data.

17.1.1 Applications of Classification

- Churn Prediction
- Customer Segmentation
- Loan Default Prediction
- Multiclass Drug Prescriptions

17.2 Types of Classification

17.2.1 Binary Classification

Binary classifiers predict one of two possible classes.

17.2.2 Multiclass Classification

Binary classifiers can be extended to multiclass classification using:

- One-versus-All Strategy
- One-versus-One Strategy

17.3 Decision Trees

A decision tree classifies data by:

- Testing features at each node
- Branching based on test results
- Assigning classes at leaf nodes

17.3.1 Training Decision Trees

Decision tree training involves:

- Selecting features that best split the data
- Pruning the tree to avoid overfitting

17.3.2 Split Evaluation Metrics

- Information Gain
- Gini Impurity

These metrics are used to measure the quality of splits in decision trees.

17.4 Regression Trees

Regression trees are similar to decision trees but are used to predict continuous values.

17.4.1 Working of Regression Trees

- Data is recursively split
- Splits are chosen to maximize information gain

17.4.2 Evaluation Metric

Mean Squared Error (MSE)

MSE is used to measure the quality of splits in regression trees.

17.5 K-Nearest Neighbors (k-NN)

K-Nearest Neighbors is a supervised algorithm used for both classification and regression.

17.5.1 Working of k-NN

- Assigns labels based on the closest labeled data points

17.5.2 Optimization of k-NN

To optimize k-NN:

- Test different values of k
- Measure accuracy
- Consider class distribution
- Evaluate feature relevance

17.6 Support Vector Machines (SVM)

Support Vector Machines build classifiers by finding a hyperplane that maximizes the margin between two classes.

17.6.1 Characteristics of SVM

- Effective in high-dimensional spaces
- Sensitive to noise
- Less efficient with very large datasets

17.7 Bias-Variance Tradeoff

The bias-variance tradeoff affects model accuracy and overall performance.

17.7.1 Techniques to Manage Bias and Variance

- Bagging
- Boosting
- Random Forests

These techniques help improve model performance by balancing bias and variance.

17.8 Random Forests

Random forests use the bagging technique to train multiple decision trees on bootstrapped data.

17.8.1 Advantages of Random Forests

- Improve accuracy
- Reduce variance
- Provide more stable predictions

17.9 Summary

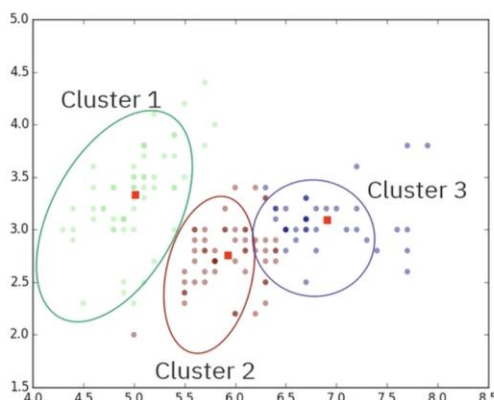
In this module, we learned about classification techniques, decision trees, regression trees, k-NN, and SVM. We also explored the bias-variance trade-off and ensemble methods, such as bagging and random forests, to improve model performance.

Module 4: The Unsupervised Learning Models

18 Clustering Strategies in Real-World Applications

18.1 Overview

Clustering is a machine learning technique that automatically groups data points into clusters based on similarities. It works with unlabeled data and identifies patterns independently to form meaningful groupings.



Applications

Identifying music genres

Segmenting user groups

Analyzing market segments

18.2 Clustering vs Classification

Classification is a supervised learning method that predicts categorical labels using labeled data. For example, a decision tree model can predict whether a customer will default on a loan based on historical labeled data.

Clustering, on the other hand:

- Works with unlabeled data
- Finds hidden patterns independently
- Groups similar data points into clusters

Example:

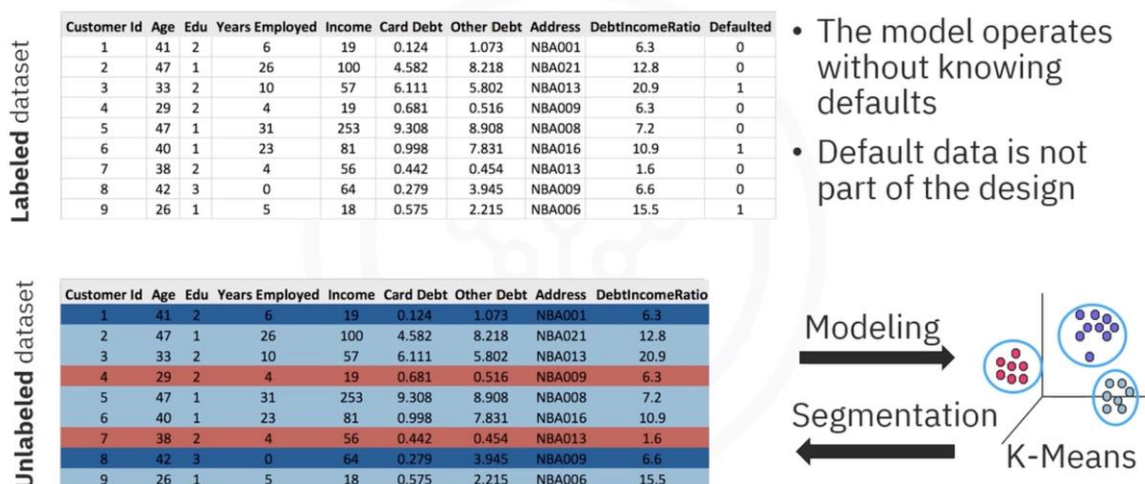
- A k-means clustering model segments customers into groups based on similar characteristics without knowing their loan default status.

18.3 Dataset Example

Consider a dataset containing:

- Historical customer features
- Loan default status

Clustering uses only the features (not labels) to form clusters of similar customers.



18.4 Applications of Clustering

18.4.1 Exploratory Data Analysis

Clustering helps uncover natural groupings in data, such as customer segmentation for targeted marketing.

18.4.2 Pattern Recognition

It improves pattern recognition by grouping similar objects, such as in image segmentation and detecting medical abnormalities.

18.4.3 Anomaly Detection

Clustering identifies outliers, which helps in detecting fraud or equipment malfunctions.

18.4.4 Feature Engineering

Clustering can create new features or reduce dimensionality, improving model performance and interpretability.

18.4.5 Data Summarization

It simplifies data by summarizing it into a smaller number of representative clusters.

18.4.6 Data Compression

Clustering reduces data size by replacing data points with cluster centers, useful in image compression.

18.4.7 Feature Identification

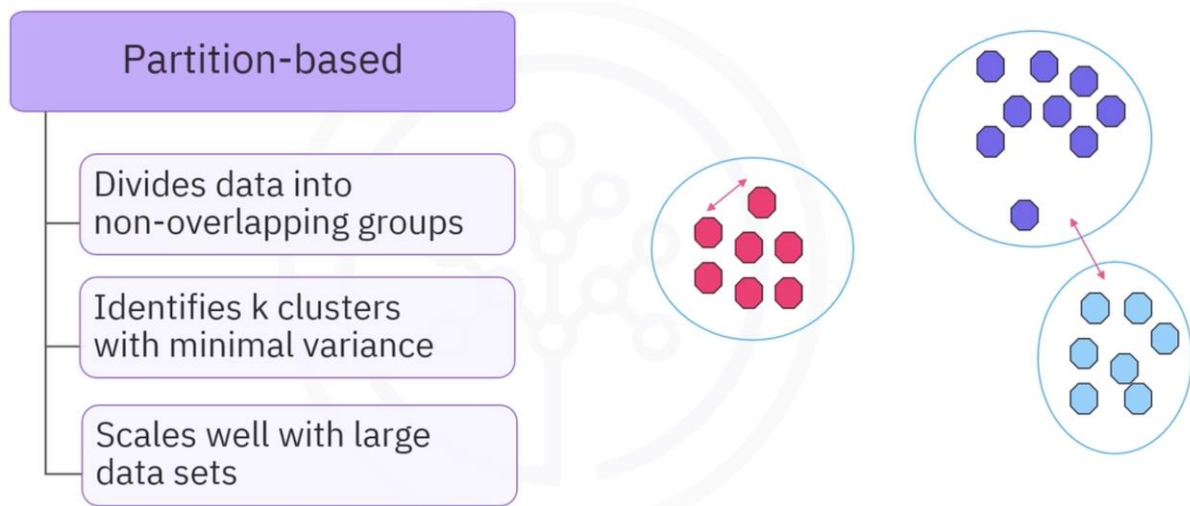
It helps identify essential features that distinguish clusters.

18.5 Types of Clustering Methods

18.5.1 Partition-Based Clustering

Partition-based algorithms divide data into non-overlapping groups.

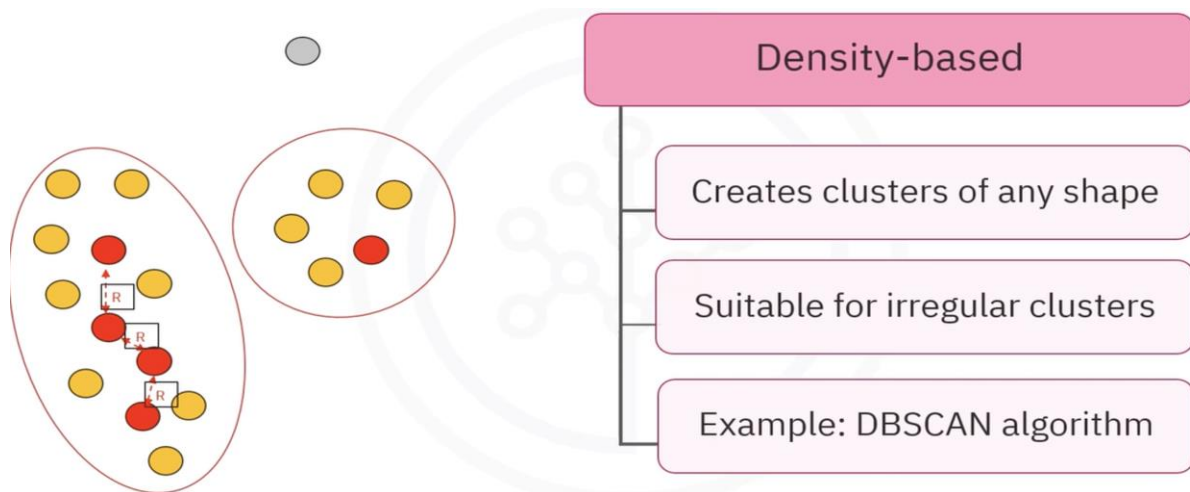
- The most common method is k-means
- It identifies k clusters with minimal variance
- Efficient and scalable for large datasets



18.5.2 Density-Based Clustering

Density-based algorithms form clusters based on data density.

- Suitable for irregularly shaped clusters
- Handles noisy datasets effectively
- Example: DBSCAN algorithm



18.6 Visualization and Examples

18.6.1 Partition-Based vs Density-Based Clustering

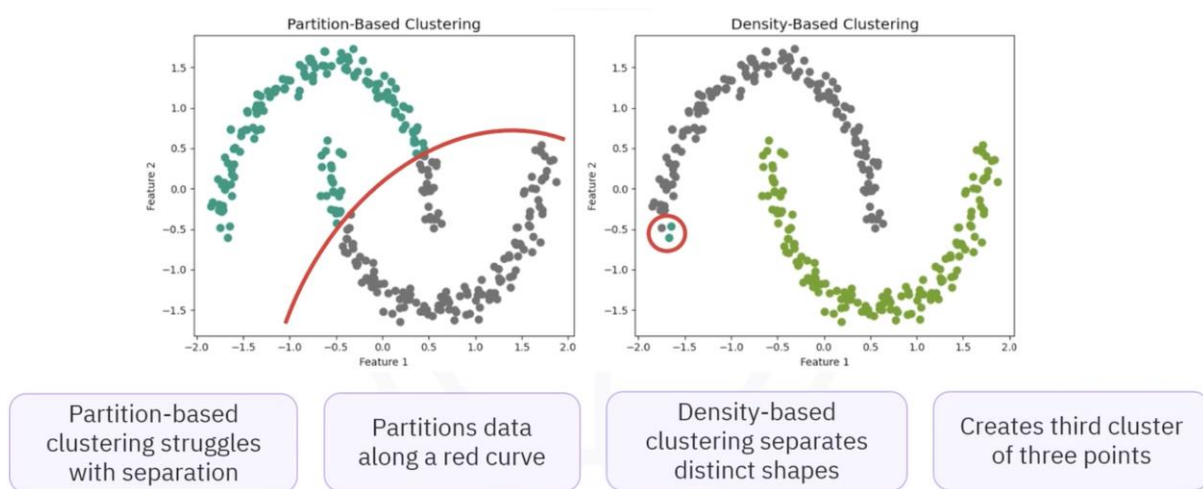
Using Scikit-learn:

- The makeBlobs function generates three distinct clusters
- The makeMoons function generates interlocking half-circles

Observations:

- Partition-based clustering struggles with non-linear shapes

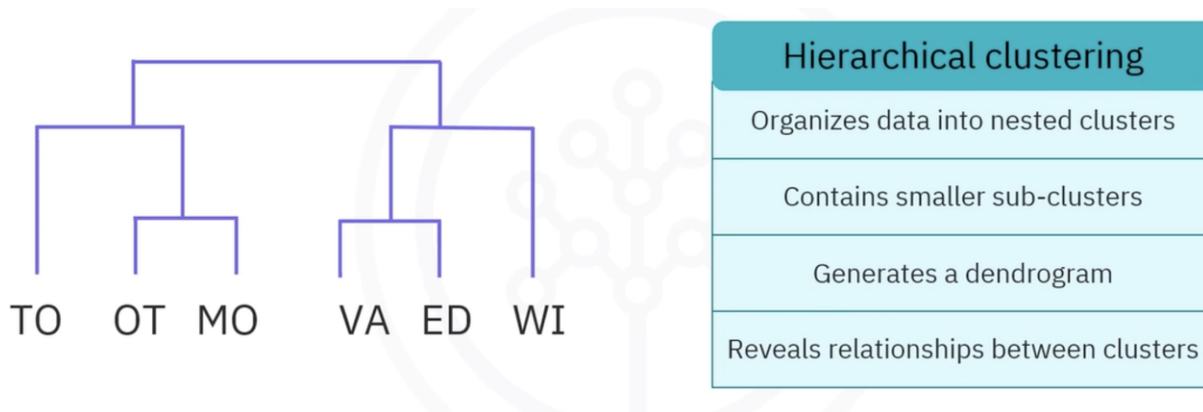
- Density-based clustering successfully separates irregular shapes, but may create extra clusters



18.7 Hierarchical Clustering

Hierarchical clustering organizes data into a tree-like structure of nested clusters.

- Generates a dendrogram to show relationships
- Each cluster contains sub-clusters



18.7.1 Real-World Example: Genetic Data

A study analyzed:

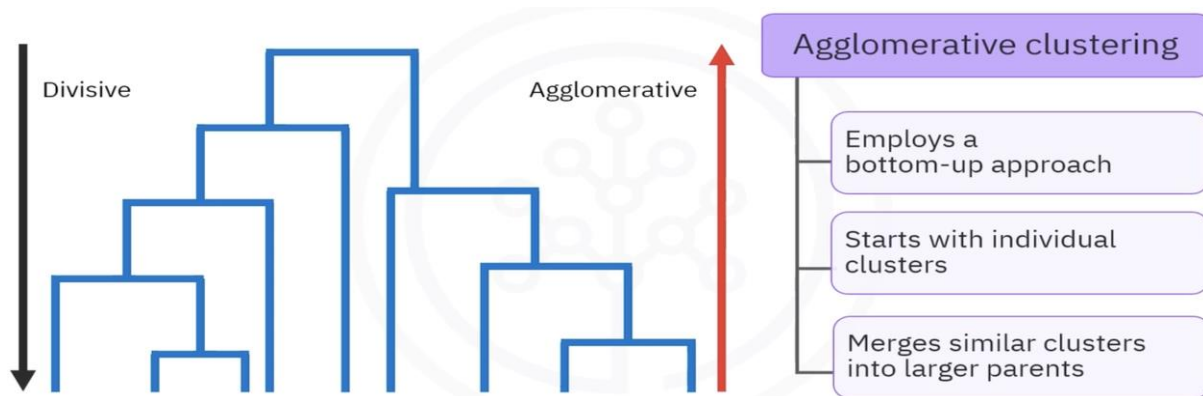
- 900+ dogs
- 85 breeds
- 200 wild grey wolves
- 48,000 genetic markers

Hierarchical clustering grouped animals based on genetic similarity into a tree-like structure (dendrogram), where each node represents clusters of related groups.

18.8 Hierarchical Clustering Strategies

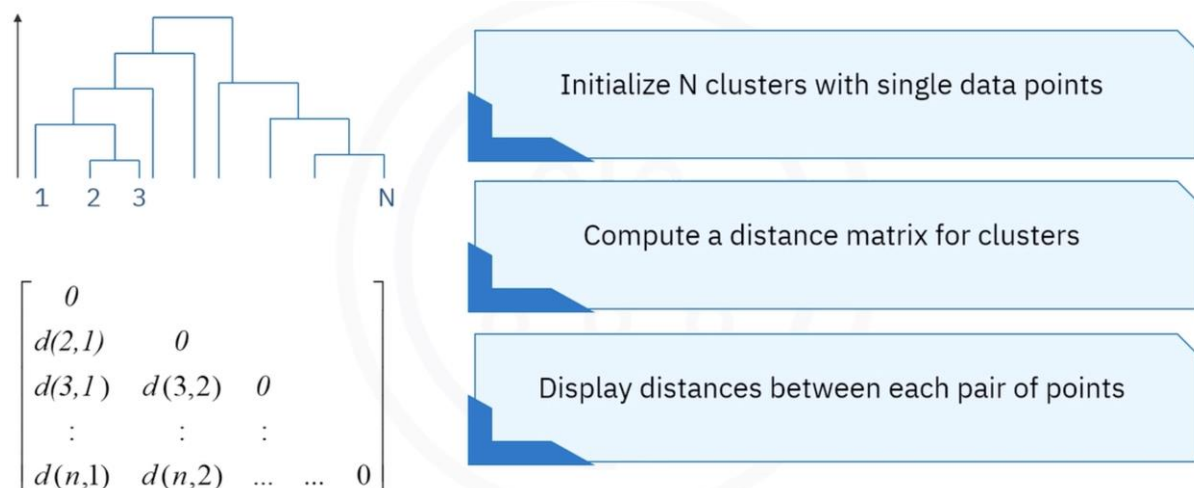
18.8.1 Agglomerative Clustering (Bottom-Up Approach)

- Each data point starts as an individual cluster
- Closest clusters are merged step by step
- Continues until all points form a single cluster or desired clusters are achieved



18.8.1.1 Steps in Agglomerative Clustering

- Select a distance metric (e.g., distance between centroids)
- Initialize N clusters (each data point is its own cluster)
- Compute a distance matrix ($n \times n$ matrix of distances between points)
- Merge the two closest clusters
- Update the distance matrix
- Repeat until the stopping condition is met



18.8.1.2 Agglomerative Clustering Example (Cities)

Consider grouping six Canadian cities by distance.

18.8.1.2.1 Process

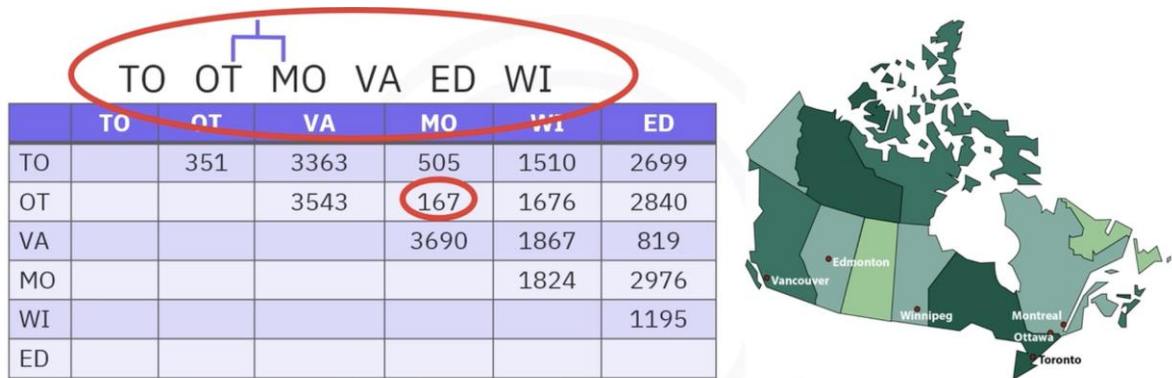
- Each city starts as its own cluster

- A distance matrix shows the distances between each pair

Steps:

- Montreal and Ottawa are the closest → merged
- Ottawa-Montreal cluster merges with Toronto
- Vancouver and Edmonton are merged
- The process continues until all clusters combine

A dendrogram visualizes this hierarchical merging process.



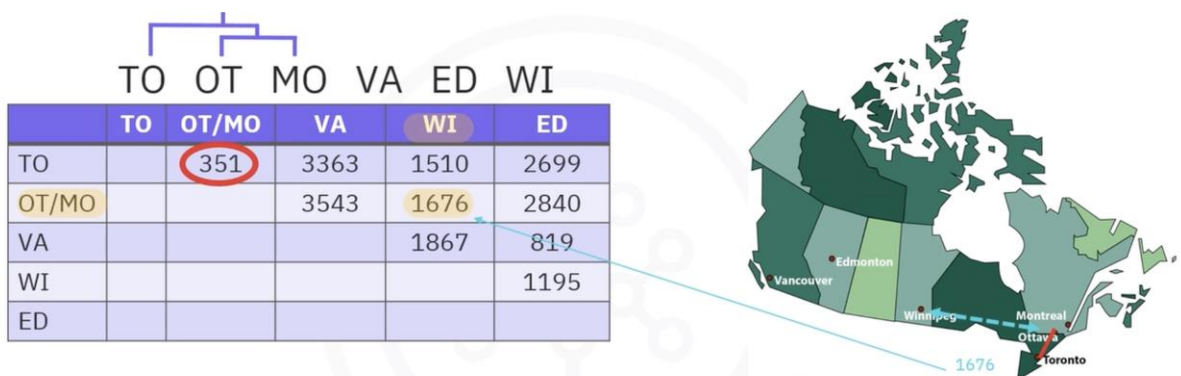
Starts with six clusters for cities

Identifies clusters to merge based on distance

Shows closest clusters in the distance matrix

Combines clusters into the next parent cluster

Visualizes hierarchy with a dendrogram



Distance matrix combines Montreal and Ottawa rows

Creates the Ottawa-Montreal cluster

Updates distances to the new cluster

Calculates midpoint between the two cities

Look for the closest clusters again

Form the Toronto-Ottawa-Montreal cluster

Visualize hierarchy with a dendrogram

	TO	OT	MO	VA	ED	WI
	TO/OT/MO			VA	WI	ED
TO/OT/MO				3543	1676	2840
VA					1867	819
WI						1195
ED						

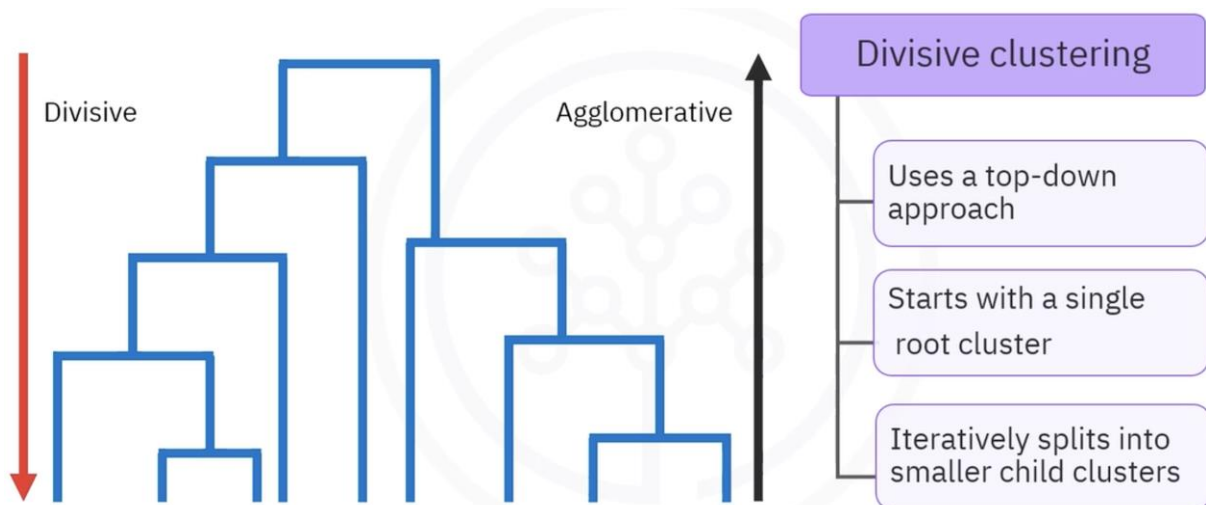


	TO	OT	MO	VA	ED	WI
	TO/OT/MO			VA/ED/WI		
TO/OT/MO						1676
VA/ED/WI						



18.8.2 Divisive Clustering (Top-Down Approach)

- Start with all data points in one cluster
- Split the cluster into smaller clusters
- Continue splitting until a stopping criterion (e.g., minimum cluster size) is reached



18.9 Summary

Clustering is a powerful unsupervised learning technique used to group similar data points.

Key points:

- It works with unlabeled data
- Used in customer segmentation, anomaly detection, and data compression
- Main types include partition-based, density-based, and hierarchical clustering
- K-means is widely used for partition-based clustering
- Hierarchical clustering forms a tree structure using agglomerative (bottom-up) or divisive (top-down) approaches
- Dendrograms help visualize cluster relationships

19 K-Means Clustering

19.1 Overview

K-Means Clustering is an iterative, centroid-based clustering algorithm used to partition a dataset into similar groups based on the distance between their centroids.

19.2 Definition of K-Means

K-Means divides data into **K non-overlapping clusters**, where K is a user-defined parameter.

- Each cluster has a **centroid** (center point)
- Clusters are formed such that:
 - **Within-cluster variance is minimized**
 - **Between-cluster dissimilarity is maximized**

19.3 Concept of Centroid

A centroid represents the **average position of all data points** in a cluster.

- It is typically marked at the center of the cluster
- Data points closest to a centroid belong to that cluster

19.4 Effect of K Value

- **Higher K value:**
 - More clusters
 - Smaller, more detailed groupings
- **Lower K value:**
 - Fewer clusters
 - Larger, less detailed groupings

19.5 K-Means Algorithm Steps

19.5.1 Initialization

- Choose the number of clusters (K)
- Randomly select K initial centroids
- These can be:
 - Existing data points
 - Random points in feature space

19.5.2 Assignment Step

- Compute the distance from each data point to each centroid
- Assign each point to the **nearest centroid**

19.5.3 Update Step

- Recalculate each centroid as the **mean of all points in its cluster**

19.5.4 Iteration

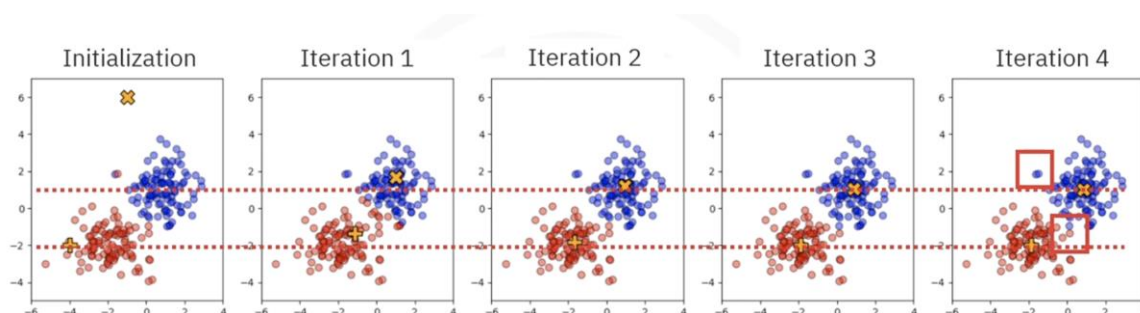
- Repeat assignment and update steps
- Continue until:
 - Centroids stop changing, or
 - Maximum iterations are reached

19.5.5 Convergence

The algorithm converges when centroid positions stabilize and no longer change.

19.6 Working Example of Iterations

- Initial centroids are randomly placed
- With each iteration:
 - Points are reassigned
 - Centroids move closer to optimal positions
- When two consecutive iterations produce the same result, convergence is achieved



19.7 Performance on Balanced vs Imbalanced Data

19.7.1 Balanced Clusters

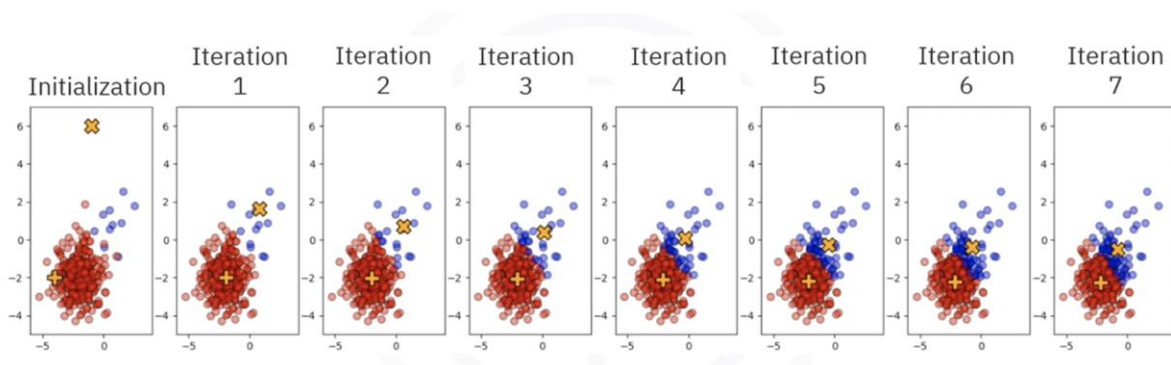
- K-Means performs well when clusters are of similar size and density

19.7.2 Imbalanced Clusters

- Poor performance when cluster sizes differ significantly
- Smaller clusters may get absorbed into larger clusters

Example:

- One cluster has 200 points
- Another has 10 points
- The smaller cluster centroid may drift toward the larger one



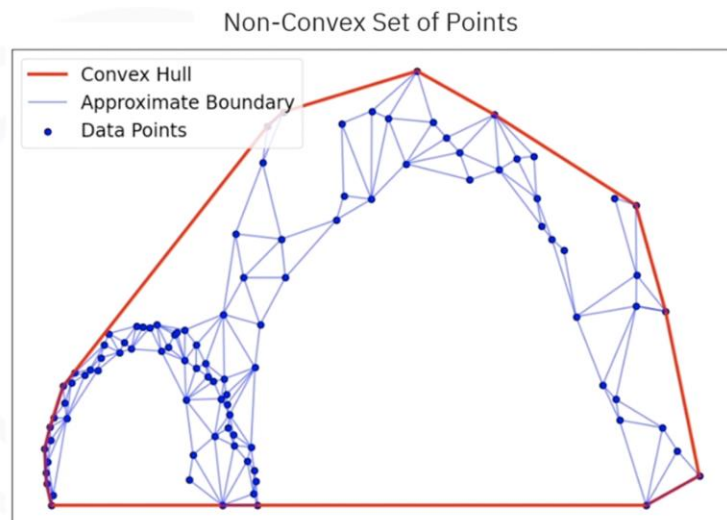
19.8 Assumptions of K-Means

- Clusters are **convex in shape**
- Clusters contain an **approximately equal number of data points**
- Data has **low noise and few outliers**

19.9 Limitations of K-Means

- Sensitive to **outliers and noise**
- Performs poorly on **non-convex clusters**
- Assumes equal cluster sizes
- Requires a predefined value of K

- Assumes convex clusters
- Assumes balanced cluster sizes
- Sensitive to outliers and noise
- Scales well to big data



19.10 Objective Function of K-Means / K-Means Optimization

The goal of K-Means is to minimize the **within-cluster variance**.

This involves minimizing the sum of squared distances between:

- Each data point
- Its corresponding cluster centroid

Goal: Minimize within-cluster sum of squares:

$$\sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

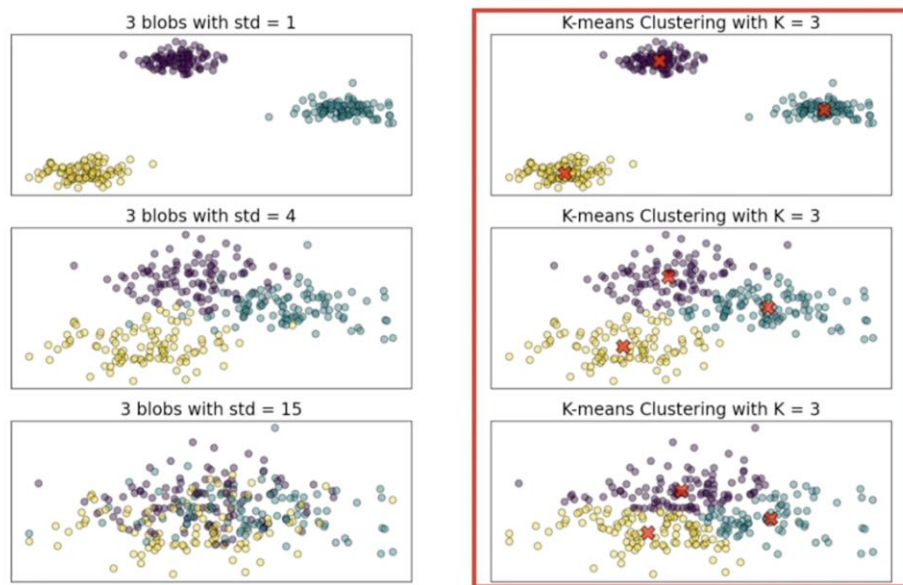
- K = Number of clusters
- C_i = i^{th} cluster
- x = Data point
- μ_i = Centroid of cluster C_i
- $\|x - \mu_i\|^2$ = Squared distance between x and its cluster centroid

19.11 Experimental Observations

19.11.1 Effect of Standard Deviation

- Low standard deviation:
 - Clusters are well-separated
 - K-Means performs well
- High standard deviation:
 - Clusters overlap
 - Accuracy decreases
- Very high standard deviation:
 - Data becomes indistinguishable
 - Clustering results are unreliable

K-Means Experiments: $K = 3$

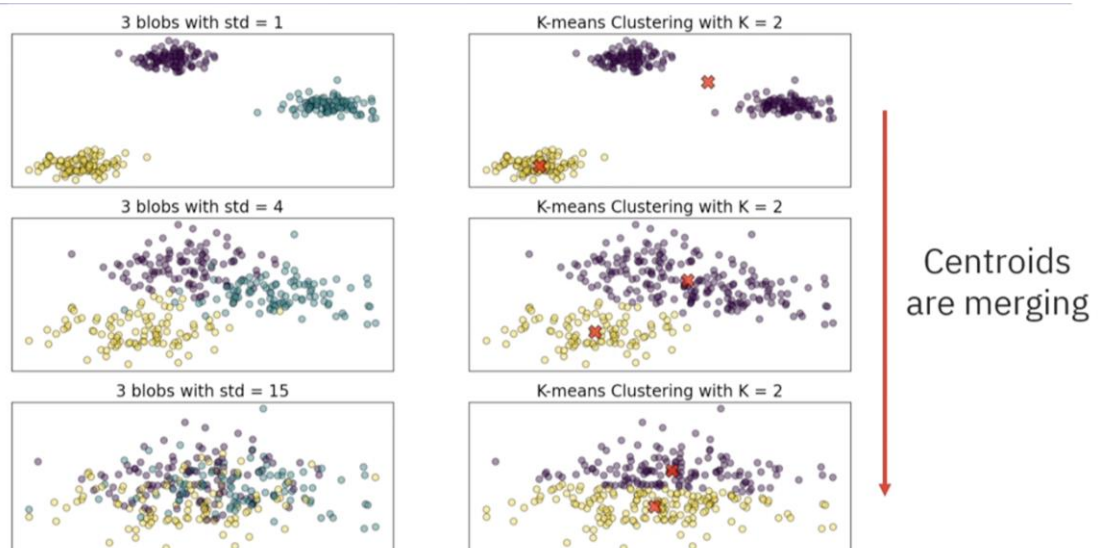


19.11.2 Effect of Incorrect K Value

19.11.2.1 K Less Than True Clusters

- Multiple actual clusters may merge into one
- Centroids are placed between clusters

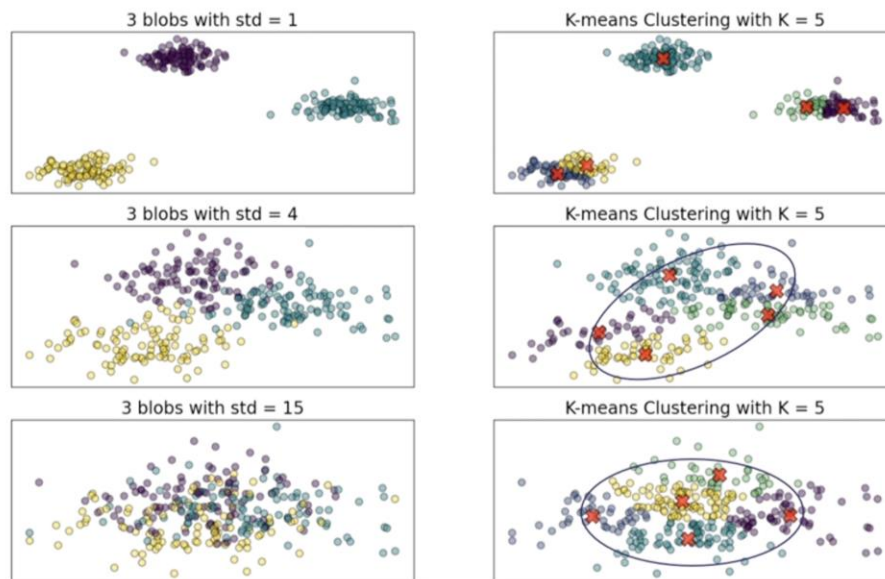
K-Means Experiments: $K = 2$



19.11.2.2 K Greater Than True Clusters

- Clusters are unnecessarily split
- Results become less meaningful

K-Means Experiments: $K = 5$



19.12 Choosing the Optimal Value of K

Selecting the right K is challenging, especially for complex datasets.

19.12.1 Data Separability

- Easier in low dimensions (2D, 3D)
- Difficult in high-dimensional data

19.12.2 Heuristic Methods

19.12.2.1 Silhouette Analysis

- Measures how similar a point is to:
 - Its own cluster (cohesion)
 - Other clusters (separation)

19.12.2.2 Elbow Method

- Plots the objective function against different K values
- The “elbow point” indicates optimal K

19.12.2.3 Davies-Bouldin Index

- Measures the similarity between clusters
- Lower values indicate better clustering

19.13 Advantages of K-Means

- Simple and easy to implement

- Efficient and scalable for large datasets
- Fast convergence

19.14 Summary

K-Means is an iterative clustering algorithm that partitions data into K clusters using centroid-based grouping. It minimizes within-cluster variance and assigns points based on distance to centroids. While efficient and widely used, it assumes convex clusters and struggles with imbalanced data, noise, and incorrect K selection. Methods like silhouette analysis, the elbow method, and the Davies-Bouldin index help evaluate clustering performance.

20 DBSCAN and HDBSCAN Clustering

20.1 Overview

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) and HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) are clustering algorithms used to identify patterns in data based on density.

20.2 DBSCAN Clustering

20.2.1 Definition

DBSCAN is a density-based spatial clustering algorithm that creates clusters based on a user-defined density value.

20.2.2 Key Concepts

- Density is defined around a spatial centroid
- The surrounding area is called a neighborhood
- DBSCAN identifies clusters by grouping points in high-density regions

20.2.3 Capabilities

- Can discover clusters of any shape and size
- Can handle arbitrary patterns in data
- Can distinguish between cluster points and noise

20.2.4 When to Use

DBSCAN is useful when:

- The dataset contains noise or outliers
- The number of clusters is unknown
- The data has irregular or complex shapes

20.3 Comparison with Centroid-Based Clustering

20.3.1 Centroid-Based Algorithms

Examples:

- K-Means
- Hierarchical Clustering

Characteristics:

- Produce spherical or convex clusters
- Assign every point to a cluster
- Perform well on simple, well-structured data

20.3.2 Limitations

- Cannot handle arbitrary shapes
- Poor handling of outliers
- Real-world data rarely follows simple patterns

20.3.3 Advantage of Density-Based Clustering

Density-based clustering:

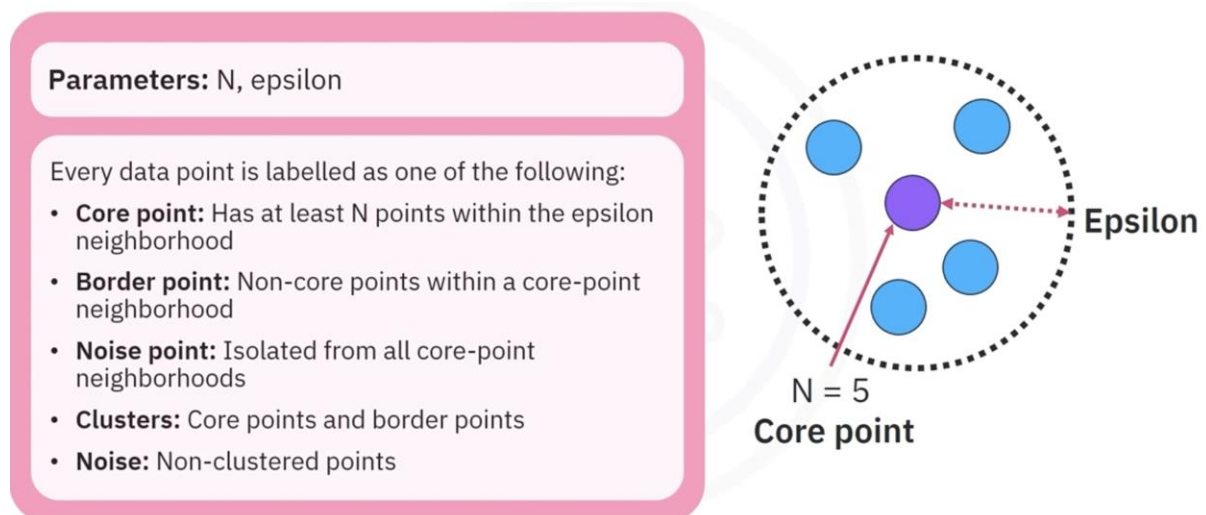
- Identifies connected regions of high density
- Handles noise effectively
- Captures complex shapes and structures

20.4 How DBSCAN Works

20.4.1 Parameters

DBSCAN requires two parameters:

- n (minimum number of points in a neighborhood)
- ϵ (epsilon: radius of the neighborhood)



20.4.2 Point Classification

Each point in the dataset is labeled as:

20.4.2.1 Core Point

- Has at least n points (including itself) within radius ϵ
- Acts as the center of a cluster

20.4.2.2 Border Point

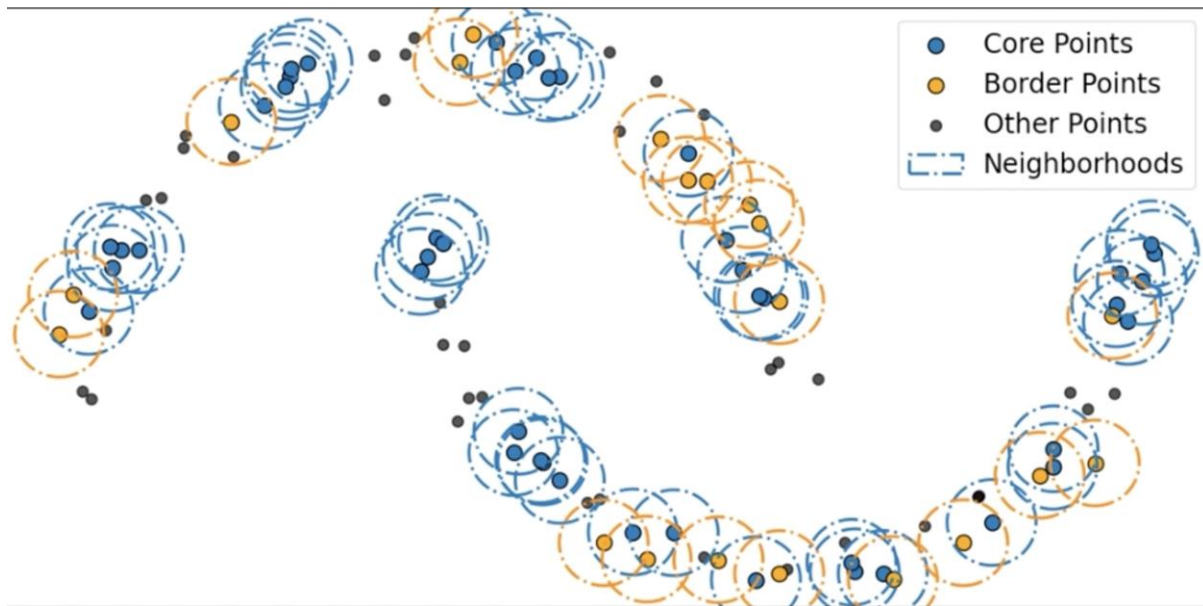
- Lies within the neighborhood of a core point
- Does not have enough neighbors to be a core point

20.4.2.3 Noise Point

- Does not belong to any cluster
- Is isolated from all core point neighborhoods

20.4.3 Cluster Formation

- Clusters are formed by expanding from core points
- Neighboring points are added to the cluster
- Border points are included, but are less densely connected
- Remaining unassigned points are labeled as noise



20.4.4 Important Property

DBSCAN is not iterative:

- It processes data in one pass
- Once a point is labeled, it is not updated

20.5 DBSCAN Example Explanation

20.5.1 Dataset

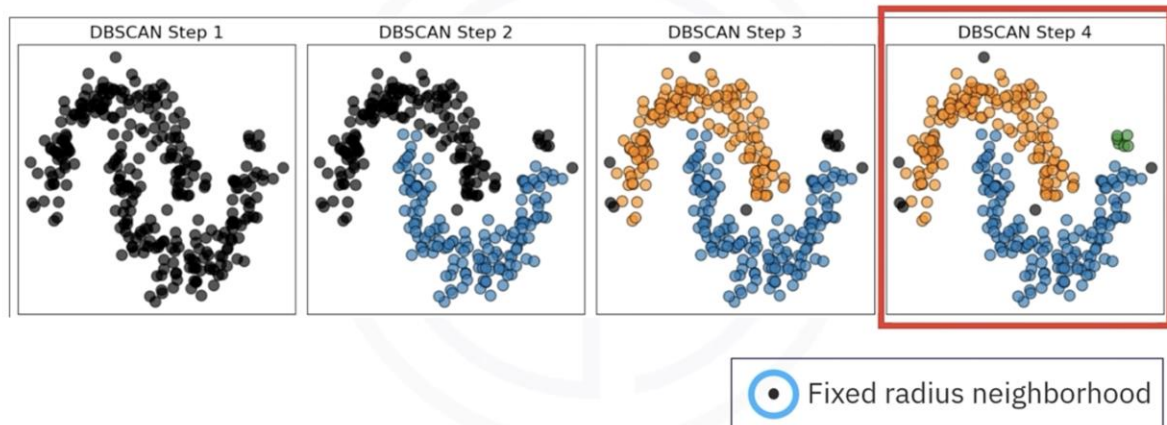
- Two noisy datasets generated using the half-moons function

20.5.2 Observations

- Core points (blue): Have at least $n = 4$ neighbors within ϵ
- Border points (orange): Belong to neighborhoods but are not dense enough
- Noise points (black): Remain unassigned

20.5.3 Clustering Process

- Initially, all points are treated as noise
- The algorithm identifies clusters step by step
- Expands clusters from core points
- Leaves isolated points as noise



20.5.4 Result

- Successfully identifies clusters of complex shapes
- Effectively separates noise from meaningful data

20.6 HDBSCAN Clustering

20.6.1 Definition

HDBSCAN is an advanced version of DBSCAN that does not require manual parameter tuning and is more flexible.

20.6.2 Key Features

- Less sensitive to noise and outliers
- Uses cluster stability instead of fixed density parameters
- Produces more robust and meaningful clusters

20.7 Cluster Stability

Cluster stability refers to:

- The ability of a cluster to remain consistent across different neighborhood sizes

HDBSCAN:

- Evaluates clusters over a range of density thresholds
- Selects clusters that remain stable

20.8 How HDBSCAN Works

20.8.1 Process

- Starts with each point as its own cluster (treated as noise)
- Gradually merges clusters by lowering density thresholds
- Builds a hierarchical clustering tree
- Converts it into a condensed tree

- Retains only the most stable clusters

20.8.2 Nature of Algorithm

- A combination of agglomerative clustering and density-based clustering

20.9 DBSCAN vs HDBSCAN

20.9.1 DBSCAN

- Requires parameters (n and ϵ)
- Fixed neighborhood size
- Sensitive to parameter selection

20.9.2 HDBSCAN

- No strict parameter requirement
- Adapts to varying densities
- Produces more detailed and stable clusters

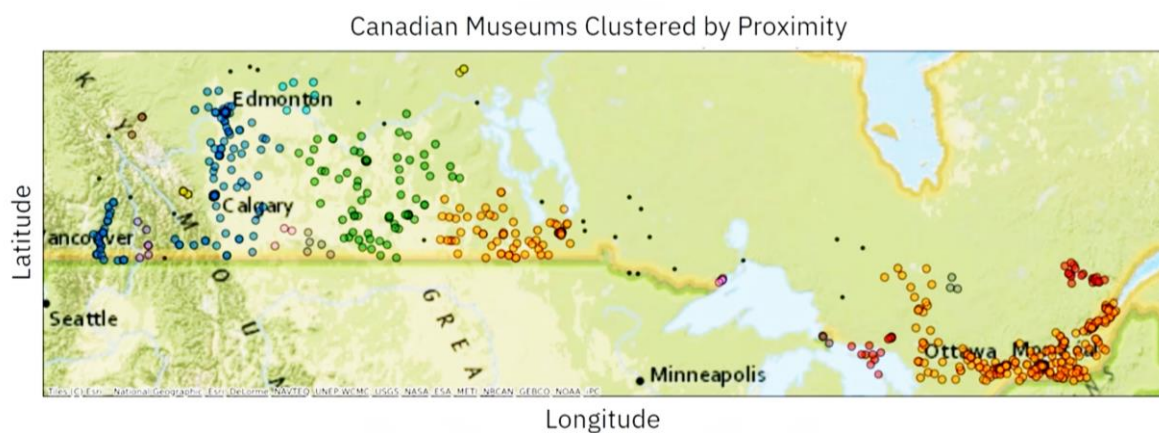
20.10 Real-World Example

20.10.1 Dataset

- Latitude and longitude data of Canadian museums

20.10.2 DBSCAN Result

- Parameters:
 - Minimum samples = 3
 - Radius = 0.15
- Found approximately 10 clusters
- Dense regions grouped into single clusters



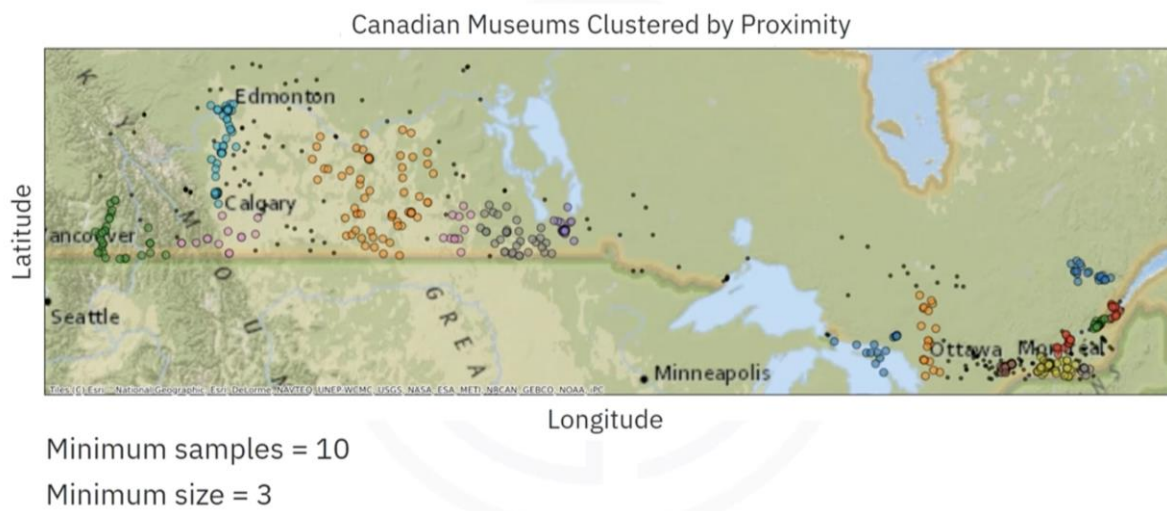
Minimum Samples = 3

Epsilon = 0.75

20.10.3 HDBSCAN Result

- Parameters:
 - Minimum samples = 10
 - Minimum cluster size = 3
- Identified more distinct clusters
- Adapted to varying densities
- Captured finer details in dense regions

HDBSCAN Adjusted Result



20.10.4 Observation

- HDBSCAN provided more detailed and meaningful clustering
- Better handled variations in density across regions

20.11 Advantages of HDBSCAN

- Automatically adapts to local density variations
- Provides better cluster separation
- Handles noise more effectively
- Produces coherent and interpretable results

20.12 Summary

DBSCAN is a density-based clustering algorithm that groups data points based on user-defined density parameters. It is effective for identifying clusters of arbitrary shapes and handling noise.

HDBSCAN extends DBSCAN by removing the need for strict parameter tuning and introducing cluster stability. It produces more robust, adaptive, and meaningful clusters, especially in complex real-world datasets.

21 Clustering, Dimension Reduction, and Feature Engineering

21.1 Overview

Clustering, dimension reduction, and feature engineering are complementary techniques in machine learning and data science. These methods work together to improve:

- Model performance
- Computational efficiency
- Scalability
- Interpretability
- Data visualization quality

After studying this topic, we understand:

- Clustering and its role in machine learning
- Dimension reduction and its importance
- Feature engineering techniques
- How these methods work together to improve predictive modeling

21.2 Relationship Between Clustering, Dimension Reduction, and Feature Engineering

21.2.1 Clustering

Clustering helps:

- Group similar data points
- Support feature selection
- Create new features
- Improve computational efficiency and scalability

21.2.2 Dimension Reduction

Dimension reduction:

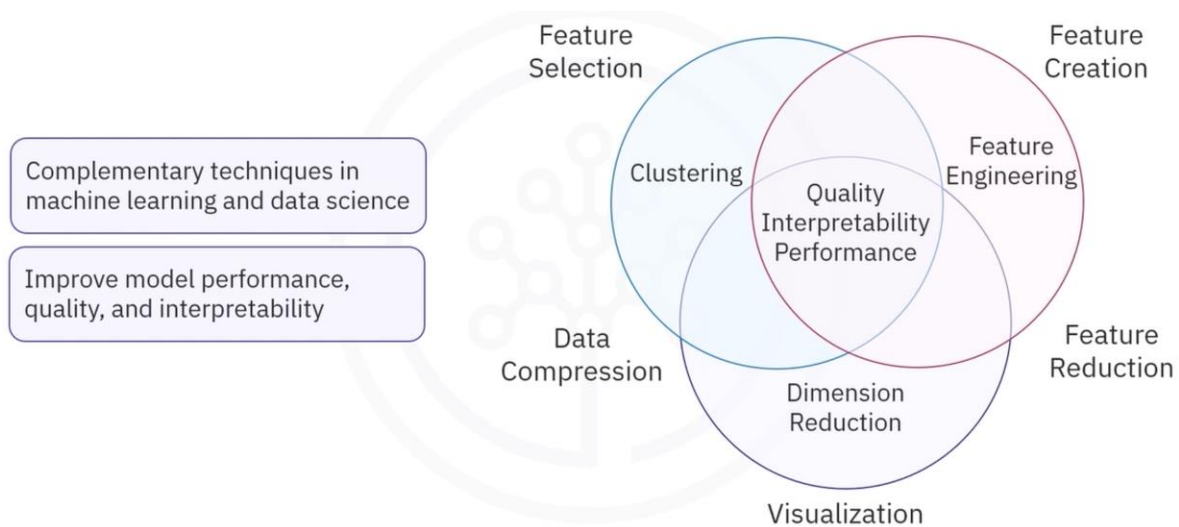
- Simplifies high-dimensional data
- Reduces the number of features
- Improves clustering performance
- Enhances visualization and interpretability

21.2.3 Feature Engineering

Feature engineering:

- Selects useful features
- Removes redundant information
- Creates transformations and interactions between variables

- Improves predictive model quality



21.3 Dimension Reduction

21.3.1 Purpose of Dimension Reduction

Dimension reduction reduces the number of variables in a dataset while preserving important information.

Benefits include:

- Simplified data structures
- Faster computations
- Improved clustering outcomes
- Better visualization
- Reduced storage requirements

21.3.2 High-Dimensional Data Challenges

Distance-based clustering algorithms such as:

- K-Means
- DBSCAN

face difficulties with high-dimensional data.

As dimensionality increases:

- Data volume expands rapidly
- Data points become sparse
- Similarity between observations decreases
- Clusters become smaller and harder to identify

This problem is commonly known as the curse of dimensionality.

21.3.3 Dimension Reduction Techniques

Common techniques include:

- PCA (Principal Component Analysis)
- t-SNE (t-Distributed Stochastic Neighbor Embedding)
- UMAP (Uniform Manifold Approximation and Projection)

These methods reduce dimensions before clustering, improving efficiency and model performance.

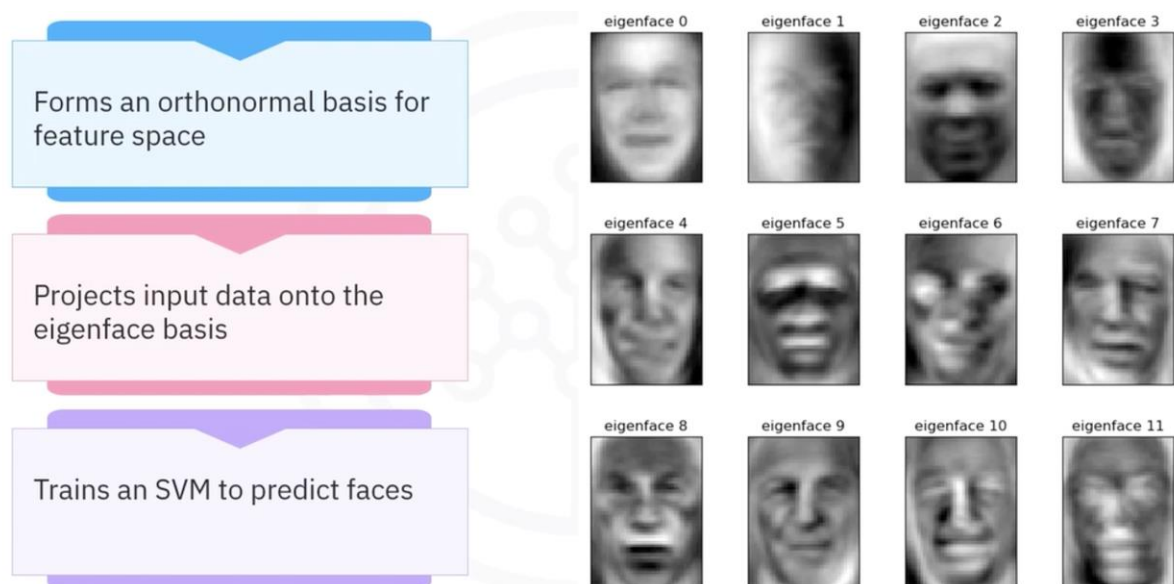
21.4 PCA and Face Recognition

21.4.1 Eigenfaces for Face Recognition

An example application of dimension reduction is face recognition using eigenfaces.

Process:

1. PCA is applied to an unlabeled face dataset
2. The top 150 eigenfaces are extracted from 966 faces
3. These eigenfaces form an orthonormal basis for the feature space
4. Input images are projected onto the eigenface basis
5. An SVM (Support Vector Machine) is trained for face prediction

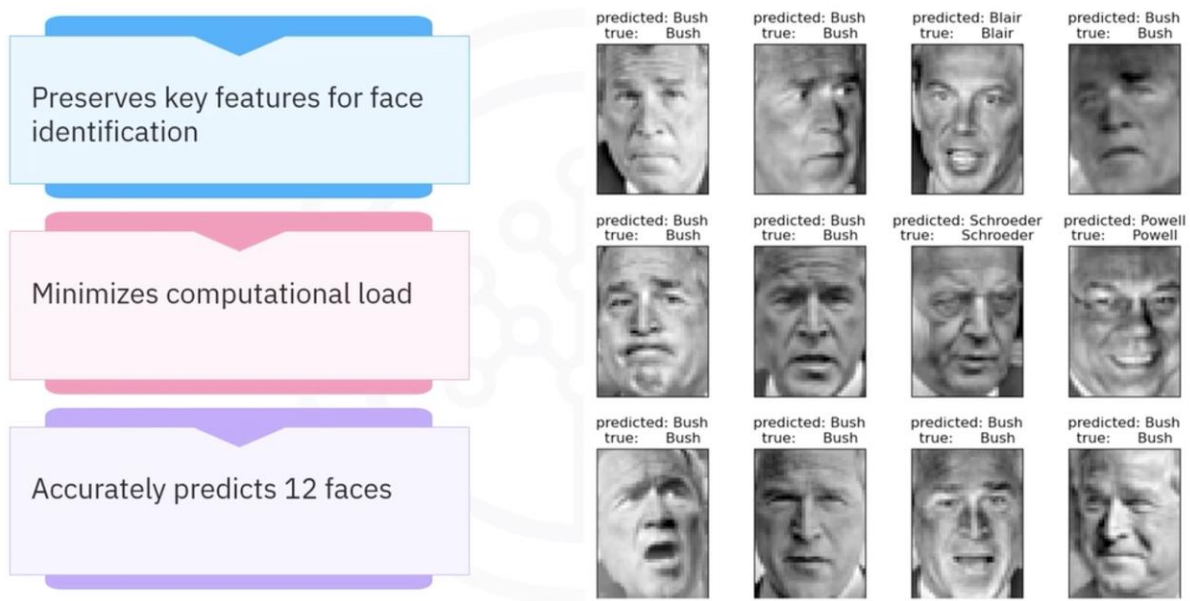


21.4.2 Benefits

Dimension reduction:

- Preserves important facial features
- Reduces computational load
- Improves recognition efficiency

The face recognition model successfully predicts 12 faces accurately.



21.5 Visualization of High-Dimensional Clusters

21.5.1 Problem

Clustering results cannot be directly visualized in feature spaces beyond three dimensions.

21.5.2 Solution

Advanced dimension reduction techniques project high-dimensional data into:

- Two dimensions
- Three dimensions

Techniques used:

- PCA
- t-SNE
- UMAP

21.5.3 Advantages

These projections:

- Improve visual interpretation
- Enable scatter plot visualization
- Help evaluate clustering quality
- Reveal hidden patterns and relationships

Reducing dimensionality often improves cluster interpretability.

21.6 Clustering for Feature Selection

21.6.1 Clustering Features

Clustering can be applied not only to observations but also to features.

By clustering similar or correlated features:

- Redundant information can be identified
- Representative features can be selected
- The total feature count can be reduced

This process is part of:

- Feature selection
- Feature engineering
- Dimension reduction

21.6.2 Feature Engineering Decisions

Clustering may reveal:

- Distinct subgroups in data
- Useful feature interactions
- Beneficial feature transformations

These insights improve predictive modeling.

21.7 K-Means Feature Clustering Example

21.7.1 Dataset Simulation

A simulation uses K-Means clustering on features.

Five features are generated using random normal distributions:

- Mean values: 1, 5, and 10
- Variance: 1 for most features
- Feature 4 variance: 2

21.7.2 Observations

- Features 1 through 3 are statistically very similar
- Features 4 and 5 are different from the others

21.7.3 Clustering Results

Running K-Means with:

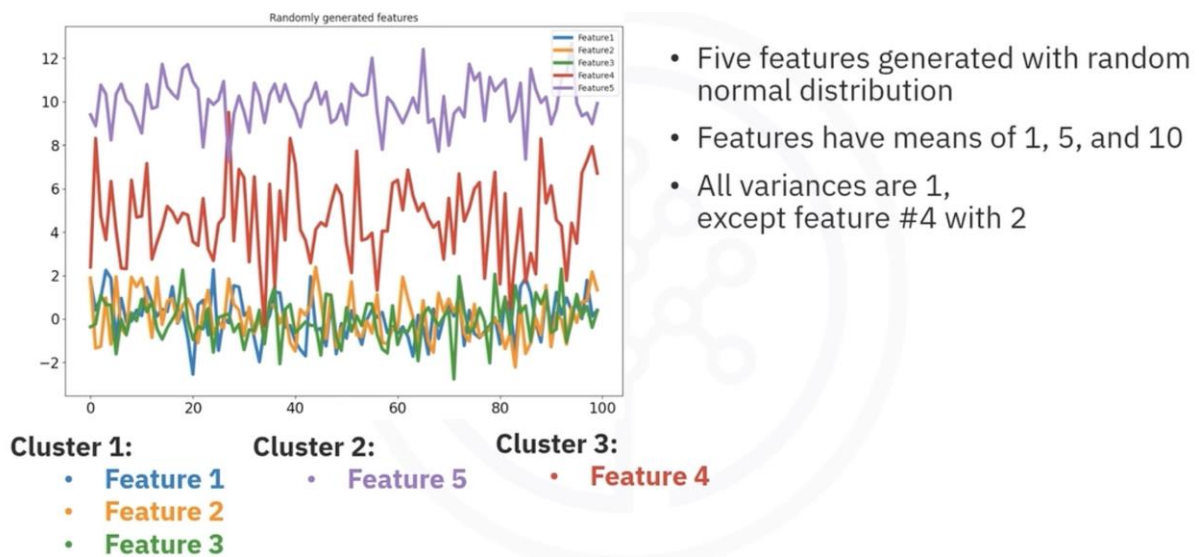
$k = 3$

correctly clusters the features.

Cluster 1 contains redundant features. During modeling, only one representative feature should be selected from this cluster.

This demonstrates:

- Feature selection
- Reduction of redundant information
- Dimension reduction through clustering



21.8 Summary

Clustering, dimension reduction, and feature engineering work together to improve:

- Model performance
- Data quality
- Interpretability
- Computational efficiency

Dimension reduction simplifies high-dimensional datasets and improves clustering results.

Techniques such as PCA, t-SNE, and UMAP are commonly used to reduce dimensions and visualize complex datasets.

Clustering can also support feature engineering by identifying redundant features and enabling feature selection.

K-Means clustering on features demonstrates how similar variables can be grouped, helping reduce dataset complexity while preserving important information.

22 Dimension Reduction Algorithms

22.1 Overview

Dimensionality Reduction Algorithms reduce the number of dataset features without sacrificing critical dataset information.

High-dimensional data is often difficult to:

- Analyze
- Visualize
- Process efficiently using machine learning models

Dimensionality Reduction Algorithms simplify datasets and improve machine learning performance by transforming original dimensions into a smaller set of meaningful features.

The major dimensionality reduction algorithms include:

- Principal Component Analysis (PCA)
- T-Distributed Stochastic Neighbor Embedding (t-SNE)
- Uniform Manifold Approximation and Projection (UMAP)

22.2 Principal Component Analysis (PCA)

22.2.1 Definition

Principal Component Analysis (PCA) is a linear dimensionality reduction algorithm that assumes dataset features are linearly correlated.

22.2.2 Purpose of PCA

PCA is used to:

- Simplify data
- Reduce dimensionality
- Reduce noise
- Minimize information loss

22.2.3 Working of PCA

PCA transforms features into a new set of uncorrelated variables called principal components.

These principal components:

- Are orthogonal to each other
- Define a new coordinate system for the feature space
- Retain as much variance as possible

22.2.4 Principal Components

The principal components are arranged in decreasing order of importance based on how much variance they explain.

- The first few principal components contain most of the useful information
- The remaining components often represent noise

22.3 T-Distributed Stochastic Neighbor Embedding (t-SNE)

22.3.1 Definition

T-Distributed Stochastic Neighbor Embedding (t-SNE) is a nonlinear dimensionality reduction algorithm that maps high-dimensional data into a lower-dimensional space.

22.3.2 Characteristics of t-SNE

t-SNE:

- Works well for visualization in two or three dimensions
- Is effective for image and text datasets
- Identifies clusters in complex high-dimensional data

22.3.3 Working of t-SNE

t-SNE focuses on preserving the similarity of nearby points.

Similarity is measured using:

- Distance between pairs of points
- Proximity of neighboring data points

22.3.4 Limitations of t-SNE

- Difficult to scale for large datasets
- Sensitive to hyperparameters
- Requires careful tuning

22.4 Uniform Manifold Approximation and Projection (UMAP)

22.4.1 Definition

Uniform Manifold Approximation and Projection (UMAP) is a nonlinear dimensionality reduction algorithm commonly used as an alternative to t-SNE.

22.4.2 Working of UMAP

UMAP:

- Constructs a high-dimensional graph representation of data

- Uses manifold theory
- Assumes data lies on a lower-dimensional manifold embedded in a higher-dimensional space

UMAP then optimizes a low-dimensional graph structure that preserves the relationships between points in the original dataset.

22.4.3 Advantages of UMAP

UMAP:

- Scales better than t-SNE
- Preserves both local and global structure
- Often provides better clustering performance than t-SNE

22.5 Comparison of PCA, t-SNE, and UMAP

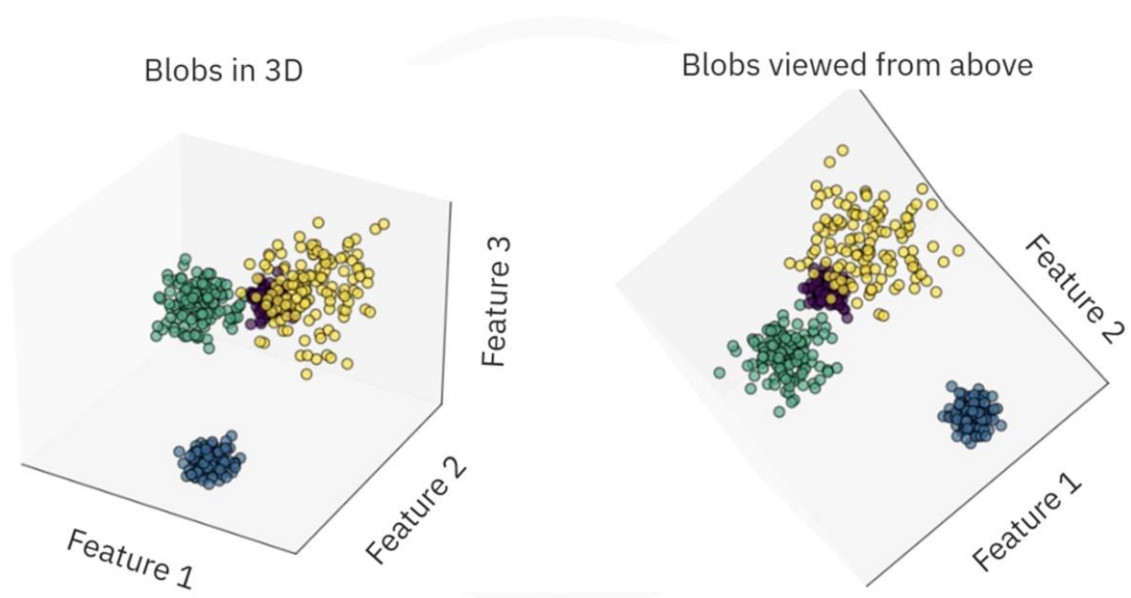
22.5.1 Simulated Dataset Example

Consider simulated 3D data generated using the MakeBlobs function in Scikit-Learn.

Observations:

- Slight overlap exists between the yellow and purple clusters
- Other clusters are clearly separated

The PCA, t-SNE, and UMAP algorithms are applied to project the dataset into two dimensions.



22.6 PCA Results

22.6.1 Performance

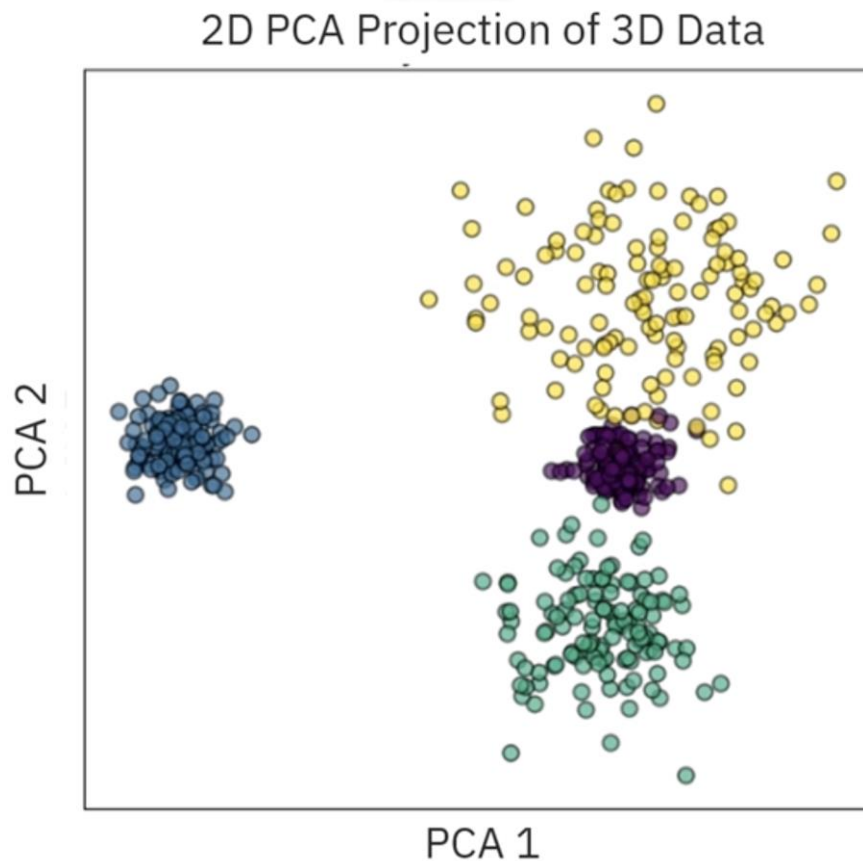
PCA separated the blobs effectively.

22.6.2 Reason

The blobs:

- Were normally distributed
- Differed mainly in means and variances
- Had linear correlations

Because of these characteristics, PCA performed well on the dataset.



22.7 t-SNE Results

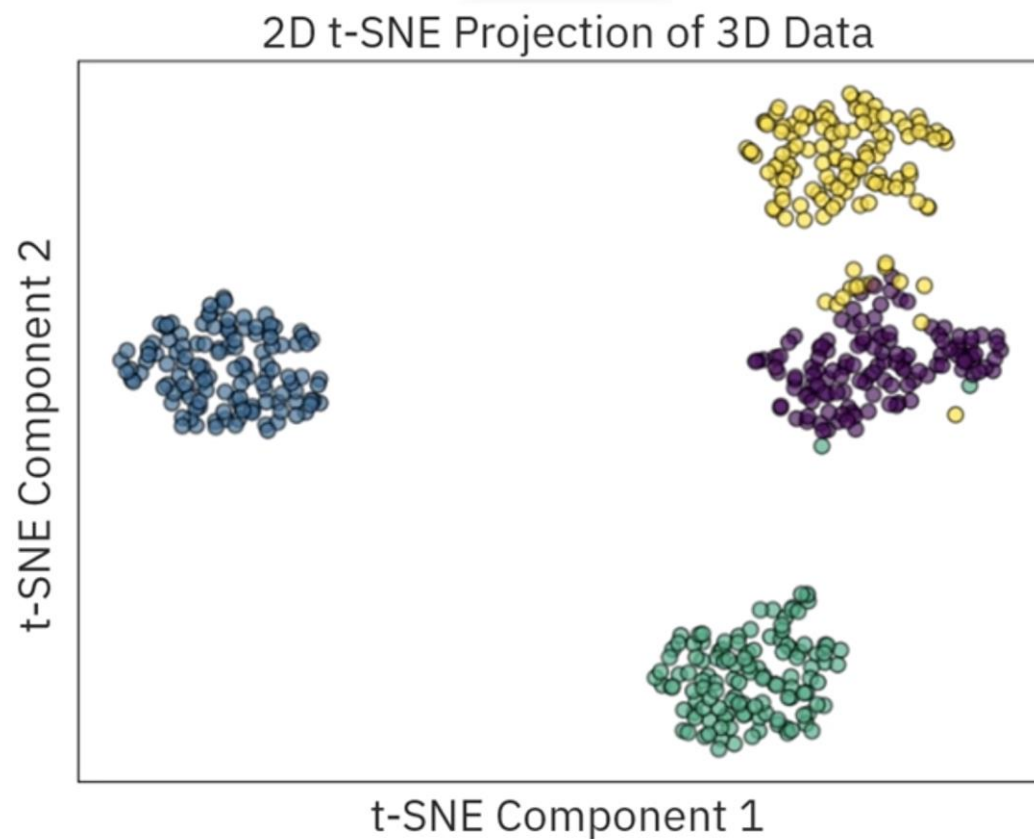
22.7.1 Performance

t-SNE clustered the data into four distinct clusters and mostly separated the blobs successfully.

22.7.2 Observations

- Some mislabeled points appeared within the purple cluster
- Mixing occurred between green and yellow clusters
- Slight overlap existed in the original data

This overlap caused some confusion in clustering, which is expected behavior for t-SNE.



22.8 UMAP Results

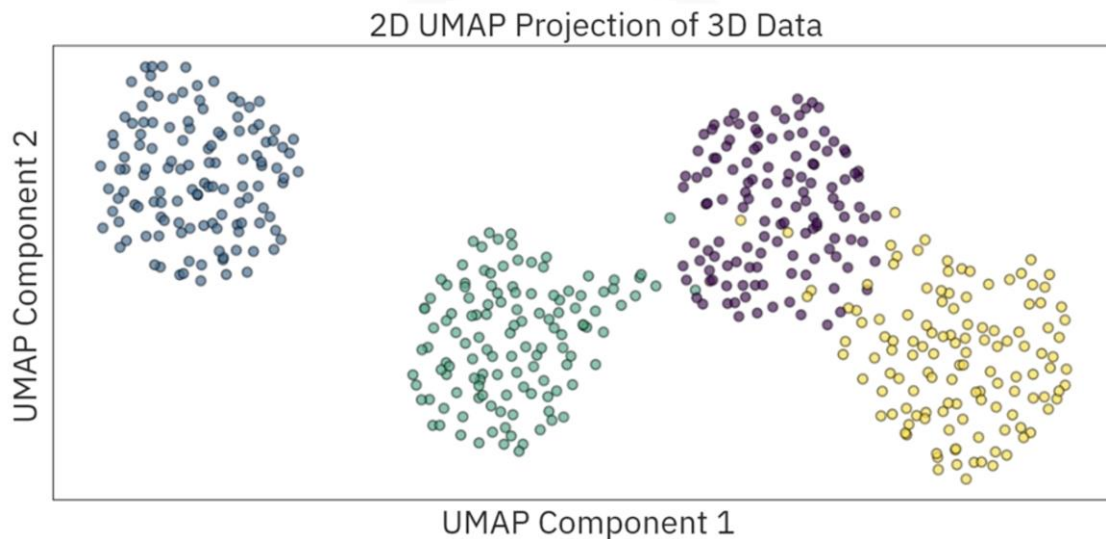
22.8.1 Performance

UMAP did not perfectly separate all clusters, but preserved the true structure of the data more effectively.

22.8.2 Observations

- Yellow and green clusters slightly overlapped with the purple cluster
- This overlap reflected the original 3D dataset structure
- UMAP preserved both local and global relationships better than t-SNE

UMAP performed slightly better because it represented the natural overlap in the original data rather than forcing completely distinct clusters.



22.9 Key Differences Between PCA, t-SNE, and UMAP

Algorithm	Type	Main Purpose	Advantages	Limitations
PCA	Linear	Reduce dimensionality while preserving variance	Fast, simple, reduces noise	Cannot capture nonlinear relationships
t-SNE	Nonlinear	Visualize clusters in high-dimensional data	Excellent cluster visualization	Slow and difficult to tune
UMAP	Nonlinear	Preserve manifold structure in lower dimensions	Faster, scalable, preserves global structure	May not perfectly separate clusters

22.10 Summary

1. Dimensionality Reduction Algorithms reduce the number of dataset features while preserving important information.
2. The major algorithms are:
 - PCA
 - t-SNE
 - UMAP
3. PCA is a linear dimensionality reduction algorithm that simplifies data and minimizes information loss.
4. t-SNE maps high-dimensional data points into a lower-dimensional space and is effective for cluster visualization.
5. UMAP approximates the manifold structure of data and preserves both local and global relationships while scaling efficiently for larger datasets.

22.10.1 What is a “manifold structure”?

A manifold means:

High-dimensional data actually lies on a lower-dimensional hidden shape.

Imagine this example:

A paper sheet exists in 3D space, but the sheet itself is still basically 2D.

Similarly:

- data may exist in 100 dimensions
- but its true structure may behave like 2D or 3D

UMAP tries to discover this hidden shape.

Example:

Suppose customer behavior data has:

- age
- salary
- purchases
- location
- interests
- device usage
- etc.

Maybe all those variables actually follow a few hidden patterns.

UMAP tries to uncover those hidden patterns.

23 Module 4 Summary and Highlights

23.1 Fundamentals of Clustering

Clustering is a core unsupervised machine learning technique used to group data points based on similarity.

It is widely used in real-world applications such as:

- Customer Segmentation: Grouping customers based on similar purchasing behavior
- Anomaly Detection: Identifying unusual patterns or outliers in data

Clustering helps reveal hidden structures within datasets without predefined labels.

23.2 K-Means Clustering and Performance Assessment

K-Means is a popular partition-based clustering algorithm that:

- Group data based on distance from the centroids
- Assigns each point to the nearest cluster center

However, it has limitations:

- Performs poorly on imbalanced datasets
- Struggles with non-convex (irregularly shaped) clusters

23.2.1 Evaluation Techniques for K-Means

To assess clustering performance, the following methods are used:

23.2.1.1 Silhouette Analysis

Measures how similar a data point is to its own cluster compared to other clusters.

23.2.1.2 Elbow Method

Identifies the optimal number of clusters by observing where the rate of decrease in variance slows down.

23.2.1.3 Davies-Bouldin Index

Evaluates the average similarity between each cluster and its most similar cluster.

23.3 Density-Based Clustering Algorithms

Density-based clustering focuses on regions of high data point concentration rather than fixed centroids.

23.3.1 DBSCAN

DBSCAN groups points based on density and is effective for:

- Detecting irregularly shaped clusters
- Handling noise and outliers

23.3.2 HDBSCAN

HDBSCAN is an advanced version of DBSCAN that:

- Does not require manual parameter tuning
- Uses cluster stability for grouping
- Works well with varying density datasets

23.4 Visualization: K-Means vs DBSCAN

[Image comparing K-means and DBSCAN clustering on irregular data shapes]

23.5 Hierarchical Clustering

Hierarchical clustering builds a tree-like structure of clusters, often visualized using a dendrogram.

23.5.1 Agglomerative (Bottom-Up)

- Starts with each data point as its own cluster
- Gradually merges clusters based on similarity

23.5.2 Divisive (Top-Down)

- Starts with one large cluster
- Recursively splits it into smaller clusters

23.6 Dimensionality Reduction and PCA

Dimensionality reduction reduces the number of features while preserving essential information.

23.6.1 Principal Component Analysis (PCA)

PCA is a linear technique that:

- Reduces feature dimensionality
- Minimizes information loss
- Removes noise from data

It is widely used in applications such as face recognition (eigenfaces).

23.6.2 Role in Clustering

Dimensionality reduction:

- Improves clustering performance
- Reduces noise in data
- Enhances computational efficiency

23.7 Advanced Visualization Techniques

High-dimensional data is often transformed into 2D or 3D space for visualization.

23.7.1 t-SNE (t-Distributed Stochastic Neighbor Embedding)

- Preserves the local structure of data
- Keeps similar points close together
- Ideal for visualization of complex datasets

23.7.2 UMAP (Uniform Manifold Approximation and Projection)

- Preserves both local and global structure
- More computationally efficient than t-SNE
- Better scalability for large datasets

23.8 Summary

In this module, we explored:

- Clustering fundamentals and applications
- K-Means algorithm and its evaluation methods
- Density-based clustering (DBSCAN, HDBSCAN)
- Hierarchical clustering approaches
- Dimensionality reduction using PCA
- Advanced visualization techniques like t-SNE and UMAP

These concepts collectively form the foundation of **unsupervised learning and exploratory data analysis** in machine learning.

Module 5: Evaluating and Validating Machine Learning Models

24 Classification Metrics and Evaluation Techniques

24.1 Overview

Supervised learning evaluation establishes how well a machine learning model can predict outcomes for unseen data. It is essential for understanding model effectiveness and involves comparing model predictions with ground truth labels.

During training:

- The model optimizes predictions based on evaluation metrics

After training:

- The model is evaluated again to determine how well it generalizes to unseen data

Supervised learning evaluation is important in both the training and testing phases.

24.2 Train-Test-Split Technique

24.2.1 Purpose

When training a machine learning model, the entire dataset should not be fed into the model at once. The train-test-split technique is used to estimate the performance of machine learning algorithms when making predictions on unseen data.

24.2.2 Dataset Split

The dataset is divided into two subsets:

24.2.2.1 Training Set

- Contains approximately 70% to 80% of the dataset
- Used to train the model

24.2.2.2 Test Set

- Used to evaluate how well the model generalizes to new unseen data

24.3 Classification Tasks

In classification tasks, the model predicts categorical labels and compares those predictions with actual labels.

Examples of classification metrics include:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1 Score

24.4 Accuracy

24.4.1 Definition

Accuracy is the ratio of correctly predicted instances to the total number of instances in the dataset.

24.4.2 Formula

Accuracy = Number of Correct Predictions / Total Number of Predictions

24.4.3 Example

Consider a pass-or-fail biology test example:

- Green squares represent pass
- Red squares represent failure
- Grey points represent misclassified observations

If 70% of the observations are classified correctly, then:

Accuracy = 70%

24.5 Confusion Matrix

24.5.1 Definition

A confusion matrix is a table used to evaluate classification performance by comparing actual labels with predicted labels.

24.5.2 Structure

- Y-axis → True Labels
- X-axis → Predicted Labels

24.5.3 Components

24.5.3.1 True Positive (TP)

- Predicted pass
- Actual result is pass

24.5.3.2 True Negative (TN)

- Predicted fail
- Actual result is fail

24.5.3.3 False Positive (FP)

- Predicted pass
- Actual result is fail

24.5.3.4 False Negative (FN)

- Predicted fail
- Actual result is pass

Example

y True label	Pass	4	2
	Fail	1	3
		Pass	Fail
		Predicted label x	

- **True positive**
You predicted pass and it is “pass”
- **True negative**
You predicted fail and it is “fail”
- **False positive**
You predicted pass, but it is “fail”
- **False negative**
You predicted fail, but it is “pass”

24.6 KNN Classifier Example

24.6.1 Iris Dataset

A KNN classifier can be trained to predict iris flower types using the Iris dataset from Scikit-learn.

24.6.2 Accuracy

The classifier achieved:

Accuracy = 93%

24.6.3 Visualization

- Background colors represent predicted classes
- Dots represent actual classes
- Few mismatched colors indicate few misclassifications

24.7 Heat Map of Confusion Matrix

24.7.1 Purpose

A heat map visually represents the confusion matrix.

24.7.2 Color Representation

- Purple → Lowest values
- Yellow → Highest values

24.7.3 Interpretation

The diagonal entries represent correct predictions. A strong diagonal indicates good classifier performance.

Example:

- Predictions classified as setosa show how many should actually have been:
 - Setosa
 - Versicolor
 - Virginica

24.8 Precision

24.8.1 Definition

Precision measures how many predicted positive instances are actually positive.

24.8.2 Formula

$\text{Precision} = \frac{\text{True Positives}}{(\text{True Positives} + \text{False Positives})}$

24.8.3 Importance

Precision is important when false positives are costly.

24.8.4 Example

Movie recommendation systems:

- Recommending an unwanted movie wastes promotional resources
- False positives increase unnecessary costs

24.9 Recall

24.9.1 Definition

Recall measures how many actual positive instances are correctly predicted.

24.9.2 Formula

$\text{Recall} = \text{True Positives} / (\text{True Positives} + \text{False Negatives})$

24.9.3 Example

If:

- Correct pass predictions = 4
- Total actual pass observations = 7

Then:

$\text{Recall} = 4 / 7 = 57.1\%$

24.9.4 Importance

Recall is important when missing positive cases is costly.

24.9.5 Example

Medical diagnosis:

- False negatives may lead to undiagnosed illnesses
- Patient health risks increase if actual illnesses are missed

24.10 F1 Score

24.10.1 Definition

The F1 score is the harmonic mean of precision and recall. It balances both metrics.

24.10.2 Formula

$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

24.10.3 Importance

F1 score is useful when:

- Precision and recall are equally important
- Optimizing only one metric is insufficient

24.10.4Example

In medical diagnosis:

- Incorrectly diagnosing patients can lead to harmful treatments
- A balanced metric is necessary

24.11 Classification Report Example

24.11.1Iris Flower Classes

Metrics can be calculated for:

- Setosa
- Versicolor
- Virginica

24.11.2Setosa Performance

Setosa achieved:

Precision = 1

Recall = 1

F1 Score = 1

24.11.3Weighted Average

The weighted average is calculated based on:

- Support of each class
- Number of observations in each class

24.12 Summary

Supervised learning evaluation determines how well a machine learning model predicts outcomes for unseen data.

The train-test-split technique:

- Divides the dataset into training and testing subsets
- Evaluates prediction performance on unseen data

Common classification evaluation metrics include:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1 Score

The F1 score is the harmonic mean of precision and recall and is useful when both metrics are equally important.

25 Regression Metrics and Evaluation Techniques

25.1 Overview

Regression models are not foolproof and often make prediction errors. Evaluating a regression model helps determine how accurately the model can predict continuous numerical values such as exam grades, prices, or emissions.

25.2 Understanding Regression Error

Consider a scenario where final exam grades are predicted using midterm scores.

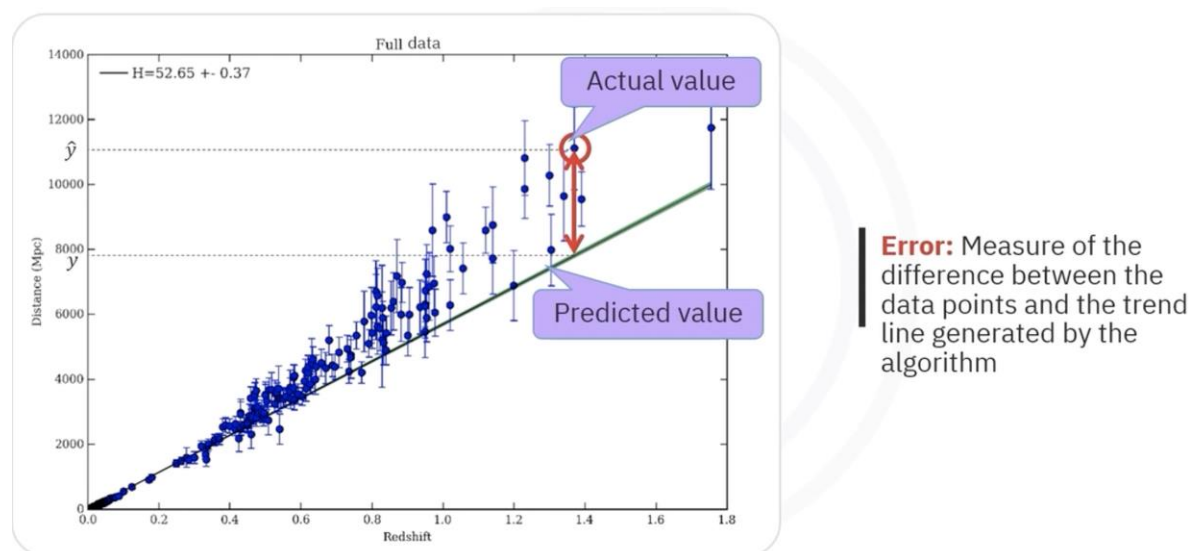
In a regression graph:

- Blue dots represent actual grades
- The regression line represents predicted grades

The difference between:

- Predicted values
- Actual values

is called the **error**.



25.2.1 Definition of Error

In linear regression, the error of a model measures the difference between:

- Actual data points
- Trend line generated by the algorithm

Since datasets contain multiple data points, errors can be measured in several ways.

25.3 Importance of Regression Metrics

Regression metrics provide insights into:

- Model accuracy
- Error magnitude
- Error distribution
- Goodness of fit

These metrics help evaluate how well the regression model performs.

25.4 Important Regression Metrics

25.4.1 Mean Absolute Error (MAE)

MAE is the average absolute difference between:

- Predicted values
- Actual observed values

It measures the average magnitude of prediction errors without considering direction.

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

25.4.2 Mean Squared Error (MSE)

MSE is the average of the squared differences between:

- Predicted values
- Actual values

Squaring increases the penalty for larger errors.

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

25.4.3 Root Mean Squared Error (RMSE)

RMSE is the square root of MSE.

It is widely used because:

- It has the same units as the target variable
- It is easier to interpret than MSE

$$\text{RMSE} = \sqrt{\text{MSE}}$$

25.4.4 R-Squared (R^2)

R-squared measures the proportion of variance in the dependent variable that can be explained by the independent variable(s).

It is also known as:

- Coefficient of Determination

25.4.4.1 Range of R^2

- $R^2 = 0 \rightarrow$ Poor model
- $R^2 = 1 \rightarrow$ Perfect model
- $0 < R^2 < 1 \rightarrow$ Real-world practical models

$$R^2 = 1 - (\text{Unexplained Variance} / \text{Total Variance})$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

$$R^2 = \frac{\sum_{i=1}^n (\hat{y}_i - \bar{y})^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$



Provide insight into a model's performance

- Accuracy
- Error distribution
- Error magnitude

25.5 Understanding Explained Variance

Explained variance is the sum of squared differences between:

- Predictions
- Average value of the actual target data

For a perfect predictor:

- Explained variance equals total variance
- Unexplained variance becomes zero
- R^2 becomes 1

For a Mean Value Model:

Suppose a model predicts only the mean value for every data point.

In this case:

- Explained variance becomes zero

- The model fails to explain the variation in the data
- R^2 becomes 0

25.6 Negative R-Squared Values

Negative R^2 values indicate:

- The model performs extremely poorly
- Unexplained variance exceeds total variance

This means the regression model is worse than simply predicting the mean.

$$\begin{aligned}
 \text{Explained Variance} &= \sum_{i=1}^n (\hat{y}_i - \bar{y})^2 \\
 \text{Unexplained Variance} &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 = n * \text{MSE} \\
 \text{Total Variance} &= \sum_{i=1}^n (y_i - \bar{y})^2 = \sigma^2 \\
 R^2 &= \frac{\sum_{i=1}^n (\hat{y}_i - \bar{y})^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = 1 - \frac{\text{Unexplained Variance}}{\text{Total Variance}}
 \end{aligned}$$

25.7 Visualization in Regression Evaluation

No single metric alone can fully describe model performance.

It is important to:

- Plot actual values
- Plot predicted values
- Visually inspect how well the regression line fits the data

Visualization helps identify:

- Poor fitting
- Outliers
- Nonlinear relationships

25.8 Assumption of R-Squared

R^2 assumes:

- A linear relationship exists between input features and the target variable

Therefore:

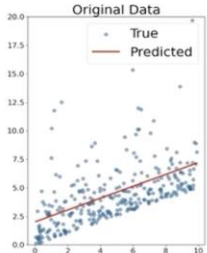
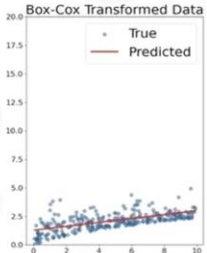
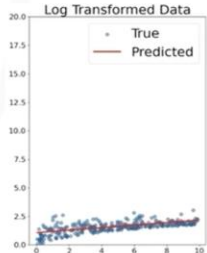
- R^2 may be misleading for nonlinear models

25.9 Effect of Data Transformations

Three regression models were compared using:

- Original target variable
- Box-Cox transformed target
- Logarithmically transformed target

<u>Untransformed data</u>	<u>Box-cox-transformed data</u>	<u>Log-transformed data</u>
Explained variance: 0.30	Explained variance: 0.40	Explained variance: 0.42
R^2: 0.29	R^2: 0.40	R^2: 0.42
MAE: 1.54	MAE: 0.47	MAE: 0.29
MSE: 4.767	MSE: 0.366	MSE: 0.148
RMSE: 2.183	RMSE: 0.605	RMSE: 0.375

25.9.1 Observation

As transformations were applied:

- Data became more concentrated around the best-fit line
- Model performance improved

25.9.2 Metric Improvements

The following metrics improved significantly:

- R^2 increased
- MAE decreased
- MSE decreased
- RMSE decreased

This indicates a better model fit.

25.10 Summary

Evaluating a regression model determines how accurately the model predicts continuous numerical values.

The error of a regression model measures the difference between:

- Actual data points
- Predicted trend line

The essential regression metrics are:

- MAE
- MSE
- RMSE
- R^2

Explained variance measures how much variation in the target variable is captured by the model.

R^2 measures the proportion of variance in the target variable that can be predicted from the input variables.

26 Evaluating Unsupervised Learning Models: Heuristics and Techniques

26.1 Overview

Evaluating unsupervised learning models is more challenging than evaluating supervised learning models because there are no predefined labels or ground truths available for training.

Unsupervised learning techniques, such as clustering and dimensionality reduction, aim to discover hidden patterns and structures in data. Therefore, evaluation methods focus on:

- Assessing the quality of discovered patterns
- Measuring how effectively the model groups similar data points
- Determining whether meaningful structures are preserved

26.2 Challenges in Evaluating Unsupervised Learning

Unsupervised learning results are often subjective, requiring careful evaluation for consistency and reliability.

26.2.1 Stability in Unsupervised Learning

Stability is an important factor in assessing model reliability.

A stable model:

- Produces similar results across different subsets of data
- Maintains consistency despite dataset perturbations or changes

Example:

- A stable clustering model should produce similar clusters even when the dataset slightly changes.

26.3 Approaches for Evaluating Unsupervised Learning Models

There is no single universal method for evaluating unsupervised learning models. Effective evaluation usually combines multiple techniques, including:

- Heuristics
- Domain expertise
- Internal evaluation metrics
- External evaluation metrics
- Ground truth comparisons
- Visualization techniques

26.4 Clustering Evaluation

The goal of clustering is to group similar data points into clusters.

Since clustering does not use predefined labels, several heuristics are used to evaluate cluster quality.

26.4.1 Types of Clustering Evaluation Methods

- Internal evaluation metrics
- External evaluation metrics
- Generalizability and stability evaluation
- Dimensionality reduction visualization
- Cluster-assisted learning
- Domain expertise feedback

26.5 Internal Clustering Evaluation Metrics

Internal evaluation metrics assess clustering quality using only the input data.

26.5.1 Silhouette Score

The silhouette score measures:

- Cohesion within clusters
- Separation between clusters

The score ranges from:

-1 to 1

Interpretation:

- Higher values indicate better-defined clusters
- Values near 1 represent strong clustering
- Negative values suggest incorrect cluster assignments

26.5.2 Davies-Bouldin Index

The Davies-Bouldin Index measures:

- Cluster compactness
- Separation from neighboring clusters

Interpretation:

- Lower values indicate better clustering quality
- Lower scores mean clusters are more distinct and compact

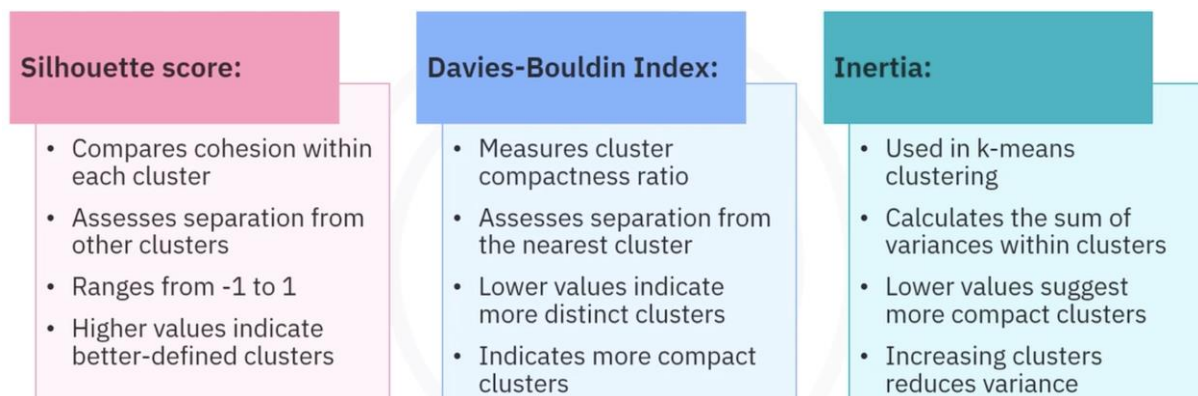
26.5.3 Inertia in K-Means Clustering

Inertia calculates:

- The sum of variances within clusters

Interpretation:

- Lower inertia indicates more compact clusters
- Increasing the number of clusters decreases inertia, creating a tradeoff



26.6 Example: Excellent Clustering Results

Applying K-Means clustering on simulated blobs produced:

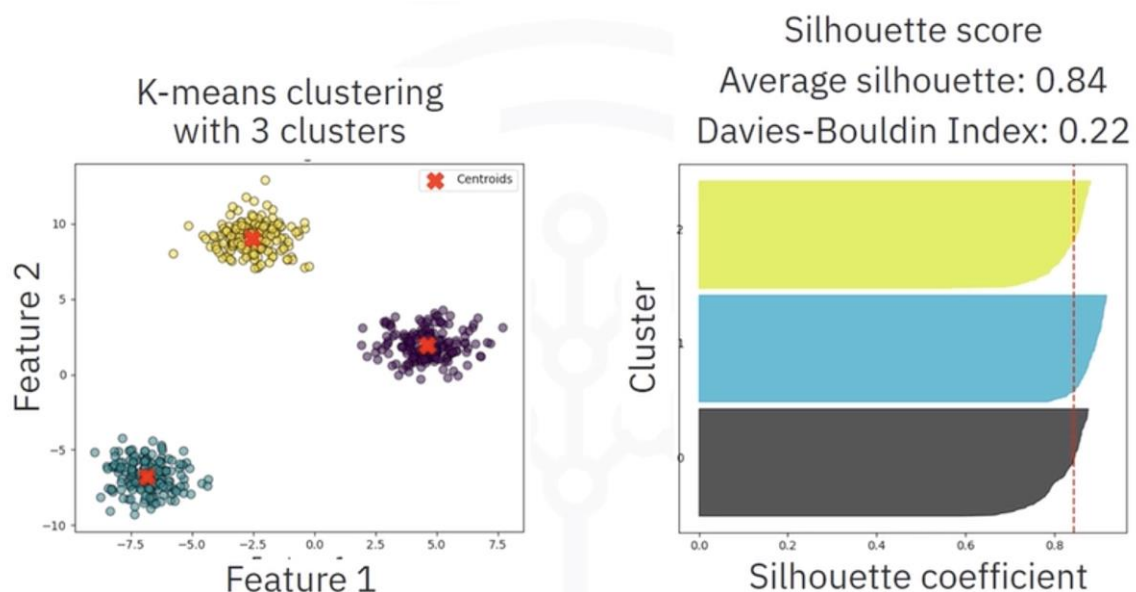
- Distinctly separated clusters
- Dense cluster formations

26.6.1 Evaluation Results

- Average Silhouette Score = 0.84
- Davies-Bouldin Index = 0.22

Interpretation:

- Both metrics indicate excellent clustering quality



- Shows a high average silhouette score of 0.84
- Indicates a low Davies-Bouldin index of 0.22

26.6.2 Silhouette Plot

In a silhouette plot:

- Each bar represents silhouette coefficients for data points in a cluster
- Different colors represent different clusters
- The vertical dashed red line indicates the average silhouette score

26.7 Example: Moderate Clustering Results

Applying K-Means clustering on more dispersed simulated blobs produced:

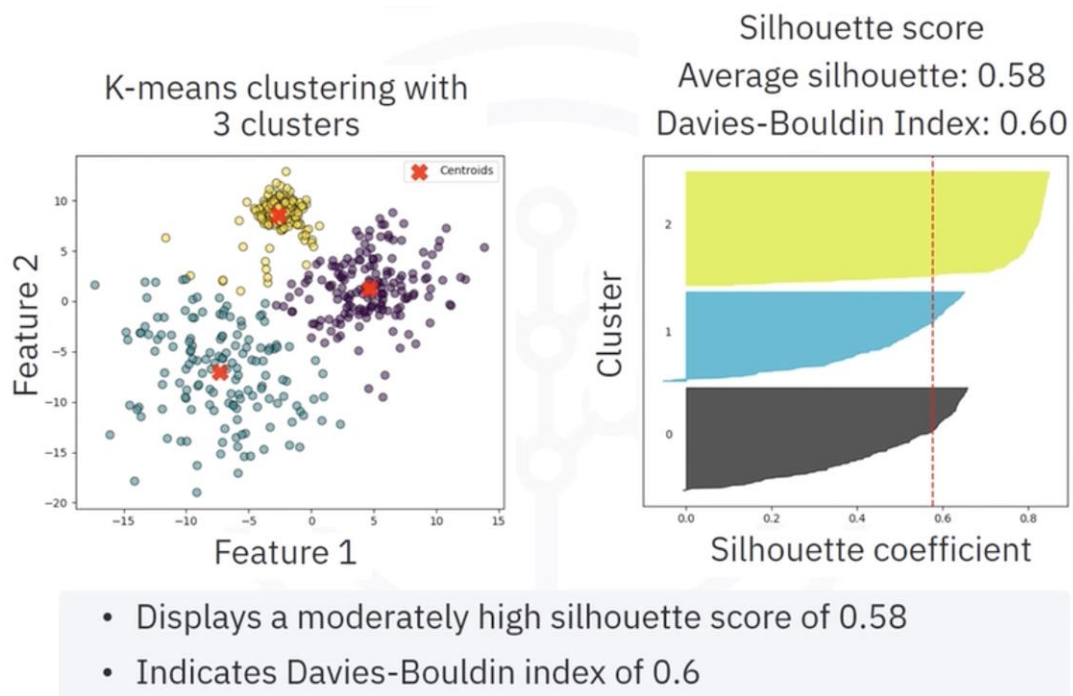
- Distinct but spread-out clusters

26.7.1 Evaluation Results

- Silhouette Score = 0.58
- Davies-Bouldin Index = 0.6

Interpretation:

- These metrics suggest reasonably good clustering results
- Some negative silhouette values indicate possible misclassified points



26.8 External Clustering Evaluation Metrics

External metrics use labeled or ground-truth data to compare predicted clusters with actual classes.

26.8.1 Adjusted Rand Index (ARI)

ARI measures similarity between:

- True labels
- Predicted clustering assignments

The score ranges from:

-1 to 1

Interpretation:

- 1 = Perfect agreement
- 0 = Random clustering
- Negative values = Worse than random clustering

26.8.2 Normalized Mutual Information (NMI)

NMI measures shared information between:

- Predicted clusters
- True labels

The score ranges from:

0 to 1

Interpretation:

- 1 = Perfect agreement
- 0 = No shared information

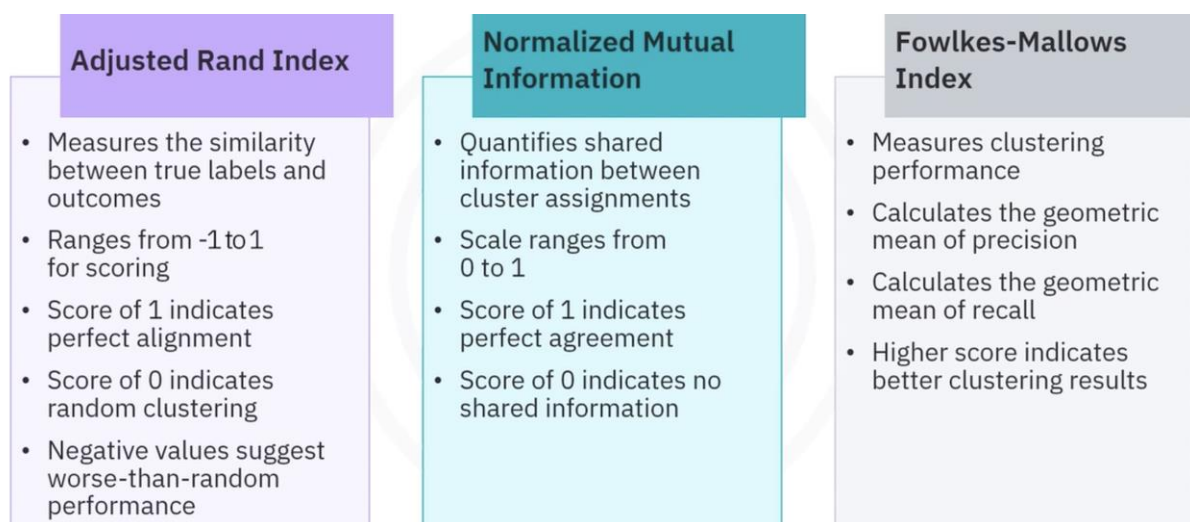
26.8.3 Fowlkes-Mallows Index (FMI)

FMI is calculated using:

- Precision
- Recall

Interpretation:

- Higher scores indicate better clustering performance



26.9 Evaluating Dimensionality Reduction

Dimensionality reduction techniques include:

- PCA (Principal Component Analysis)
- t-SNE
- UMAP

Evaluation determines how well reduced-dimensional data preserves important information.

26.9.1 Explained Variance Ratio in PCA

Explained Variance Ratio measures:

- The amount of variance captured by principal components

Purpose:

- Helps determine the number of principal components needed

26.9.1.1 PCA Visualization Example

A scatter plot using:

- PC1 (Principal Component 1)
- PC2 (Principal Component 2)

can visualize clusters in lower-dimensional space.

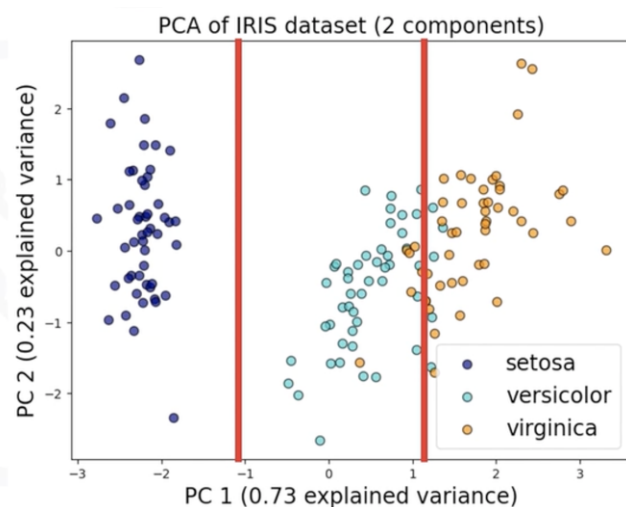
Example:

- Iris flower dataset
- Points color-coded by species:
 - Setosa
 - Versicolor
 - Virginica

Observation:

- PC1 nearly separates the clusters

- PC1 nearly separates the clusters effectively
- Four features make direct class visualization challenging
- PCA enables indirect visualization with two dimensions



26.9.2 Reconstruction Error

Reconstruction error measures:

- How accurately the original data can be reconstructed from reduced dimensions

Interpretation:

- Lower reconstruction error indicates better information preservation

26.9.3 Neighborhood Preservation

Neighborhood preservation evaluates:

- Whether relationships between nearby data points remain preserved after dimensionality reduction

This metric is especially important for:

- t-SNE
- UMAP
- Other manifold learning algorithms

26.9.4 Explained Variance Plot

A bar plot of explained variance:

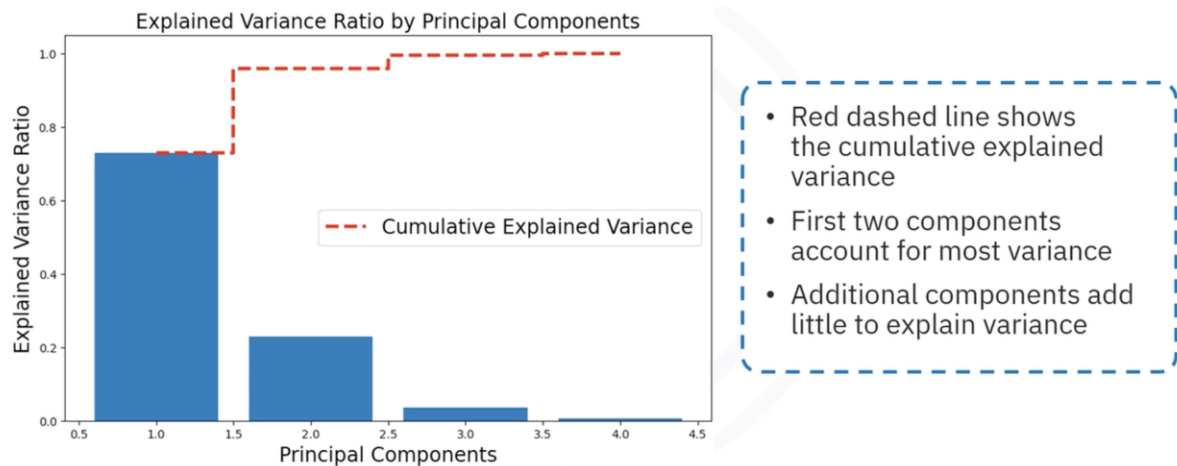
- Displays the variance contribution of each principal component
- Components are arranged in descending order

The cumulative explained variance is often shown using:

- A dashed red line

Observation:

- The first few components usually capture most of the variance
- Additional components contribute less information



26.10 Importance of Visualization and Domain Expertise

Evaluating unsupervised learning models often requires:

- Subjective analysis
- Visualization techniques
- Domain expertise

Useful visualization methods include:

- Scatter plots
- Dendrograms
- PCA projections
- t-SNE projections
- UMAP projections

These tools help interpret the results of clustering and dimensionality reduction.

26.11 Summary

Unsupervised learning evaluation focuses on assessing the quality of discovered patterns and structures without predefined labels.

Important concepts include:

- Stability and consistency of models
- Internal clustering metrics:
 - Silhouette Score
 - Davies-Bouldin Index
 - Inertia
- External clustering metrics:
 - Adjusted Rand Index
 - Normalized Mutual Information
 - Fowlkes-Mallows Index
- Dimensionality reduction evaluation:

- Explained Variance Ratio
- Reconstruction Error
- Neighborhood Preservation

Visualization tools and domain expertise are critical for interpreting unsupervised learning results.

27 Cross-Validation and Advanced Model Validation Techniques

27.1 Overview

Model validation is the process of optimizing a machine learning model while ensuring that it can generalize well to unseen data. It helps prevent overfitting during model training and hyperparameter tuning.

After studying this topic, we understand:

- What model validation is
- What data snooping means
- How to avoid data leakage
- Different validation strategies
- The concept of K-Fold Cross-Validation
- Stratified Cross-Validation
- Handling skewed target variables in regression problems

27.2 Model Validation

Model validation focuses on improving a model without compromising its ability to predict unseen data accurately.

Its primary goals are:

- Prevent overfitting
- Improve model generalization
- Optimize hyperparameters safely

27.3 Basic Train-Test Split Method

A dataset is commonly divided into:

- Training Set
- Test Set

27.3.1 Training Set

Used to:

- Train the machine learning model
- Learn patterns from data

27.3.2 Test Set

Used to:

- Evaluate the model
- Estimate how well the model performs on unseen data

27.4 Hyperparameters

Most machine learning models contain optional parameter settings called hyperparameters.

Hyperparameters:

- Control how the model learns
- Affect model performance
- Must be optimized carefully

Examples:

- Number of neighbors in KNN
- Depth of a Decision Tree
- Learning rate in Gradient Descent

27.5 Data Snooping

Data snooping occurs when the test data is used during model optimization.

Example:

- Trying multiple hyperparameter values
- Selecting the one that performs best on the test data

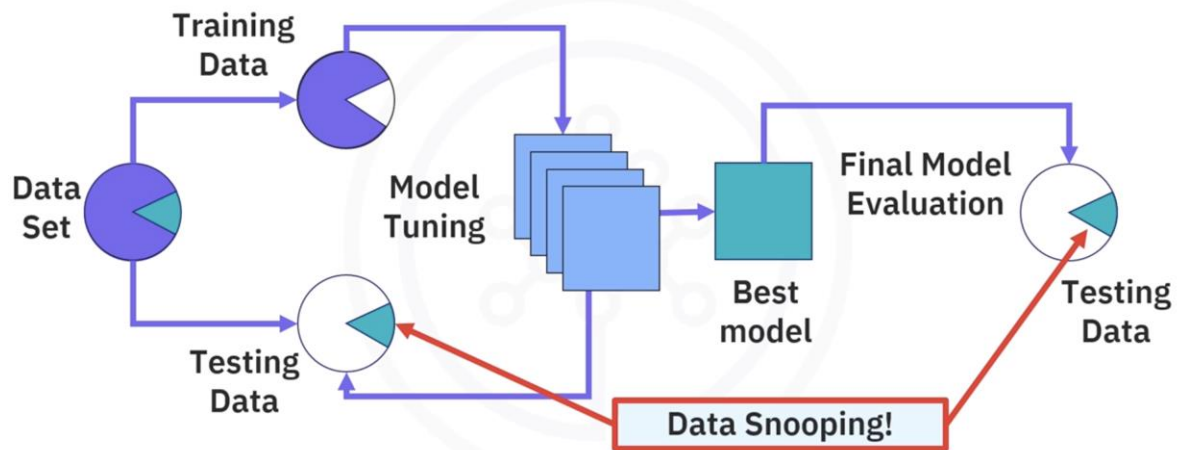
This indirectly fits the model to the test set instead of unseen data.

27.5.1 Problems with Data Snooping

- Causes overfitting
- Leads to poor generalization
- Produces unreliable evaluation results

Data snooping is a form of:

- Data Leakage



27.6 Proper Model Validation Strategy

To avoid overfitting and data snooping, data should be divided into three parts:

27.6.1 Training Set

Used for:

- Training the model
- Optimizing hyperparameters

27.6.2 Validation Set

Used for:

- Evaluating model performance during optimization
- Comparing different hyperparameter settings

27.6.3 Test Set

Used for:

- Final evaluation only
- Completely unseen data

The test set must remain untouched until the final model is selected.

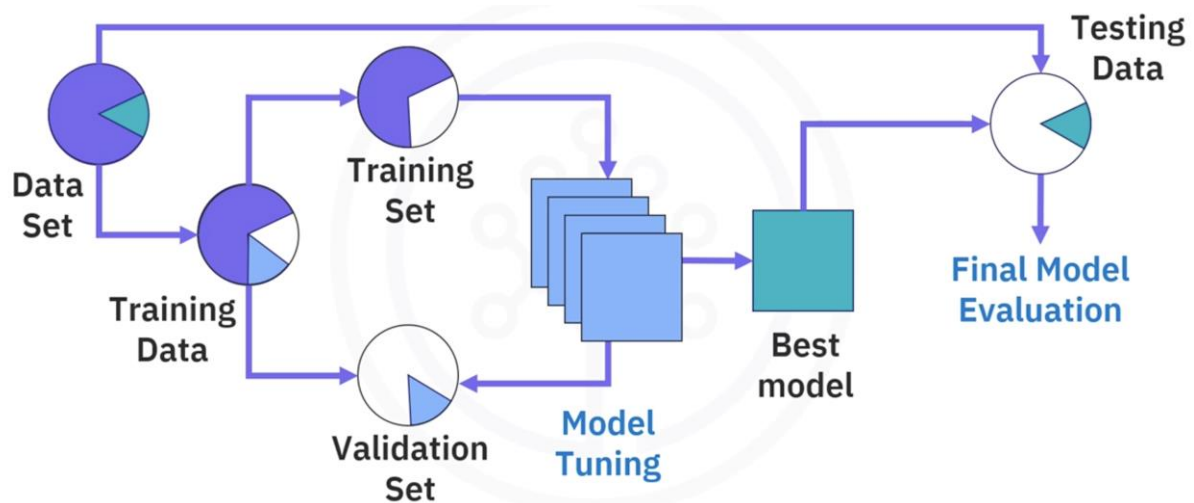
27.7 Cross-Validation

Cross-validation is a validation technique used to:

- Tune hyperparameters
- Estimate model generalization performance

27.7.1 Cross-Validation Workflow

1. Split the data into training and testing data
2. Further split the training data into:
 - Training subset
 - Validation subset
3. Train the model repeatedly using different hyperparameters
4. Evaluate performance on validation data
5. Select the best hyperparameters
6. Perform final evaluation on unseen test data



This process produces a validated model with better generalization capability.

27.8 Problems with a Single Validation Set

Using only one validation set can create several issues.

27.8.1 Overfitting to Validation Data

The model may adapt specifically to the chosen validation set.

27.8.2 Limited Training Data

If the dataset is small:

- Too much data is reserved for validation/testing
- Less data remains for training

27.8.3 Poor Representation

Training and validation sets may not represent the overall population properly.

27.8.4 Unstable Performance

Results may vary significantly depending on the selected validation subset.

27.9 K-Fold Cross-Validation

K-Fold Cross-Validation solves many validation problems.

27.9.1 Concept

The dataset is divided into K equal-sized folds.

Typical values:

- 5-fold
- 10-fold

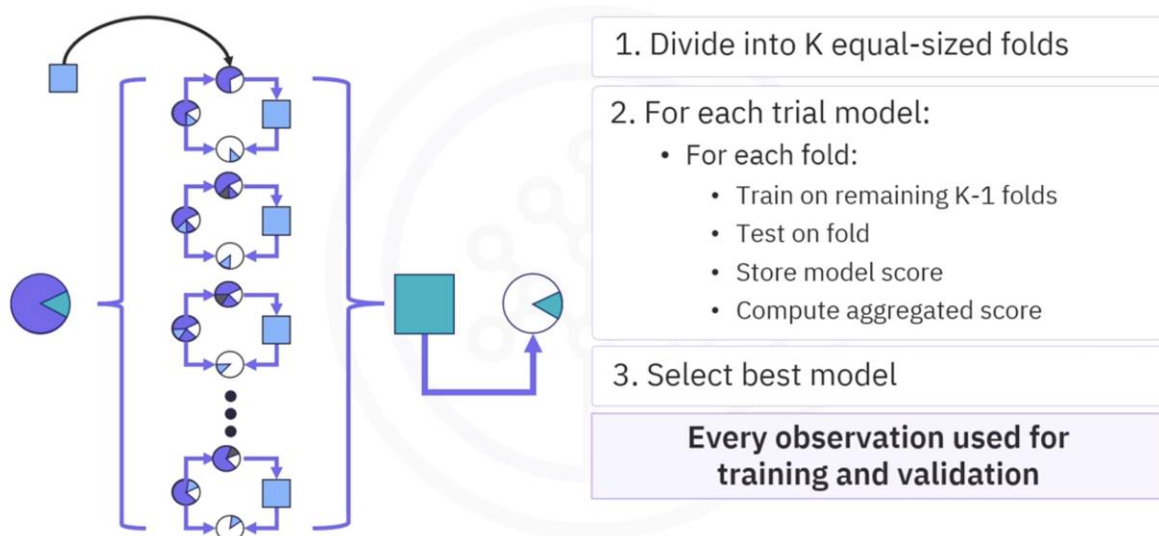
27.9.2 K-Fold Procedure

For each fold:

1. Select one fold as validation data
2. Use remaining K-1 folds for training
3. Train the model
4. Evaluate the model on the selected fold
5. Store the evaluation score

After all folds are processed:

- Aggregate all scores
- Select the best hyperparameters



27.10 Benefits of K-Fold Cross-Validation

27.10.1 Better Data Utilization

Every data point is used for:

- Training
- Validation

27.10.2 Reduced Overfitting

It smooths out patterns specific to one training subset.

27.10.3 More Reliable Evaluation

Provides a stronger estimate of model performance on unseen data.

27.10.4 Improved Generalization

Helps determine how well the model will perform in real-world scenarios.

27.11 Stratified Cross-Validation

In classification problems, datasets may contain:

- Many observations in one class
- Very few observations in another class

This creates an:

- Imbalanced Classification Problem

27.11.1 Purpose of Stratified Cross-Validation

Stratified Cross-Validation preserves class distribution in every fold.

Benefits:

- Prevents evaluation bias
- Produces more reliable performance estimates

27.12 Skewed Target Variables in Regression

Regression problems may contain highly skewed target variables.

Many machine learning models assume:

- Normally distributed targets

27.12.1 Problem with Skewed Data

Lower target values may occur much more frequently than higher values.

This can negatively affect:

- Model fitting
- Prediction accuracy

27.13 Transforming Skewed Targets

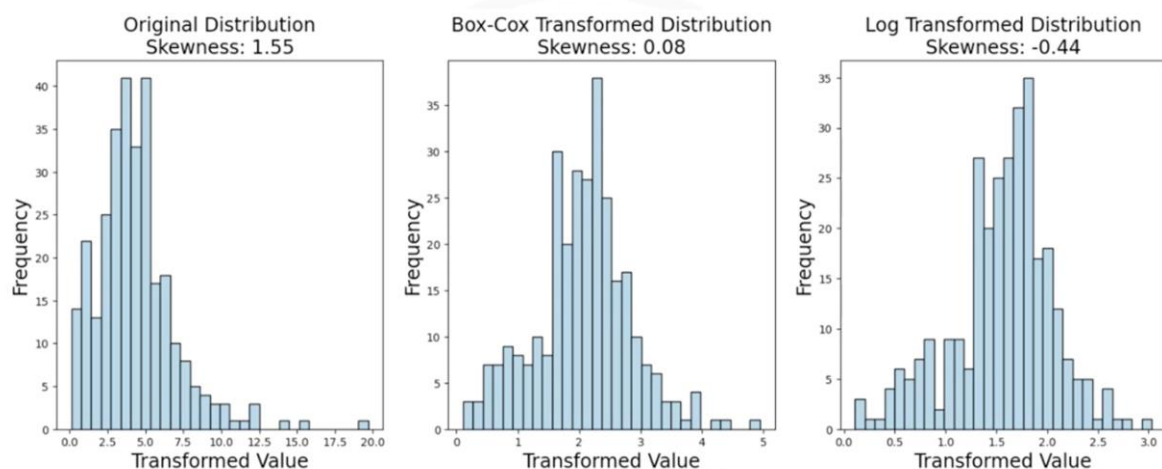
Target transformation techniques help reduce skewness.

Common methods:

- Log Transformation
- Box-Cox Transformation

27.13.1 Benefits of Transformations

- Reduce skewness
- Improve data distribution
- Help the linear regression fit the data better



27.14 Summary

Model validation helps prevent overfitting while optimizing machine learning models.

Key concepts covered:

- Train-Test Split
- Hyperparameter Tuning
- Data Snooping
- Validation Sets

- Cross-Validation
- K-Fold Cross-Validation
- Stratified Cross-Validation
- Handling Skewed Regression Targets

K-Fold Cross-Validation is a powerful technique because it:

- Uses data efficiently
- Reduces overfitting
- Improves model generalization

Stratified Cross-Validation preserves class distributions and prevents bias in classification problems.

28 Regularization in Linear Regression

28.1 Overview

Regularization is a regression technique used to prevent overfitting in machine learning models. It constrains the model during training and discourages it from fitting the training data too closely.

Regularization achieves this by suppressing the size of model coefficients.

After studying this topic, we understand:

- What regularization is
- How regularization works in linear regression
- The differences between Linear, Ridge, and Lasso Regression

28.2 Purpose of Regularization

In machine learning, overfitting occurs when a model learns the training data too well, including noise and outliers, which reduces its performance on unseen data.

Regularization helps reduce overfitting by:

- Penalizing large coefficient values
- Simplifying the model
- Improving generalization performance

28.3 Regularized Cost Function

With regularization, the cost function is modified as follows:

Regularized Cost Function = Mean Squared Error + $\lambda \times$ Penalty Term

Where:

- λ (lambda) controls the influence of the penalty term
- The penalty term measures the size of the coefficients

28.4 Linear Regression

Linear Regression models the relationship between variables by fitting a straight line to the dataset.

The prediction equation is:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

Where:

- $\hat{y}$ = Predicted value
- θ_0 = Bias or intercept term
- $\theta_1, \theta_2, \dots, \theta_n$ = Model coefficients (weights)
- $x_1, x_2, \dots, x_n$ = Feature variables

The goal of ordinary linear regression is to minimize:

- Mean Squared Error (MSE) between predicted and actual values

28.5 Ridge Regression

Ridge Regression is a regularized version of linear regression that uses an L2 penalty.

28.5.1 L2 Penalty

Penalty = Sum of squares of coefficients

28.5.2 Characteristics of Ridge Regression

- Shrinks the coefficient values
- Reduces overfitting
- Handles noisy data better than ordinary linear regression
- Does not reduce coefficients exactly to zero

28.6 Lasso Regression

Lasso Regression is another regularized form of linear regression that uses an L1 penalty.

28.6.1 L1 Penalty

Penalty = Sum of absolute values of coefficients

28.6.2 Characteristics of Lasso Regression

- Shrinks coefficients
- Can reduce some coefficients exactly to zero
- Performs feature selection automatically
- Useful for sparse datasets and data compression

Ridge regression adds an L_2 penalty: $\lambda \|\theta\|_2 = \lambda \sum_{i=1}^N \theta_i^2$

Lasso regression adds an L_1 penalty: $\lambda \|\theta\|_1 = \lambda \sum_{i=1}^N |\theta_i|$

28.7 Sparse Coefficients

Sparse coefficients mean:

- Only a small number of variables significantly contribute to predictions
- The remaining variables have little or no impact

Lasso regression performs very well in sparse environments because it identifies and removes unimportant features.

28.8 Signal-to-Noise Ratio (SNR)

SNR refers to how strongly meaningful information stands out from noise in the data.

28.8.1 High SNR

- The useful signal is strong
- Easier for models to identify patterns

28.8.2 Low SNR

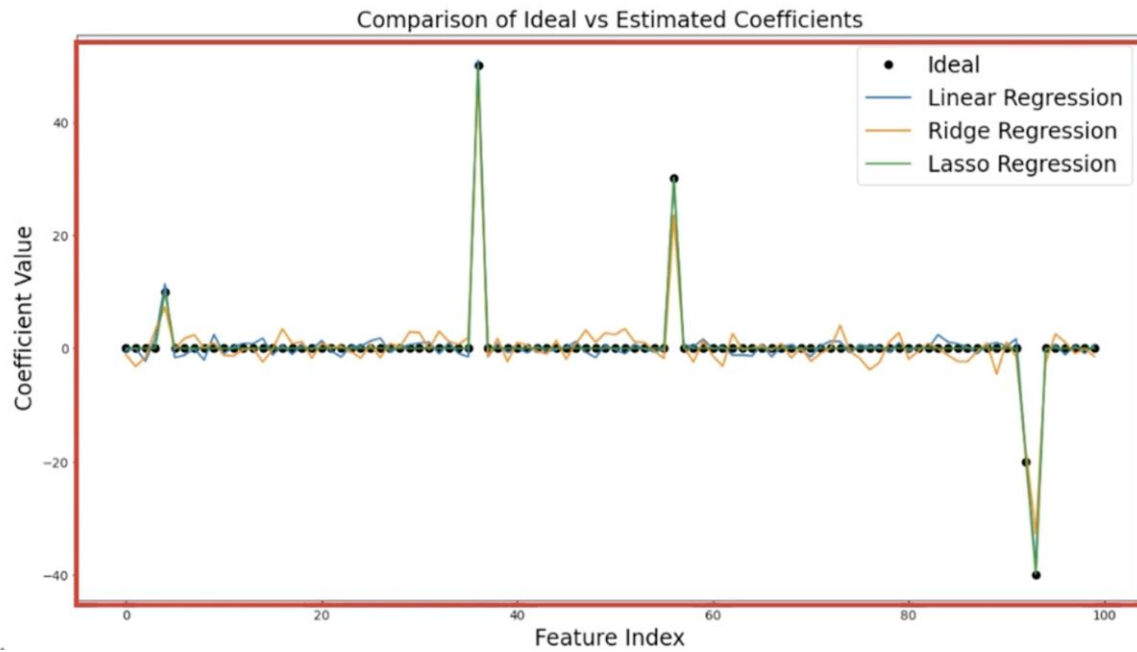
- The data contains significant noise
- Harder for models to identify correct relationships

28.9 Regression Performance in Sparse Coefficients with High SNR

In sparse datasets with high SNR:

- Linear Regression predicts non-zero coefficients well
- Ridge Regression also predicts non-zero coefficients well
- Lasso Regression predicts non-zero coefficients well and identifies zero coefficients exactly

Lasso performs best because of its feature selection capability.



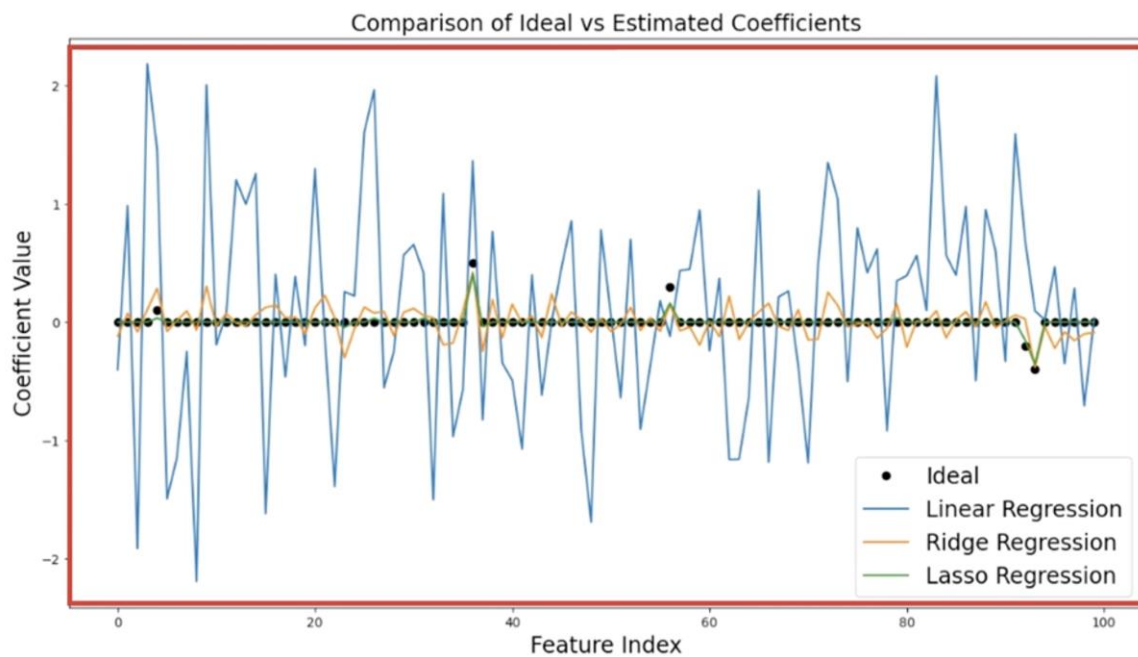
28.10 Regression Performance in Sparse Coefficients with Low SNR

In sparse datasets with low SNR:

- Linear Regression performs poorly
- It overestimates coefficients and becomes highly sensitive to noise
- Ridge and Lasso perform significantly better than ordinary linear regression
- Lasso is much better at identifying zero coefficients

This demonstrates that:

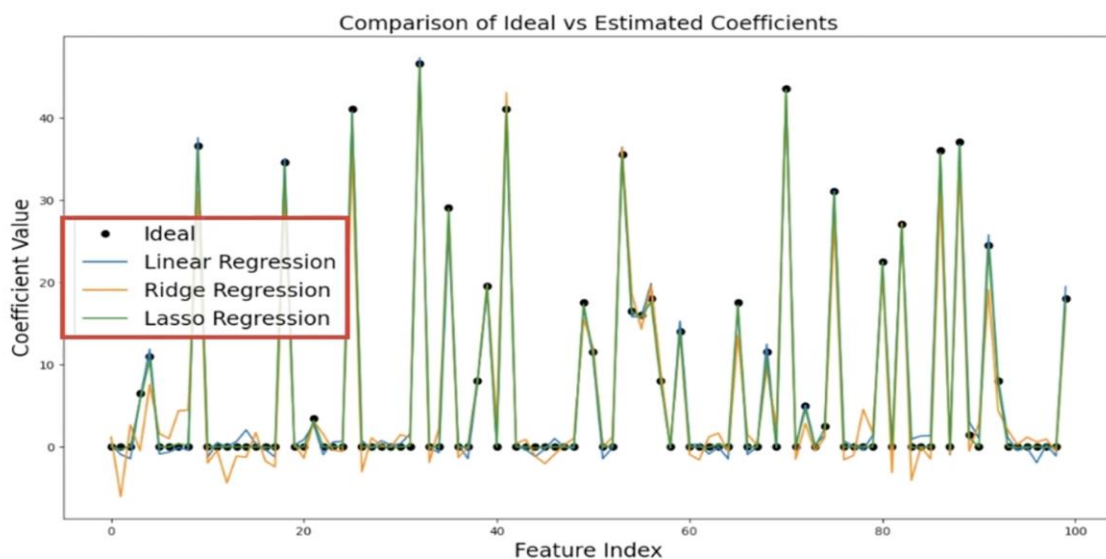
- Ordinary Linear Regression is sensitive to noisy data
- Lasso remains an excellent feature selector even in noisy environments



28.11 Regression Performance in Non-Sparse Coefficients with High SNR

In non-sparse datasets with high SNR:

- All three methods perform well
- Ridge Regression shows slightly more error
- Lasso still identifies zero coefficients more accurately



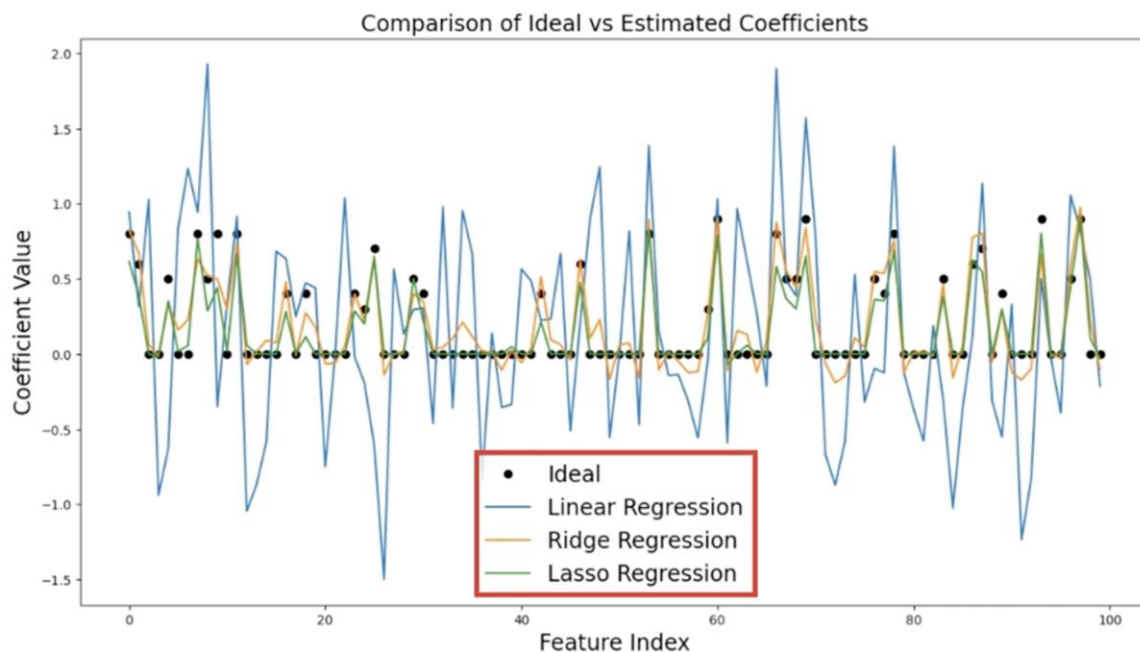
28.12 Regression Performance in Non-Sparse Coefficients with Low SNR

In non-sparse datasets with low SNR:

- Linear Regression again performs poorly
- Ridge Regression slightly outperforms Lasso for predicting non-zero coefficients
- Lasso remains better at identifying zero coefficients

Even in noisy conditions:

- Ridge and Lasso outperform ordinary linear regression



28.13 Model Comparison Using Test Data

A comparison was performed using:

- Lasso Regression
- Ridge Regression
- Ordinary Linear Regression

28.13.1 Experiment Setup

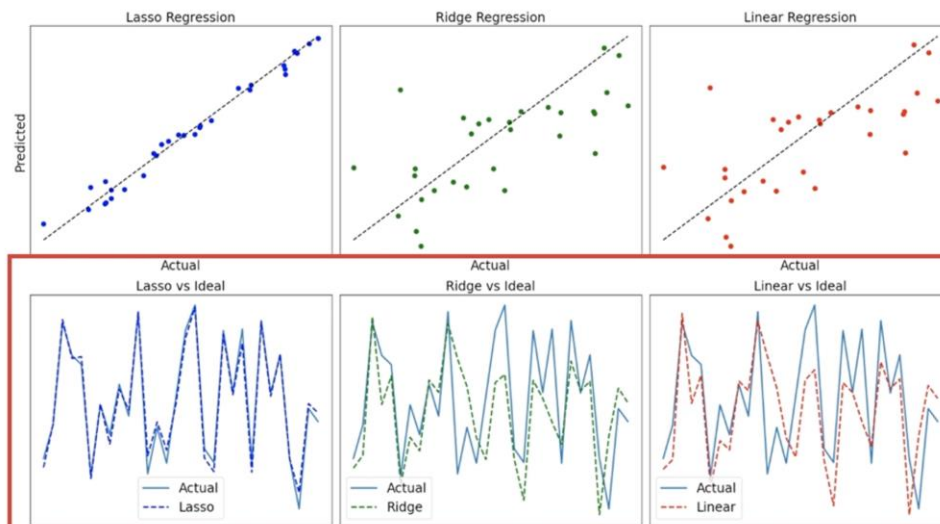
- Models were trained on 70% of the dataset
- Predictions were tested on the remaining data
- Performance was evaluated using scatter plots and Mean Squared Error (MSE)

28.13.2 Results

- Lasso predictions were most concentrated around the ideal 45-degree line
- Lasso predictions tracked actual values better than Ridge and Linear Regression
- Lasso achieved the lowest MSE

28.13.3 MSE Comparison

- Lasso MSE was approximately 30 times lower than Ridge and Ordinary Linear Regression



MSE
Lasso: 305
Ridge: 9744
Linear: 9855

28.14 Performance Summary

28.14.1 Ordinary Linear Regression

- Performs well in high SNR environments
- Performs poorly in low SNR environments
- Sensitive to noise and outliers
- Prone to overfitting

28.14.2 Ridge Regression

- Reduces overfitting using L2 regularization
- Performs well in noisy environments
- Better stability than ordinary linear regression

28.14.3 Lasso Regression

- Uses L1 regularization
- Best overall performance in most scenarios
- Excellent for feature selection
- Handles sparse datasets effectively
- Performs strongly in both high and low SNR environments

Linear Regression	Sparse Coefficients		Non-sparse Coefficients	
	High SNR	Low SNR	High SNR	Low SNR
Regular	****	*	****	*
Ridge	***	***	***	***
Lasso	*****	*****	*****	*****

28.15 Key Concepts Summary

- Regularization prevents overfitting

- It constrains model coefficients during training
- Linear Regression minimizes Mean Squared Error without penalties
- Ridge Regression uses an L2 penalty
- Lasso Regression uses an L1 penalty
- Lasso can reduce coefficients to zero, making it useful for feature selection
- Linear Regression is highly sensitive to noise and outliers
- Regularization techniques improve model performance and generalization

29 Data Leakage and Other Pitfalls

29.1 Overview

Data leakage occurs when a machine learning model is trained using information that would not be available in a real-world deployment. This causes the model to appear highly accurate during training and testing but perform poorly in production.

This section explains:

- Data leakage
- Data snooping
- Leakage mitigation techniques
- Feature importance interpretation pitfalls
- Common machine learning modeling pitfalls

29.2 Understanding Data Leakage

29.2.1 House Price Prediction Example

Suppose we want to train a model to predict house prices.

Along with normal features such as:

- Square footage
- Number of rooms
- Location

A new feature is engineered using the average of actual home prices across the entire dataset.

The model performs extremely well during testing. However, this feature contains information from future or unseen data that would not be available after deployment.

As a result:

- The model becomes misleadingly accurate during training and testing
- Real-world performance becomes poor after deployment

29.2.2 Definition of Data Leakage

Data leakage occurs when training data contains information that would not be available in real-world predictions after deployment.

Examples include:

- Future data leaking into training data
- Using statistics derived from the entire dataset
- Training data containing hidden information from testing data

29.2.3 Why Data Leakage Is Dangerous

Data leakage:

- Deceives the model
- Produces unrealistically high accuracy
- Prevents proper evaluation
- Causes poor generalization in production

Since leaked information may also exist in the test set, evaluation metrics often fail to detect the issue.

29.3 Data Snooping

29.3.1 Definition

Data snooping occurs when:

- The training set contains information about the testing set
- The model accesses data it should not have access to

29.3.2 Examples

Examples include:

- Using tomorrow's stock price to predict today's stock price
- Engineering features using the entire dataset before splitting the data

29.4 Preventing Data Leakage

29.4.1 Independent Data Processing

Data processing pipelines should always run independently on:

- Training data
- Validation data
- Testing data

29.4.2 Mitigation Techniques

29.4.2.1 Avoid Global Statistics

Avoid features based on:

- Global averages

- Dataset-wide statistics
- Information from future observations

29.4.2.2 Proper Dataset Separation

Ensure strict separation between:

- Training set
- Validation set
- Test set

Avoid:

- Overlap
- Contamination
- Shared information

29.4.2.3 Use Only Available Features

Ensure that deployed models only use features available during real-world predictions.

29.4.2.4 Careful Cross-Validation

Prevent leakage across validation folds during cross-validation.

This becomes especially important with:

- Temporal data
- Sequential datasets
- Time-series prediction tasks

29.5 Cross-Validation and Pipelines

29.5.1 Proper Pipeline Usage

When performing hyperparameter tuning:

- Fit the pipeline separately on each training fold
- Apply the fitted pipeline only to its corresponding validation fold

29.5.2 Example Machine Learning Pipeline

A typical pipeline may contain:

- Scaler
- PCA (Principal Component Analysis)
- KNN Classifier

29.5.3 Hyperparameter Tuning

A parameter grid may include:

- Number of PCA components
- Number of KNN neighbors

Grid Search Cross-Validation is then used to:

- Optimize parameters
- Evaluate performance

29.5.4 Important Best Practice

The pipeline itself must be passed into Grid Search.

This ensures:

- Proper separation of folds
- No data leakage during preprocessing

29.6 Time-Series Cross-Validation

29.6.1 Problem with Random Splits

For temporal data, random train-test splitting can leak future information into training data.

29.6.2 Correct Approach

Use sequential splits where:

- Training data always comes before testing data
- Future data is never used to predict the past

29.6.3 Time-Series Split

Instead of:

```
train_test_split()
```

Use:

```
TimeSeriesSplit()
```

During Grid Search:

```
GridSearchCV(cv=tscv)
```

29.6.4 Behavior of Time-Series Split

Scikit-learn's TimeSeriesSplit:

- Preserves temporal order
- Uses past data for training
- Uses future data for validation

With each split:

- Training data expands
- Test data shrinks

29.7 Feature Importance Interpretation Pitfalls

29.7.1 Redundant or Correlated Features

Highly correlated or redundant features:

- Share important values
- Appear less important individually

This can hide genuinely useful variables.

29.7.2 Incorrect Feature Selection

Blindly selecting only the “most important” features can:

- Remove valuable explanatory variables
- Reduce model performance

29.7.3 Scale Sensitivity

Some algorithms, such as Linear Regression:

- Do not naturally account for feature scale

Unscaled data can distort:

- Coefficient values
- Importance rankings

29.7.4 Correlation vs Causation

Feature importance indicates:

- Correlation

It does not necessarily indicate:

- Causation

Important features may not actually drive outcomes.

29.7.5 Ignoring Feature Interactions

Some models fail to properly account for feature interactions.

Example:

- Two separate features may appear weak individually
- Their interaction or product may strongly improve predictions

29.7.5.1 Example Scenario

Linear Regression:

- May treat the features as unimportant individually

Random Forest Regression:

- May implicitly detect the interaction
- Achieve strong performance

However:

- The actual important variable may be the interaction itself

29.8 Common Modeling Pitfalls

29.8.1 Poor Feature Engineering

Using raw data without:

- Feature selection
- Feature transformation

can prevent the discovery of the optimal model.

29.8.2 Incorrect Evaluation Metrics

Choosing inappropriate metrics or misinterpreting them can:

- Mislead evaluation
- Produce incorrect conclusions

29.8.3 Ignoring Class Imbalance

Failing to handle class imbalance causes:

- Bias toward majority classes
- Poor minority-class predictions

29.8.4 Blind Reliance on Automation

Automated Machine Learning (AutoML) tools are powerful, but understanding:

- The data
- The model
- The assumptions

remains essential.

29.8.5 Non-Causal What-If Analysis

If the model lacks features with a causal impact on the target variable:

- What-if scenarios become unreliable

Predictions based on hypothetical changes may become:

- Misleading
- Inaccurate

29.9 Summary

29.9.1 Data Leakage

Data leakage occurs when training data includes information unavailable during real-world deployment.

29.9.2 Leakage Prevention Techniques

Leakage can be mitigated by:

- Proper dataset separation
- Preventing contamination
- Using careful cross-validation
- Avoiding future information
- Proper hyperparameter tuning procedures

29.9.3 Feature Importance Pitfalls

Common feature importance issues include:

- Feature redundancy
- Scale sensitivity
- Mistaking correlation for causation
- Ignoring feature interactions

29.9.4 Additional Modeling Pitfalls

Other common modeling mistakes include:

- Poor feature selection
- Misinterpreting evaluation metrics
- Ignoring class imbalance
- Blind reliance on automation
- Using non-causal data for what-if analysis

30 Module 5 Summary and Highlights

30.1 Supervised Learning Evaluation

Supervised learning evaluation assesses a model's ability to predict outcomes for unseen data. This is commonly performed using a train/test split to estimate model performance.

30.1.1 Classification Evaluation Metrics

Key metrics used for classification model evaluation include:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1 Score

The F1 Score balances both precision and recall.

30.1.2 Regression Evaluation Metrics

Regression models are evaluated using:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R-squared (R^2)
- Explained Variance

These metrics measure prediction accuracy and model performance.

30.2 Unsupervised Learning Evaluation

Unsupervised learning models are evaluated based on pattern quality and clustering consistency.

30.2.1 Common Evaluation Metrics

- Silhouette Score

- Davies-Bouldin Index
- Adjusted Rand Index (ARI)

These metrics help determine how effectively patterns and clusters are identified in the data.

30.3 Dimensionality Reduction Evaluation

Dimensionality reduction techniques are evaluated to determine how well they preserve the original data structure.

30.3.1 Evaluation Metrics

- Explained Variance Ratio
- Reconstruction Error
- Neighborhood Preservation

These measures assess the quality of reduced-dimensional representations.

30.4 Model Validation

Model validation helps prevent overfitting and ensures better generalization to unseen data.

30.4.1 Validation Techniques

Data is commonly divided into:

- Training Set
- Validation Set
- Test Set

Hyperparameters are carefully tuned using validation data to improve model performance without overfitting.

30.5 Cross-Validation

Cross-validation provides robust model evaluation by repeatedly training and testing the model on different data splits.

30.5.1 Common Cross-Validation Methods

- K-Fold Cross-Validation
- Stratified Cross-Validation

These techniques reduce the risk of overfitting to the test dataset.

30.6 Regularization Techniques

Regularization helps prevent overfitting in linear regression models by adding penalty terms.

30.6.1 Types of Regularization

30.6.1.1 Ridge Regression (L2)

Adds squared coefficient penalties to reduce model complexity.

30.6.1.2 Lasso Regression (L1)

Adds absolute coefficient penalties and can perform feature selection by shrinking some coefficients to zero.

30.7 Data Leakage

Data leakage occurs when training data contains information that would not be available in real-world scenarios.

30.7.1 Prevention of Data Leakage

- Properly separate training, validation, and test datasets
- Avoid using future or target-related information during training
- Perform careful feature selection

30.8 Common Modeling Pitfalls

Several common mistakes can negatively impact machine learning model performance and interpretation.

30.8.1 Common Issues

- Misinterpreting feature importance
- Ignoring class imbalance
- Making causal assumptions without sufficient evidence

30.9 Feature Importance Considerations

Feature importance analysis should be interpreted carefully.

30.9.1 Important Considerations

- Redundant features can distort importance values
- Some methods are sensitive to feature scaling
- High importance does not imply causation
- Incorrect assumptions can lead to misleading conclusions

Module 6

31 Machine Learning Course Summary and Key Concepts

31.1 What is Machine Learning?

Machine Learning (ML) is a subset of Artificial Intelligence (AI) that uses data and algorithms to enable computers to:

- Imitate human learning
- Make decisions
- Gradually improve accuracy over time

31.2 Applications of Machine Learning

Machine learning is widely used in different industries:

31.2.1 Healthcare

Doctors use machine learning to:

- Prescribe correct medicines to patients

31.2.2 Finance and Banking

Banks use machine learning to:

- Approve or reject loan applications

31.2.3 E-Commerce

Businesses use machine learning to:

- Generate personalized customer recommendations

31.3 Types of Machine Learning Methods

Machine learning models learn using different approaches:

- Supervised Learning
- Unsupervised Learning
- Semi-Supervised Learning
- Reinforcement Learning

The choice of technique depends on:

- Problem type
- Data type

- Available resources
- Desired outcome

31.4 Machine Learning Tools and Pipelines

Machine learning tools provide complete pipelines that include:

- Data preprocessing
- Model building
- Model evaluation
- Model optimization
- Model implementation

These tools help in:

- Handling big data
- Performing statistical analysis
- Making predictions

31.5 Regression in Machine Learning

Regression is a **supervised learning technique** that models the relationship between:

- Continuous target variable
- Explanatory features

31.5.1 Simple Regression

- Uses one independent variable
- Can be linear or nonlinear

31.5.2 Multiple Regression

- Uses two or more independent variables
- Extension of simple linear regression

31.6 Logistic Regression

In logistic regression training:

- The goal is to find the best parameters mapping inputs to outputs
- Used for predicting classes with minimal error

31.7 Classification in Machine Learning

Classification is a supervised learning method that:

- Predicts labels for new data

- Works with categorical (discrete) outputs

31.7.1 Applications of Classification

- Email filtering
- Speech-to-text
- Handwriting recognition
- Biometric identification
- Document classification

31.8 Key Machine Learning Algorithms

31.8.1 K-Nearest Neighbors (KNN)

KNN is a supervised algorithm that:

- Uses labeled data points
- Labels new data based on similarity
- Used for both classification and regression

31.8.2 Support Vector Machines (SVM)

SVM is a supervised learning method that:

- Represents data in multidimensional space
- Used for classification and regression

Applications:

- Speech recognition
- Anomaly detection
- Noise filtering

31.8.3 Decision Trees

A decision tree:

- Classifies data using a tree structure
- Each node represents a test
- Each branch represents an outcome
- Leaf nodes represent final classes

31.8.4 Regression Trees

Regression trees:

- Predict continuous values
- Split the dataset recursively
- Aim to maximize information gain

- Reduce randomness in splits

31.9 Clustering and Dimensionality Reduction

31.9.1 Clustering

Clustering groups data based on similarity:

- Identifies hidden patterns
- Used in:
 - Music genre classification
 - User segmentation
 - Market analysis

31.9.2 Dimensionality Reduction

Dimensionality reduction:

- Reduces number of features
- Preserves important information
- Helps in visualization of high-dimensional data
- Improves model performance

31.10 Model Evaluation

31.10.1 Reduced Learning Evaluation

Measures how well a model predicts unseen data by comparing:

- Predictions vs ground truth

31.10.2 Classification Metrics

- Accuracy
- Confusion Matrix
- Precision
- Recall

31.10.3 Regression Evaluation

Regression models are evaluated based on:

- Accuracy of continuous predictions (e.g., exam scores)

31.10.4 Unsupervised Evaluation

Used for:

- Clustering quality
- Pattern discovery

- Grouping effectiveness

31.11 Model Validation

Model validation helps:

- Optimize ML models
- Prevent overfitting
- Improve generalization to unseen data

31.11.1 Important Concept: Data Leakage

- **Data snooping** occurs when test data is used during training or optimization
- This leads to incorrect model evaluation

Correct approach:

- Train on training data
- Test only on unseen test data